



Trabajo de Fin de Grado en Ingeniería del Software, Ingeniería de computadores
Facultad de Informática, Universidad Complutense de Madrid

Detección explicable de voz generada con IA

Jorge Aured Zarzoso
Germán Bravo Quintián
Rafael López Rodríguez
Iván Pastor Sacristán

Directores:
Marta Caro Martínez
Antonio Fernando García Sevilla

Curso académico 2025/2026

Resumen

TODO resumen en español

Palabras clave: TODO

Abstract

TODO abstract en ingles

Keywords: TODO

Índice general

Resumen	I
Abstract	II
1 Introducción	1
1.1 Motivación y Contexto	1
1.2 Definición del Problema	1
1.3 Objetivos	2
1.4 Planificación	2
2 Estado del arte	4
2.1 Modelos de clasificación	4
2.1.1 Perceptrón multicapa - MLP	4
2.1.2 Wav2Vec2	5
2.1.3 Random Forest	6
2.1.4 LightGBM	8
2.1.5 Support Vector Machines (SVM)	13
2.1.6 AASIST	16
2.1.7 Regresión Logística	18
2.1.8 Regresión Ridge	19
2.1.9 K-Nearest Neighbors (k-NN)	20
2.2 Modelos de explicación	22
2.2.1 SHAP	23
2.2.2 LIME	24
2.2.3 Grad-CAM	25
2.2.4 Integrated Gradients	26
2.2.5 DICE	27
2.2.6 SincNet	27
2.2.7 ProtoPNet	28

2.2.8	Occlusion	29
2.2.9	RISE	29
2.3	Conclusiones	30
3	Metodología	31
3.1	Dataset	31
3.2	Extracción de características	33
3.3	Métricas de evaluación	37
3.4	Proceso común de experimentación	40
3.4.1	División de los datos	40
3.4.2	Validación Cruzada y Grid Search	41
3.4.3	Funciones de explicación	41
3.4.4	Estado aleatorio	42
3.5	Modelos	42
3.5.1	Random Forest	43
3.5.2	LightGBM	44
3.5.3	Support Vector Machine (SVM)	46
3.5.4	AASIST	48
3.5.5	Perceptrón multicapa	56
3.5.6	Wav2Vec2	60
3.5.7	Regresión Logística	64
3.5.8	KNN	67
3.6	Resultados	70
3.6.1	Modelos de datos tabulares	70
3.6.2	Random Forest	73
3.6.3	LightGBM	82
3.6.4	Support Vector Machine	91
3.6.5	AASIST	95
3.6.6	Perceptrón	104
3.6.7	Wav2Vec2	105
3.6.8	Regresión Logística	119
3.6.9	KNN	119
3.7	Discusión de los resultados	119
3.8	Conclusiones	119

4 Conclusiones	120
4.1 Trabajo futuro	120
5 Pipeline	122
5.1 AASIST	124
5.2 Wav2Vec2	124
Contribución de Iván Pastor Sacristán	126
Contribución de Jorge Aured Zarzoso	128
Contribución de Germán Bravo Quintián	130
Contribución de Rafael López Rodríguez	133
Anexo	135

Índice de figuras

2.1	Voto por mayoría en el “bosque”.	9
3.1	Ejemplo real de matriz de confusión.	39
3.2	Explicación local de Random Forest con SHAP	74
3.3	Explicación local de Random Forest con SHAP	75
3.4	Explicación global de Random Forest con SHAP	76
3.5	Explicación local de Random Forest con SHAP tras quitar spectral flatness	77
3.6	Explicación local de Random Forest con SHAP tras quitar spectral flatness	78
3.7	Explicación local de Random Forest con SHAP tras quitar spectral flatness, mfcc 1 y spectral bandwidth	80
3.8	Explicación local de Random Forest con SHAP tras quitar spectral flatness, mfcc 1 y spectral bandwidth	81
3.9	Explicación local de Random Forest con LIME	81
3.10	Explicación local de LightGBM con SHAP	83
3.11	Explicación local de LightGBM con SHAP	84
3.12	Explicación global de LightGBM con SHAP	85
3.13	Explicación local de LightGBM con SHAP tras quitar spectral flatness	86
3.14	Explicación local de LightGBM con SHAP tras quitar spectral flatness	87
3.15	Explicación local de LightGBM con SHAP tras quitar spectral flatness y mfcc 1	88
3.16	Explicación local de LightGBM con SHAP tras quitar spectral flatness y mfcc 1	89
3.17	Explicación local de LightGBM con LIME	90
3.18	Explicación local de SVM con SHAP	92
3.19	Explicación local de SVM con SHAP	93
3.20	Explicación global de SVM con SHAP	94
3.21	Explicación local de SVM con SHAP tras quitar spectral flatness	95
3.22	Explicación local de SVM con SHAP tras quitar spectral flatness	96
3.23	Explicación local de SVM con SHAP tras quitar spectral flatness, mfcc 1 y spectral bandwidth	98

3.24	Explicación local de SVM con SHAP tras quitar spectral flatness, mfcc 1 y spectral bandwidth	99
3.25	Explicación local de SVM con LIME	99
3.26	Resultados del preentrenamiento en AASIST.	100
3.27	Shap global energía.	107
3.28	Ejemplo concreto.	108
3.29	Explicación local energía Lime.	109
3.30	Ranking características Shap.	110
3.31	Ranking características Lime.	111
3.32	Ranking agregado de características.	112
3.33	shap local espectrogramas Wav2Vec2	113
3.34	Shap general de los espectrogramas	114
3.35	lime local espectrogramas Wav2Vec2	115
3.36	Lime general de los espectrogramas	115
3.37	IG local espectrogramas Wav2Vec2	116
3.38	Integrated gradients general de los espectrogramas	116
3.39	Grad Cam local espectrogramas Wav2Vec2	117
3.40	Grad cam general de los espectrogramas	118
3.41	Influencia general de los espectrogramas	118
5.1	Arquitectura del pipeline.	123
1	Resultados de “logit” de la ocultación temporal en AASIST.	138
2	Resultados de margen de la ocultación temporal en AASIST.	138
3	Resultados de “logit” de la ocultación frecuencial en AASIST.	139
4	Resultados de margen de la ocultación frecuencial en AASIST.	139
5	Mapas filtro-tiempo de la capa sinc-convolution generados con IG.	140
6	Mapas filtro-tiempo de la capa sinc-convolution generados con IG normalizados.	140
7	Gráfica filtro-tiempo de la capa sinc-convolution.	141
8	Importancia temporal al aplicar IG sobre onda de audio cruda.	141
9	Resultados de RISE.	142
10	Gráficas de importancia global de SHAP para el clasificador y el regresor.	142
11	Influencia de características espectrales en Wav2Vec2	142
12	Influencia de características espectrales en Wav2Vec2	143
13	Influencia de características espectrales en Wav2Vec2	143
14	Influencia de características espectrales en Wav2Vec2	143

15	Explicación local espectrales Lime.	144
16	Explicación loca pitch Lime.	144
17	Explicación local formantes Lime.	144
18	Explicación local mfccs Lime.	144

Índice de tablas

3.1	Métricas principales de los modelos baseline de datos tabulares	70
3.2	Classification report de los modelos baseline de datos tabulares	71
3.3	Análisis de errores de los modelos baseline de datos tabulares	71
3.4	Resultados de Validación Cruzada con 5 folds de los modelos baseline de datos tabulares	71
3.5	Mejores hiperparámetros para Random Forest con Grid Search y CV de 5 folds	72
3.6	Mejores hiperparámetros para LightGBM con Grid Search y CV de 5 folds . .	72
3.7	Mejores hiperparámetros para SVM con Grid Search y CV de 5 folds	73
3.8	Métricas principales de Random Forest sin spectral flatness	74
3.9	Classification report de Random Forest sin spectral flatness	75
3.10	Análisis de errores de Random Forest sin spectral flatness	77
3.11	Métricas principales de Random Forest sin spectral flatness, mfcc1 y spectral bandwidth	79
3.12	Classification report de Random Forest sin spectral flatness, mfcc1 y spectral bandwidth	79
3.13	Análisis de errores de Random Forest sin spectral flatness, mfcc1, y spectral bandwidth	80
3.14	Counter factuels generados por DiCE para Random Forest	82
3.15	Métricas principales de LightGBM sin spectral flatness	82
3.16	Classification report de LightGBM sin spectral flatness	83
3.17	Análisis de errores de LightGBM sin spectral flatness	84
3.18	Métricas principales de LightGBM sin spectral flatness y mfcc1	89
3.19	Classification report de LightGBM sin spectral flatness y mfcc 1	90
3.20	Análisis de errores de LightGBM sin spectral flatness y mfcc1	90
3.21	Counter factuels generados por DiCE para LightGBM	91
3.22	Métricas principales de SVM sin spectral flatness	91
3.23	Classification report de SVM sin spectral flatness	92
3.24	Análisis de errores de SVM sin spectral flatness	93

3.25	Métricas principales de SVM sin spectral flatness, mfcc1 y spectral bandwidth .	96
3.26	Classification report de SVM sin spectral flatness, mfcc1 y spectral bandwidth	97
3.27	Análisis de errores de SVM sin spectral flatness, mfcc1 y spectral bandwidth .	97
1	Valores e interpretación de características acústicas del audio I_12_N	135

1. Introducción

1.1. Motivación y Contexto

En los últimos años, la sociedad ha sido testigo de un aumento considerable en el uso de la Inteligencia Artificial (IA) para suplantar la identidad de personas de forma maliciosa mediante la clonación de voz, técnica conocida como *deepfake*. Esta evolución tecnológica ha alcanzado un punto crítico donde la distinción entre un audio humano y uno sintético es prácticamente imperceptible para el oído humano. Como se ve en **1Barrington2025**; **2Nadine2025**, estudios recientes indican que, mientras la detección de audios 100 % generados resulta sencilla para un usuario promedio, la precisión al clasificar voces reales frente a clones de alta calidad apenas ronda el 60 %.

Esta vulnerabilidad supone una amenaza latente en sectores estratégicos como la ciberseguridad, el periodismo y el ámbito judicial. Por otro lado, las técnicas de detección de *deepfakes* no han avanzado al mismo ritmo que las de generación, lo que se traduce en una proliferación de estafas y desinformación. Por este motivo, ya no es posible confiar únicamente en las capacidades naturales del ser humano para diferenciar la autenticidad de una voz, lo que convierte el desarrollo de herramientas tecnológicas robustas en una necesidad social imperativa.

1.2. Definición del Problema

El rendimiento del modelo de clasificación no es el único factor determinante en esta investigación. En la actualidad, los modelos de aprendizaje profundo se han convertido en “cajas negras”, sistemas opacos donde no se comprende cómo se generan las predicciones. Esta falta de transparencia da lugar a problemas de confianza respecto a dichas decisiones, especialmente en ámbitos críticos como la medicina o la ingeniería, donde la vida de las personas puede estar en juego.

Para solventar este problema ha surgido la IA explicable (XAI), la cual toma dichos modelos opacos y proporciona una justificación de qué características de los datos afectan más a la

decisión del modelo. En esta investigación, abordamos el problema dual de maximizar la capacidad de discriminación entre audios reales y generados y, simultáneamente, dotar al sistema de una interpretación humana coherente mediante la aplicación de XAI.

1.3. Objetivos

El objetivo principal de este trabajo es desarrollar un marco de trabajo para la detección de audio generado sintéticamente que priorice no solo la eficacia en la clasificación, sino también la transparencia de sus resultados. Para ello, se plantean los siguientes objetivos específicos:

- **Exploración de Modelos de Clasificación:** Evaluar la viabilidad de distintos enfoques algorítmicos, desde métodos estadísticos tradicionales hasta arquitecturas de aprendizaje profundo de última generación, para identificar cuál se adapta mejor a la naturaleza del audio clonado.
- **Estudio de Características Acústicas:** Investigar qué rasgos del habla (frecuencia, timbre, periodicidad o pausas) contienen la información más relevante para distinguir la huella de una IA frente a la de un humano.
- **Integración de Métodos de Explicabilidad:** Proponer el uso de herramientas de interpretación que permitan visualizar qué partes del audio están influyendo en la decisión del sistema, convirtiendo el modelo en una herramienta útil para un análisis humano posterior.

1.4. Planificación

Para garantizar la consecución de los objetivos propuestos, el proyecto se ha estructurado siguiendo un flujo de trabajo sistemático. La metodología planificada se divide en las siguientes fases consecutivas:

1. **Investigación de modelos y técnicas:** Estudio exhaustivo del estado del arte para identificar las arquitecturas de clasificación y los métodos de explicabilidad más prometedores en el dominio del procesamiento de voz y la detección de anomalías.
2. **Adquisición y preparación de los datos:** Selección y procesamiento de un conjunto de datos equilibrado que contenga muestras de audio tanto reales como sintéticas, asegurando la correcta extracción de características y la adecuación de los formatos para el entrenamiento.

3. **Prueba de los modelos y técnicas:** Fase de experimentación preliminar donde se evalúa el comportamiento individual de distintos algoritmos, ajustando parámetros básicos y analizando su capacidad inicial para discriminar entre las clases propuestas.
4. **Implementación del pipeline completo:** Integración de los componentes de pre-procesamiento, clasificación y explicabilidad en un sistema unificado, permitiendo una transición fluida desde la entrada del audio en crudo hasta la generación de un veredicto justificado.
5. **Análisis de los resultados:** Evaluación crítica del sistema mediante métricas de rendimiento y auditorías de interpretabilidad, comparando los hallazgos obtenidos con las hipótesis iniciales para extraer conclusiones sólidas sobre la viabilidad del enfoque propuesto.

2. Estado del arte

En este capítulo, se presenta el marco teórico que sirve de base para el desarrollo del proyecto. En primer lugar, en la sección de [Modelos de clasificación](#), se contemplan los distintos modelos seleccionados para la tarea de clasificación de los audios. En este bloque, se explica teóricamente tanto su arquitectura como las principales innovaciones, propiedades o ventajas específicas que consideramos relevantes para la detección de audio falsificado.

Asimismo, también se contempla una sección de [Modelos de explicación](#), donde se exponen los fundamentos teóricos de los distintos modelos de XAI que consideramos de interés para este trabajo. Para la elección de los modelos de clasificación, se ha buscado diversidad en diversos aspectos, como el alcance de la explicación, el grado de adaptabilidad a diferentes modelos o el momento del proceso de aprendizaje donde se incorpora la explicabilidad.

En conjunto, esta sección permite identificar las herramientas más importantes del proyecto y fundamentar las decisiones metodológicas desarrolladas en capítulos posteriores.

2.1. Modelos de clasificación

La detección de audio deepfake se fundamenta en la capacidad de los algoritmos para discriminar entre patrones biométricos naturales y artefactos generados sintéticamente. Dado que los métodos de falsificación varían en complejidad desde técnicas de concatenación simple hasta modelos de difusión avanzados, es necesario abordar el problema mediante diversos enfoques de aprendizaje automático.

A continuación se describen los modelos de clasificación que se usan actualmente para este estudio de audios fake, abarcando desde algoritmos estadísticos clásicos y modelos basados en árboles, hasta arquitecturas de aprendizaje profundo más complejas.

2.1.1. Perceptrón multicapa - MLP

Un perceptrón multicapa es una red neuronal que se utiliza en aprendizaje supervisado en el campo del aprendizaje automático. El MLP está compuesto por múltiples capas de neuronas

interconectadas, en las que las salidas de las neuronas de una capa se convierten en entradas para la siguiente capa. La primera capa se llama capa de entrada, la última capa se llama capa de salida y las capas intermedias se llaman capas ocultas. El MLP presenta una gran capacidad para modelar relaciones no lineales y para generalizar. Como punto negativo del uso de este modelo, podemos remarcar la necesidad de un gran conjunto de datos de entrenamiento para evitar el sobreajuste. Por último, cabe resaltar que los MLP's fueron los precursores de otras redes neuronales más complejas como la redes convolucionales y las redes recurrentes.

Fuente: **mlp_gamco**.

2.1.2. Wav2Vec2

Wav2Vec2 es un modelo de aprendizaje profundo end-to-end (el modelo procesa todo el flujo de datos, desde la entrada cruda hasta el resultado final) para procesamiento de voz que permite obtener representaciones acústicas de alta calidad directamente desde la señal cruda de audio, sin necesidad de las características del audio. Fue propuesto por Meta en 2020.

Como parte del entrenamiento, el modelo aprende unidades de habla discreta mediante un gumbel softmax (técnica utilizada en redes neuronales para aproximar el muestreo de distribuciones categóricas discretas de forma diferenciable) para representar las representaciones latentes en la tarea contrastiva.

Tras el preentrenamiento en el habla no etiquetada, el modelo se ajusta con datos etiquetados con una pérdida de Clasificación Temporal Conexionista (CTC*) para ser utilizada en tareas posteriores de reconocimiento de voz.

*CTC es un algoritmo y función de pérdida utilizado en redes neuronales profundas para entrenar modelos de aprendizaje automático en tareas de modelado de secuencias cuando los datos de entrada y salida no están alineados

El modelo se compone de un codificador de características convolucional multicapa que toma la entrada cruda de audio y devuelve representaciones latentes de voz para T instantes de tiempo. Luego son pasados a un Transformer* para construir representaciones capturando información de toda la secuencia.

*Transformer es un modelo de aprendizaje automático usado en procesamiento de lenguaje natural diseñado para entender y generar secuencias de forma eficiente.

El codificador de características consiste en muchos bloques convolucionales seguidos de una capa de normalización y una función de activación GELU (Gaussian Error Linear Unit).

GELU pondera las entradas por su probabilidad bajo una distribución gaussiana suavizando su activación cerca de 0.

También realiza representaciones contextualizadas con Transformers siendo que la salida del codificador se pasa a una red que sigue la arquitectura de un Transformer, y usa una capa convolucional que actúa como un embedding posicional relativo.

Por último, Wav2Vec2 consta de un módulo de cuantización que discretiza la salida del codificador a un conjunto finito de representaciones del habla. Esta cuantización se hace con la técnica de Gumbel Softmax.

El entrenamiento del modelo se basa en un preentrenamiento con enmascaramiento en donde se toma la señal de audio y se transforma en representaciones latentes mediante un encoder. Esto se hace por bloques consecutivos de tamaño M que pueden solaparse.

Para cada paso de tiempo enmascarado, el modelo debe identificar la representación latente cuantizada correcta. Para ello, usa una pérdida contrastiva basada en la similitud del coseno.

También se realiza un fine-tuning supervisado para reconocimiento automático del habla usando CTC como función de pérdida y aplicando una versión de SpecAugment (técnica de aumento de datos diseñada específicamente para el reconocimiento automático de voz) para evitar sobreajuste.

Fuente: **2020wav2vec**.

2.1.3. Random Forest

Random Forest es un algoritmo de aprendizaje automático basado en la combinación de múltiples árboles de decisión, propuesto por Leo Breiman en 2001 **2001randomforest**. Pertenecen a la familia de métodos *ensemble*, que mejoran el rendimiento agregando las predicciones de varios modelos más simples.

Arquitectura y funcionamiento

El algoritmo construye un “bosque” de árboles de decisión donde cada árbol se entrena de forma ligeramente diferente, introduciendo aleatoriedad controlada en dos niveles:

- **Bootstrap aggregating (bagging):** Cada árbol se entrena con una muestra aleatoria del conjunto de datos original, permitiendo repeticiones. Esto significa que algunos ejemplos aparecen varias veces en el entrenamiento de un árbol, mientras que otros (alrededor de un tercio) quedan fuera (*out-of-bag*) y sirven para validación interna.
- **Selección aleatoria de características:** En cada división de un nodo del árbol, solo se considera un subconjunto aleatorio de todas las características disponibles. Esto evita que unas pocas características muy fuertes dominen todas las decisiones.

Proceso de clasificación

Cuando llega una nueva muestra de audio con sus características (formantes, MFCC, etc.), cada árbol del bosque realiza su propia clasificación. La predicción final se decide por mayoría:

- Si la mayoría de los árboles clasifican el audio como deepfake, la predicción final es deepfake.
- Si la mayoría lo clasifica como real, la predicción final es real.

Este mecanismo de votación reduce drásticamente la varianza del modelo: aunque un árbol individual pueda cometer errores, el consenso del bosque suele ser acertado. Esto se puede ver en la siguiente imagen [2.1](#)

Hiperparámetros

Vamos a hablar de los hiperparámetros que tiene Random Forest y para qué sirven `sklearn_random_forest` ya que conocerlos es crucial a la hora de usar un modelo.

- **n_estimators:** Define la cantidad de árboles de decisión que conforman el bosque aleatorio. A medida que aumentamos este número, el modelo tiende a mejorar su rendimiento porque se promedian más predicciones individuales, lo que reduce la varianza y el riesgo de sobreajuste. Sin embargo, a partir de cierto punto (generalmente entre 100 y 500 árboles) los beneficios se vuelven marginales, mientras que el tiempo de entrenamiento y predicción sigue incrementándose linealmente, por lo que es recomendable encontrar un equilibrio entre rendimiento y eficiencia computacional.
- **max_depth:** Controla la profundidad máxima que pueden alcanzar los árboles individuales dentro del bosque. Si no se establece un límite (None), los árboles crecerán hasta que todas las hojas sean puras o contengan el mínimo de muestras permitido, lo que puede llevar a un sobreajuste en datos de entrenamiento. Por el contrario, valores pequeños limitan la complejidad del modelo, actuando como una forma de regularización que previene el sobreajuste pero podría aumentar el sesgo si la profundidad es demasiado restrictiva.
- **min_samples_split:** Determina el número mínimo de muestras requeridas para dividir un nodo interno durante la construcción del árbol. Valores más altos hacen que el árbol sea más conservador al crear nuevas ramas, ya que necesita más evidencia (muestras) para justificar una división. Esto ayuda a prevenir el sobreajuste al evitar que el modelo

aprenda patrones demasiado específicos de grupos pequeños de datos, aunque valores excesivamente altos podrían impedir que el modelo capture patrones importantes.

- **max_features:** Especifica cuántas características debe considerar cada árbol al buscar la mejor división en cada nodo. Al limitar las características disponibles, se fuerza a los árboles a ser más diversos entre sí, lo que reduce la correlación entre ellos y mejora la capacidad de generalización del conjunto. Para clasificación, el valor recomendado suele ser la raíz cuadrada del número total de características ($\sqrt{n}$), mientras que para regresión se suele usar un tercio del total. Valores más pequeños aumentan la aleatoriedad y la diversidad, mientras que valores más grandes hacen que los árboles sean más similares entre sí.
- **max_samples:** Cuando se utiliza muestreo bootstrap (que es el comportamiento por defecto), este parámetro controla el tamaño de la muestra aleatoria que se toma del conjunto de datos original para entrenar cada árbol. Valores inferiores a 1.0 (por ejemplo, 0.7) significan que cada árbol se entrena solo con el 70 % de los datos, lo que aumenta la diversidad entre los árboles y fortalece la capacidad de generalización del bosque. Este parámetro es especialmente útil cuando se trabaja con conjuntos de datos muy grandes o cuando se sospecha que los árboles individuales están demasiado correlacionados.

Ventajas específicas para detección de deepfakes

- **Robustez ante ruido:** Las características de audio pueden contener variaciones por condiciones de grabación; Random Forest maneja bien estos datos ruidosos.
- **No requiere normalización:** A diferencia de redes neuronales o SVM, no es necesario escalar las características, lo que simplifica el pipeline.
- **Resistencia al sobreajuste:** La combinación de bagging y aleatoriedad evita que el modelo memorice el conjunto de entrenamiento.

2.1.4. LightGBM

LightGBM (*Light Gradient Boosting Machine*) es un framework de gradient boosting desarrollado por Microsoft en 2017 **2017lightgbm**, que optimiza el proceso de entrenamiento mediante innovaciones en la forma de construir los árboles y muestrear los datos. Mientras que Random Forest construye árboles de forma independiente y paralela, LightGBM los construye secuencialmente, donde cada nuevo árbol se especializa en corregir los errores de los anteriores.

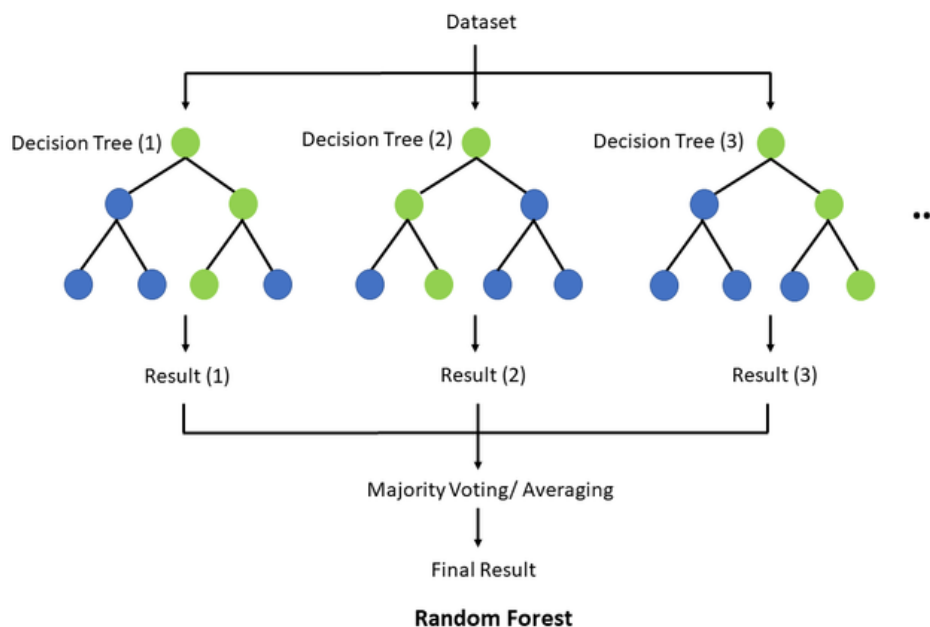


Figura 2.1: Voto por mayoría en el “bosque”.

Arquitectura y funcionamiento

El proceso de boosting funciona de la siguiente manera:

1. Se entrena un primer árbol simple que intenta clasificar las instancias según las etiquetas.
2. Se identifican las que este primer árbol clasificó incorrectamente.
3. Se entrena un segundo árbol enfocado específicamente en esos casos difíciles.
4. Se combinan ambos árboles para mejorar la precisión global.
5. El proceso se repite construyendo decenas o cientos de árboles, cada uno corrigiendo los residuos de los anteriores.

Innovaciones clave

- **Crecimiento por hoja (*leaf-wise*):** Los árboles tradicionales crecen nivel por nivel (balanceados). LightGBM, en cambio, crece expandiendo la hoja que más puede reducir el error en cada paso, lo que produce árboles más profundos en las regiones problemáticas y más eficientes en las fáciles.
- **Muestreo basado en gradientes (*GOSS, Gradient-based One-Side Sampling*):** Durante el entrenamiento, LightGBM identifica qué instancias tienen mayor error y las

prioriza. Las instancias con error pequeño se muestrean aleatoriamente para reducir el costo computacional. Esto permite entrenar con datasets grandes sin procesar toda la información redundante.

- **Agrupación de características (*EFB*, *Exclusive Feature Bundling*):** LightGBM agrupa las características que no suelen tomar valores iguales a 0 para reducir la dimensionalidad, acelerando el entrenamiento sin pérdida de información significativa.

Proceso de clasificación

Cuando una nueva instancia llega para ser clasificada:

- Pasa secuencialmente por todos los árboles construidos.
- Cada árbol aporta una corrección o ajuste a la predicción anterior.
- La suma ponderada de todas las contribuciones produce la clasificación final (probabilidad de pertenecer a una clase).

Hiperparámetros

Vamos a explicar los hiperparámetros más conocidos en los modelos LightGBM `sklearn_lightgbm`.

- **max_depth:** Limita explícitamente la profundidad máxima que pueden alcanzar los árboles durante el entrenamiento. Es especialmente importante cuando se trabaja con conjuntos de datos pequeños, ya que los árboles poco profundos tienen menos probabilidad de sufrir sobreajuste. Si max_depth es menor o igual a cero, significa que no hay límite para la profundidad máxima, lo que puede ser arriesgado con datos limitados.
- **num_leaves:** Controla la complejidad del modelo en LightGBM, ya que determina el número máximo de hojas por árbol. A diferencia de otros algoritmos que crecen por niveles, LightGBM utiliza un crecimiento por hojas (leaf-wise), lo que significa que elige la hoja con la mayor pérdida delta para crecer, permitiendo una convergencia más rápida y mayor precisión. Sin embargo, este enfoque puede provocar sobreajuste si no se configura adecuadamente. En la práctica, el valor de num_leaves debe ser menor que 2 elevado a la max_depth para evitar que los árboles crezcan demasiado profundo. Por ejemplo, si max_depth=7, establecer num_leaves en 70 u 80 suele dar mejor precisión que usar 127.
- **min_data_in_leaf:** Define la cantidad mínima de observaciones que debe contener una hoja para que sea creada. Configurarla con un valor grande (cientos o miles para datasets grandes) evita que los árboles crezcan demasiado profundo y aprendan patrones demasiado específicos de grupos muy pequeños de datos. Sin embargo, si el valor es

excesivamente alto, puede provocar subajuste al impedir que el modelo capture patrones importantes. Su valor óptimo depende del número de muestras de entrenamiento y de `num_leaves`.

- **learning_rate**: También conocido como `shrinkage_rate` o η , este parámetro controla la velocidad a la que se actualizan los pesos del modelo después de cada iteración de boosting. Valores pequeños (por ejemplo, 0.01 o 0.05) hacen que el modelo aprenda más lentamente pero generalmente logran mejor precisión, aunque requieren más iteraciones (`num_iterations`) para converger. Por el contrario, valores altos (cerca de 1.0) aceleran el aprendizaje pero aumentan el riesgo de sobreajuste y pueden hacer que el modelo oscile alrededor del óptimo sin alcanzarlo. La práctica común es usar `learning_rate` bajos junto con un mayor número de iteraciones.
- **n_estimators**: Determina el número máximo de iteraciones de boosting que se realizarán, lo que equivale al número de árboles que se construirán. Para problemas de clasificación multiclase, LightGBM construye internamente `num_class * num_iterations` árboles. El valor predeterminado suele ser 100, pero puede aumentarse para mejorar el rendimiento, siempre ajustando también la tasa de aprendizaje. Una estrategia inteligente es combinarlo con `early_stopping_rounds`, que detiene el entrenamiento si la métrica de validación no mejora durante varias rondas consecutivas, ahorrando tiempo computacional y previniendo sobreajuste.
- **feature_fraction**: Especifica la fracción de características que se seleccionarán aleatoriamente para construir cada árbol. Por ejemplo, `feature_fraction=0.8` significa que cada árbol utilizará aleatoriamente el 80 % de las variables disponibles. Esto aumenta la diversidad entre los árboles, reduce la correlación entre ellos y ayuda a prevenir el sobreajuste, además de acelerar el entrenamiento al considerar menos splits posibles. Valores típicos oscilan entre 0.5 y 0.9, siendo 1.0 el valor por defecto que significa que se usan todas las características.
- **bagging_fraction**: Similar a `feature_fraction`, pero aplicado al muestreo de datos en lugar de características. Este parámetro selecciona aleatoriamente una fracción de los datos sin reemplazo para entrenar cada árbol, lo que también se conoce como bagging, proceso que se ha explicado previamente en la sección de Random Forest. Para que sea efectivo, debe acompañarse de `bagging_freq`, que especifica cada cuántas iteraciones se realiza un nuevo muestreo. Por ejemplo, con `bagging_fraction=0.7` y `bagging_freq=5`, cada 5 iteraciones se toma una muestra aleatoria del 70 % de los datos. Esta técnica reduce el tiempo de entrenamiento y ayuda a combatir el sobreajuste.

- **reg_alpha y reg_lambda:** Estos parámetros aplican regularización L1 (lasso) y L2 (ridge) respectivamente para controlar la complejidad del modelo y prevenir el sobreajuste. La regularización añade una penalización a la función de pérdida por tener valores de parámetros grandes, lo que favorece modelos más simples y generalizables. Por defecto valen 0, lo que significa que no se aplica regularización. Aumentar estos valores puede ser especialmente útil cuando se sospecha de sobreajuste o cuando se trabaja con muchas características redundantes.
- **objective:** Define la tarea de aprendizaje que debe realizar el modelo y la función de pérdida que se optimizará durante el entrenamiento. Es uno de los parámetros más importantes porque determina fundamentalmente cómo se evalúa el error y cómo se actualizan los pesos. Para problemas de regresión, los valores más comunes son “regression” (error cuadrático medio L2), “regression_l1” (error absoluto L1, más robusto ante outliers) y ‘huber’ (combinación híbrida de L1 y L2 para datos con ruido). Para clasificación binaria se usa “binary” (pérdida logística), para clasificación multiclase se usa “multiclass” y para ranking se usa “lamdarank”. En la práctica, elegir el objective correcto según el problema es el primer paso antes de ajustar cualquier otro parámetro.
- **boosting_type:** Determina el algoritmo específico que LightGBM utilizará para construir los árboles secuencialmente. Las opciones principales son: “gbdt” (Gradient Boosting Decision Tree), que es el método tradicional y el valor por defecto, donde cada árbol corrige los errores del anterior; “dart” (Dropouts meet Multiple Additive Regression Trees), que utiliza un enfoque inspirado en dropout de redes neuronales, apagando aleatoriamente algunos árboles durante el entrenamiento para reducir el sobreajuste; “goss” (Gradient-based One-Side Sampling), que acelera el entrenamiento muestreando selectivamente las instancias con gradientes grandes; y “rf” (Random Forest), que cambia el comportamiento a un verdadero bosque aleatorio donde los árboles se construyen independientemente sin boosting. La elección afecta tanto la velocidad de entrenamiento como la precisión y el riesgo de sobreajuste.

Ventajas específicas para detección de deepfakes

- **Alta precisión:** El enfoque secuencial de corrección de errores suele superar a Random Forest en problemas complejos como la detección de deepfakes.
- **Eficiencia computacional:** El muestreo inteligente permite entrenar con grandes volúmenes de audios en menos tiempo que otros métodos de boosting.

- **Manejo de desequilibrios:** Si existen desbalances entre clases, LightGBM incorpora mecanismos para ponderar las muestras adecuadamente.
- **Early stopping:** Se puede configurar para que detenga el entrenamiento cuando añadir más árboles no mejora la validación, evitando overfitting.

2.1.5. Support Vector Machines (SVM)

Las Máquinas de Vectores de Soporte (*Support Vector Machines*, SVM) son un conjunto de algoritmos de aprendizaje supervisado desarrollados por Vladimir Vapnik y su equipo en AT&T Bell Laboratories durante la década de 1990 **1995svm**. Originalmente diseñadas para clasificación binaria, las SVM se han convertido en uno de los métodos más robustos y teóricamente fundamentados del aprendizaje automático.

Fundamento conceptual: El hiperplano óptimo

El objetivo fundamental de una SVM es encontrar un hiperplano que separe las dos clases (en nuestro caso, voces reales y voces generadas por IA) con el máximo margen posible. Para comprender este concepto, es útil visualizar un problema simple en dos dimensiones:

- Imaginemos que representamos cada audio como un punto en un plano, donde los ejes son dos características (por ejemplo, el primer formante y el primer MFCC).
- Las voces reales se agrupan en una región del plano y las generadas por IA en otra.
- Un hiperplano en dos dimensiones es simplemente una línea recta que intenta separar ambos grupos.

La clave de SVM reside en que no cualquier línea divisoria es válida: el algoritmo busca aquella que maximiza la distancia (margen) entre la línea y los puntos más cercanos de cada clase. Estos puntos críticos, que “sostienen” el margen, reciben el nombre de **vectores de soporte** (*support vectors*), de donde el algoritmo toma su nombre.

Funcionamiento del algoritmo

El proceso de entrenamiento de una SVM puede entenderse en los siguientes pasos:

1. **Identificación de vectores de soporte:** El algoritmo examina todos los puntos de entrenamiento (instancias) e identifica aquellas que son más difíciles de clasificar, es decir, las que están más cerca de la frontera entre clases.

2. **Maximización del margen:** A partir de estos vectores de soporte, la SVM calcula el hiperplano que maximiza la distancia a ambos lados. Este margen máximo proporciona la mejor capacidad de generalización: cuanto más ancho sea el margen, menor será la probabilidad de error al clasificar nuevas instancias.
3. **Clasificación de nuevas muestras:** Una vez entrenado el modelo, para clasificar una nueva instancia, simplemente se calcula a qué lado del hiperplano se sitúa.

El truco del kernel: Separando lo inseparable

Uno de los mayores desafíos en la clasificación es que las instancias rara vez son linealmente separables en el espacio original de características. Es decir, no existe una línea recta (o hiperplano) que pueda separarlas perfectamente. Para abordar esta limitación, las SVM introducen el concepto de **kernel** o núcleo.

El *truco del kernel* (*kernel trick*) funciona de la siguiente manera conceptual:

- Los datos originales (no separables linealmente) se transforman implícitamente a un espacio de mayor dimensionalidad mediante una función matemática.
- En este nuevo espacio, es posible que los datos sí sean linealmente separables.
- La ventaja del kernel es que realiza esta transformación sin necesidad de calcular explícitamente las coordenadas en el espacio de alta dimensión, lo que sería computacionalmente costoso.

Los kernels más comúnmente utilizados son:

- **Kernel lineal:** Asume que los datos son aproximadamente separables por un hiperplano. Útil cuando el número de características es muy grande.
- **Kernel polinomial:** Crea fronteras de decisión curvas mediante combinaciones polinómicas de las características originales.
- **Kernel RBF (*Radial Basis Function*):** Es el más utilizado en muchos problemas. Proyecta los datos a un espacio de dimensionalidad infinita, permitiendo fronteras de decisión altamente complejas y no lineales. Se basa en la similitud entre muestras medida por distancia euclidiana:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

donde γ controla la influencia de cada muestra: un valor pequeño considera vecindarios amplios (frontera más suave), mientras que un valor grande se enfoca en vecindarios reducidos (frontera más ajustada a los datos).

Hiperparámetros

Vamos a ver cuáles son los hiperparámetros más importantes en los SVM `sklearn_svm`.

- **C**: Controla el equilibrio entre tener un margen amplio y clasificar correctamente los puntos de entrenamiento. Un valor bajo de C busca un margen grande aunque se cometan algunos errores (modelo más generalista). Un valor alto de C penaliza fuertemente los errores, ajustándose más a los datos de entrenamiento (riesgo de overfitting).
- **kernel**: Es la función que transforma los datos a un espacio de mayor dimensión donde puedan ser separables linealmente. La elección del kernel determina fundamentalmente la capacidad del modelo para encontrar patrones complejos.
- **gamma** γ : Específico de kernels no lineales (RBF, sigmoide o polinomial). Define cuánta influencia tiene un solo punto de entrenamiento: valores bajos significan que cada punto tiene un alcance amplio, produciendo límites de decisión más suaves y generales; valores altos hacen que cada punto influya solo en su vecindario cercano, lo que puede generar límites muy ajustados a los datos de entrenamiento pero con alto riesgo de sobreajuste.
- **degree**: Cuando se utiliza el kernel polinomial, este parámetro especifica el grado del polinomio usado para transformar los datos. Un grado mayor permite capturar relaciones más complejas y no lineales, pero también aumenta el riesgo de sobreajuste y el costo computacional. Normalmente se utilizan grados bajos (2, 3 o 4) porque valores muy altos tienden a producir modelos inestables y difíciles de generalizar.
- **coef0**: Este parámetro es relevante para los kernels polinomial y sigmoide, donde actúa como un término independiente en la función del kernel. En el kernel polinomial, `coef0` controla cuánto influyen los términos de menor grado respecto a los de mayor grado; en el sigmoide, ayuda a definir la forma de la función de activación. Aunque no es tan crítico como C o gamma, puede necesitar ajuste para optimizar el rendimiento del modelo.
- **probability**: Este parámetro booleano determina si el modelo SVM debe calibrarse para proporcionar estimaciones de probabilidad junto con las predicciones de clase. Activarlo (`True`) permite obtener la probabilidad de pertenencia a cada clase mediante validación cruzada interna, lo cual es útil para interpretar la confianza del modelo. Sin embargo, este proceso aumenta significativamente el tiempo de entrenamiento y puede afectar ligeramente la precisión de las clasificaciones.

Ventajas específicas para detección de deepfakes

- **Fundamento teórico sólido:** A diferencia de otros métodos heurísticos, SVM está basada en la teoría de optimización convexa, lo que garantiza que la solución encontrada es óptima (máximo margen global).
- **Eficaz en espacios de alta dimensión:** Las características de audio pueden generar vectores de gran dimensionalidad (decenas o cientos de coeficientes). SVM maneja bien esta situación sin colapsar por la maldición de la dimensionalidad.
- **Robustez ante overfitting:** La maximización del margen proporciona una regularización natural que mejora la generalización, crucial cuando se trabaja con datasets limitados de voces deepfake.
- **Versatilidad mediante kernels:** La capacidad de elegir diferentes kernels permite adaptar el modelo a la estructura no lineal de los datos de audio sin necesidad de transformaciones manuales.
- **Interpretabilidad limitada pero útil:** Aunque no tan interpretable como los árboles de decisión, el análisis de los vectores de soporte puede revelar qué audios son más representativos o ambiguos en la frontera de decisión.

2.1.6. AASIST

AASIST (Audio Anti-Spoofing Using Integrated Spectro-Temporal graph attention networks) es un modelo de detección de “spoofing” de audio desarrollado en 2022 (**aasist2022**). La principal motivación para el desarrollo de este clasificador surge de que los patrones que sirven para diferenciar audios falsificados de audios bona fide se suelen encontrar en intervalos temporales o espectrales. Por esto, en la práctica se usaban ensembles costosos, desde el punto de vista computacional, donde cada subsistema se centraba en ciertas características. AASIST consigue reemplazar ese enfoque por un único sistema que combina información espectro-temporal buscando robustez ante diversos tipos de ataques.

Dentro de los modelos utilizados en este trabajo, AASIST destaca frente al resto por ser el más específico para la tarea de audio anti-spoofing, a diferencia de otros con usos fuera de este ámbito. Esta especialización para separar audios “bona fide” de muestras falsificadas se refleja tanto en su arquitectura, explicada en el siguiente punto, como en la motivación que llevó a su diseño.

Arquitectura y funcionamiento

El modelo tiene las siguientes fases:

1. Extracción de características a partir del audio en crudo:
 - Se usa un **encoder** basado en RawNet2 que acaba generando un mapa de características F , $F \in \mathbb{R}^{C \times S \times T}$ siendo C , número de canales, S , número de bins espectrales y T , longitud temporal. Este encoder permite trabajar directamente sobre la forma de onda cruda, sin necesidad de extraer previamente características, y generar representaciones de alto nivel con estructura espacio-temporal.
 - Para ello, primero, se pasa el audio por una primera capa *sinc-convolution* con 70 filtros. La diferencia frente al RawNet2 original, es que en AASIST se interpreta la salida de esta capa como una “imagen” 2D de 1 canal.
 - A continuación, esta primera representación se pasa por seis bloques residuales con **pre-activación** que van comprimiendo y transformando la información útil para tener una representación lo más compacta posible.
2. Posteriormente, se generan dos **grafos completos** que modelan, de forma independiente, los dominios temporal y espectral, siendo G_t y G_s respectivamente. A cada grafo, se le aplica “**graph pooling**” para reducir el número de nodos, conservando los más relevantes lo que permite bajar el coste computacional posterior.

Estos nodos son representaciones vectoriales resultantes del mapa de características producido por el encoder y las aristas que los unen representan las relaciones entre estos.
3. Ambos grafos se combinan estructuralmente en un tercer grafo, G_{st} con $N_t + N_s$ nodos entre los que se generan todas las aristas nuevas posibles en ambas direcciones, manteniendo las aristas de los grafos anteriores.
4. Bloque HS-GAL (Heterogeneous Stacking Graph Attention Layer) + “pooling”:
 - Se aplica **atención heterogénea** usando uno de los tres vectores de proyección según la relación entre los tipos de los nodos entre los que se hace la relación: temporal-temporal, espectral-espectral o temporal-espectral.
 - Un “**stack node**”, conectado de forma unidireccional a todos los nodos, que va acumulando la información espectro-temporal entre sucesivas capas HS-GAL.
 - Tras cada HS-GAL, se aplica otra vez la técnica “**graph pooling**” con el mismo objetivo anterior.

Sobre la combinación del punto 3 para establecer la conectividad entre grafos, se aplica la operación MGO (*Max Graph Operation*) que usa una combinación entre HS-GAL +

pooling donde se realiza la verdadera integración entre ambos dominios para que los nodos heterogéneos aprendan cómo deben relacionarse entre sí. De forma paralela, se procesan dos ramas en las que se ejecutan consecutivamente dos secuencias MGO. Dentro de cada rama, el “stack node” se propaga de la primera HS-GAL a la segunda. Finalmente, se combinan las salidas de ambas ramas usando un máximo elemento a elemento, teniendo como resultado un grafo heterogéneo.

5. Por último, usando las operaciones “**max**” y “**average**” sobre los nodos que originalmente formaban parte del grafo espectral y del grafo temporal, se generan 4 vectores que se concatenan con el “stack node” para formar la representación final en la capa de salida.

Innovaciones clave

- **HS-GAL**: una capa de atención, que permite ir integrando simultáneamente ambos grafos, combinada con un “stack node” para ir acumulando información espectro-temporal.
- **MGO**: un método con dos ramas aplicando paralelamente HS-GAL más una fase de “pooling”, lo que permite que cada secuencia pueda aprender distintos grupos de patrones.
- **Nuevo esquema de readout**: permite concatenar la información contenida en el “stack node” con las estadísticas sacadas de las operaciones “max” y “average” finales.

Ventajas específicas para detección de deepfakes

- **Integración espectro-temporal**: al modelar conjuntamente los grafos temporal y espectral en un grafo heterogéneo, permite encontrar información cruzada que se podría perder si se tratara por separado.
- **Stack node**: esta técnica “acumulativa” ayuda a encontrar “huellas” de suplantación (spoofing) cuando no están presentes en un solo nodo y se encuentran distribuidas en varios instantes o bandas.
- **Readout**: al juntar estadísticas globales (average) con evidencia puntual fuerte (max), facilita la detección de diferentes patrones de “spoofing”.
- **Ramas paralelas**: en MGO, se introducen dos ramas procesando en paralelo, permitiendo captar pistas complementarias.

2.1.7. Regresión Logística

A pesar de su nombre, la Regresión Logística es un algoritmo de clasificación lineal utilizado para predecir la probabilidad de que una instancia pertenezca a una categoría específica. Es un

modelo fundamental en el aprendizaje estadístico debido a su simplicidad, eficiencia y facilidad de interpretación.

Fundamento conceptual: La función Sigmoide

A diferencia de la regresión lineal, que predice valores continuos, la regresión logística utiliza la función logística o sigmoide para confinar el resultado de una ecuación lineal entre 0 y 1. La fórmula matemática es:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Donde z es la combinación lineal de las características de entrada ($w^T x + b$). El valor resultante se interpreta como la probabilidad de pertenencia a la clase positiva. Si la probabilidad es superior a un umbral (generalmente 0.5), se clasifica como “Real”, de lo contrario, como “Deepfake”.

Proceso de entrenamiento

El modelo aprende los pesos (w) minimizando una función de pérdida denominada **Log-Loss** o entropía cruzada binaria. Este proceso se realiza mediante algoritmos de optimización como el Gradiente Descendente. Durante el entrenamiento, el modelo ajusta los coeficientes para penalizar fuertemente las predicciones que se alejan de la etiqueta real.

Ventajas específicas para detección de deepfakes

- **Interpretabilidad directa:** Los coeficientes asignados a cada característica (como el pitch o los MFCC) indican directamente la importancia y la dirección de la influencia de cada parámetro en la detección del fraude.
- **Eficiencia:** Es computacionalmente muy ligero, lo que permite procesar grandes volúmenes de datos de audio en tiempo real con recursos mínimos.
- **Línea base robusta:** Sirve como un excelente modelo de referencia (*baseline*) para comparar el rendimiento de arquitecturas más complejas.

2.1.8. Regresión Ridge

El método de regresión Ridge permite detectar la multicolinealidad dentro de un modelo de regresión lineal. Fue propuesto por Hoerl y Kennard y es usado para trabajar con modelos que presentan sesgo.

Este método consiste en que dado una matriz ($X^T X$) altamente condicionada o cercana a singular, esto es, que es extremadamente sensible a pequeñas variaciones de los datos de entrada, es posible agregar constantes positivas a los elementos de la diagonal para asegurar que la matriz resultante no sea altamente condicionada.

Referencia 1:

En relación con el machine learning, la regresión Ridge sirve para corregir el sobreajuste de los datos de entrenamiento.

Cabe aclarar que la regularización es un método estadístico para reducir los errores causados por el sobreajuste de los datos de entrenamiento mientras que la regresión corrige específicamente la multicolinealidad en el análisis de regresión.

Este método resulta útil se desarrollan modelos con un gran número de parámetros.

La multicolinealidad denota cuando dos o más predictores tienen una relación casi lineal.

La regresión Ridge funciona de la siguiente manera:

- Coeficientes: utiliza un estimador estándar de coeficientes matriciales por mínimos cuadrados ordinarios (MCO): $B = (X^T X)^{-1} X^T y$. Esto se consigue encontrando la línea que mejor se ajuste a un conjunto de datos determinado mediante el cálculo de los coeficientes de cada variable independiente que, en conjunto, den como resultado la menor suma residual de cuadrados.
- Suma residual de cuadrados (RSS): mide la adecuación de un modelo de regresión lineal a los datos de entrenamiento. $RSS = \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$. Esta fórmula mide la precisión de la predicción del modelo para los valores reales de los datos de entrenamiento. Si $RSS = 0$, el modelo predice perfectamente las variables dependientes.

En un modelo de regresión lineal, si dos o mas variables comparten una correlación lineal alta, el MCO puede devolver coeficientes erróneos siendo esto prueba de sobreajuste.

La regresión Ridge lo que hace es modificar el MCO calculando coeficientes que tienen en cuenta predictores potencialmente correlacionados corrigiendo los coeficientes de alto valor introduciendo un término de regularización en la función RSS.

Término de regularización $L2 = \|B\|^2 = B_1^2 + B_2^2 + \dots + B_n^2$. Esto hace que la formula del RSS quede de la siguiente manera: $RSS_{L2} = \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 + \lambda \sum_{j=1}^p B_j^2$.

La agregación del L2 al RSS contrarresta los coeficientes especialmente altos al reducir todos los valores de los coeficientes.

Referencia 2:

2.1.9. K-Nearest Neighbors (k-NN)

El algoritmo de los K-Vecinos más Cercanos (k-NN) es un método de aprendizaje supervisado basado en instancias. A diferencia de otros modelos, k-NN no construye un modelo explícito durante el entrenamiento, sino que almacena todas las instancias de entrenamiento para realizar la clasificación en el momento de la consulta.

Arquitectura y funcionamiento

El principio de k-NN es la proximidad: asume que las muestras de audio con características acústicas similares pertenecerán a la misma categoría.

1. **Cálculo de distancia:** Cuando se presenta una nueva muestra de audio, el algoritmo calcula la distancia (generalmente la distancia euclidiana) entre esa muestra y todas las demás del conjunto de entrenamiento.
2. **Selección de vecinos:** Se seleccionan los k ejemplares más cercanos (donde k es un número entero definido por el usuario).
3. **Votación mayoritaria:** La muestra se asigna a la clase que sea más frecuente entre sus k vecinos.

El papel del parámetro k

La elección de k es crítica: un valor muy pequeño (ej. $k = 1$) hace que el modelo sea sensible al ruido en el audio, mientras que un valor muy grande puede suavizar demasiado las fronteras de decisión.

Ventajas específicas para detección de deepfakes

- **Sin suposiciones sobre los datos:** No asume que las características del audio sigan una distribución lineal, lo que le permite capturar patrones complejos en la distribución de artefactos sintéticos.
- **Eficacia con características bien definidas:** Si la extracción de características logra separar bien las clases, k-NN es extremadamente preciso.
- **Aprendizaje incremental:** Es fácil añadir nuevos ejemplos de ataques de audio sintético al sistema sin necesidad de reentrenar todo el modelo.

Como hemos visto, se ha presentado un espectro diverso de clasificadores que permiten abordar la detección de spoofing desde diferentes ángulos. Los modelos lineales y de vecinos cercanos, como la **Regresión Logística** y **k-NN**, ofrecen una base sólida y eficiente para el análisis de características acústicas predefinidas. Por otro lado, métodos como **Random Forest** y **LightGBM** incrementan la robustez del sistema ante datos ruidosos y relaciones no lineales.

Por último, modelos como **Wav2Vec2** y **AASIST** procesan la señal de audio cruda y modelan relaciones espectro-temporales mediante grafos y mecanismos de atención. La combinación de estos enfoques permite no solo alcanzar altas tasas de precisión, sino también adaptarnos según la disponibilidad de recursos computacionales. Esta base taxonómica de modelos constituye el núcleo operativo sobre el cual se aplicarán posteriormente las técnicas de explicabilidad.

2.2. Modelos de explicación

La adopción de modelos de aprendizaje profundo ha permitido alcanzar una precisión sin precedentes en la detección de audio sintético. Sin embargo, estos sistemas operan frecuentemente como “cajas negras”, lo que dificulta su aplicación en ámbitos críticos como en forense o ciberseguridad. La Inteligencia Artificial Explicable (XAI) surge como la solución técnica para dotar a estos modelos de transparencia, permitiendo a los expertos entender el “por qué” de cada clasificación.

A continuación, se describen las técnicas de explicabilidad, organizados según el siguiente enfoque técnico:

- **Métodos de Atribución y Perturbación:** Se analizan técnicas post-hoc, es decir, que no dependen del modelo al que se aplican, como **SHAP** y **LIME**, que permiten identificar qué características acústicas individuales son determinantes para el modelo; métodos basados en perturbaciones, como **Occlusion** y **RISE**, que miden la importancia de regiones modificando de forma controlada la entrada; y métodos basados en gradientes, como **Grad-CAM** e **Integrated Gradients**, fundamentales para visualizar la relevancia en el plano tiempo-frecuencia.
- **Razonamiento Contrafáctico:** Los modelos contrafactuales generan ejemplos alternativos a una muestra original que permiten entender la frontera de decisión del modelo estudiado (“¿qué cambios harían que este audio fake pareciera real?”). Como ejemplo de este tipo de enfoque, se explora el modelo **DiCE** que es uno de los más representativos para datos tabulares.
- **Modelos Interpretables por Diseño:** Se profundiza en arquitecturas cuya transparencia es intrínseca, como **SincNet**, que utiliza filtros con significado físico, y **ProtoPNet**, que basa su decisión en la comparación con prototipos o ejemplos conocidos.

Además de esa clasificación, en interpretabilidad, los métodos suelen clasificarse según qué alcance tiene la explicación, a qué conjuntos de modelos se pueden aplicar y en qué momento aparece la interpretabilidad dentro del proceso de aprendizaje. Por un lado, pueden distinguirse según el alcance de la explicación en métodos **locales**, **globales** o de **cohortes**: los locales explican una predicción concreta para una muestra individual; los globales describen el comportamiento del modelo sobre el conjunto total de datos; y los “cohort explanations” analizan el comportamiento del modelo sobre un subconjunto de muestras que comparten cierta característica. Por otro lado, según a qué modelos se aplican, existen métodos “**model-agnostic**”, que pueden aplicarse a cualquier modelo porque solo necesitan acceder a sus entradas y salidas; métodos “**model-specific**”, diseñados para modelos concretos; y métodos “model-

class-specific”, aplicables a una familia de modelos que comparten una estructura común, como árboles, redes neuronales o modelos lineales. Por último, también pueden clasificarse en “ante-hoc” y “post-hoc”: los métodos “**ante-hoc**” son aquellos en los que la interpretabilidad viene incorporada en el diseño del propio modelo, mientras que los “**post-hoc**” generan explicaciones una vez entrenado el modelo, sin necesidad de que este sea interpretable desde un primer momento.

2.2.1. SHAP

Basado en la teoría de juegos cooperativos, SHAP (Shapley Additive Explanations) es una técnica cuyo objetivo es cuantificar la contribución de cada característica a la decisión final de un modelo (**2017shap**). Su fundamento teórico proviene de los valores de Shapley, método utilizado para repartir de manera justa los beneficios obtenidos entre los distintos jugadores que participan en un juego.

Trasladado al contexto de la detección de audios generados, la idea principal de SHAP consiste en medir cuánto aporta cada característica acústica cuando se usa junto a distintas combinaciones del resto de variables. De este modo, no se evalúa únicamente la contribución aislada de una característica, sino también el impacto de cada característica en presencia de otras, permitiendo captar mejor las relaciones entre variables. A partir de esta idea, SHAP asigna a cada característica un valor que representa su contribución marginal a la predicción del modelo. Entonces, una variable con un valor positivo empuja la clasificación hacia valores altos respecto a un valor de referencia, mientras que un valor negativo provoca lo contrario.

El valor de Shapley ϕ_i para una característica i se calcula como:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$$

donde F es el conjunto total de características, S es un subconjunto de F que no contiene a i , y $f(S)$ representa la predicción del modelo al considerar únicamente las características de S .

Se trata de un método “post-hoc”, generalmente “model-agnostic”, que ofrece explicaciones locales para muestras concretas aunque usando agregación de resultados, también se pueden conseguir explicaciones globales o por cohortes. Este método es especialmente valorado en el análisis forense debido a su consistencia matemática y su capacidad para ofrecer tanto explicaciones locales como globales. Mientras que las explicaciones locales proporcionan información sobre la influencia de cada característica sobre el resultado de una muestra concreta, las explicaciones globales ofrecen una visión de la relevancia media de las características sobre el conjunto total de datos.

SHAP, aunque se pueda aplicar sobre varios tipos de datos como imágenes o texto, se utiliza principalmente sobre datos tabulares. En el caso del audio, esto se corresponde con características acústicas previamente extraídas como MFCC o pitch.

2.2.2. LIME

LIME (Local Interpretable Model-agnostic Explanations) es una técnica de explicabilidad “post-hoc” cuyo objetivo es explicar predicciones individuales mediante aproximaciones locales interpretables (**2016lime**). Se considera agnóstico del modelo, ya que solo necesita observar las salidas para las nuevas muestras perturbadas.

Para conseguir esta explicación, LIME parte de una muestra concreta x cuya predicción se quiere interpretar. A partir de ella, se generan múltiples versiones perturbadas modificando o eliminando parcialmente algunas de sus características. Cada una de estas muestras perturbadas se introduce en el modelo original f , obteniendo un conjunto de predicciones que permiten evaluar cómo cambia la salida del modelo cuando varía la entrada.

Una vez obtenidas estas predicciones, LIME calcula la similitud entre cada muestra perturbada y la muestra original mediante una función de proximidad π_x . Las muestras más parecidas a x reciben un peso mayor, ya que son las que mejor representan el comportamiento local del modelo alrededor de la instancia analizada. En cambio, las muestras más alejadas tienen menor influencia en la explicación, puesto que se consideran menos relevantes para interpretar esa predicción concreta.

A continuación, se entrena un modelo interpretable g utilizando las muestras perturbadas y sus correspondientes predicciones, ponderadas según la función de proximidad π_x . Este modelo sustituto no pretende imitar el comportamiento global del modelo sobre el que se aplica LIME, sino aproximar su funcionamiento únicamente en una región local alrededor de x . De esta forma, LIME facilita la interpretación de un modelo complejo, obteniendo el análisis de un modelo más sencillo y comprensible.

La explicación $\xi(x)$ se obtiene resolviendo:

$$\xi(x) = \arg \min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

donde G es la familia de modelos interpretables considerados, $\mathcal{L}(f, g, \pi_x)$ mide la pérdida de fidelidad entre el modelo original y el modelo sustituto en el entorno de x , y $\Omega(g)$ controla la complejidad del modelo interpretable.

En el contexto de la detección de audios generados, LIME puede aplicarse sobre distintas representaciones del audio. Si se trabaja con características acústicas previamente extraídas,

como MFCC o pitch, las perturbaciones se realizan sobre dichas variables para analizar cómo afectan a la predicción del modelo.

La explicación resultante corresponde a los parámetros del modelo interpretable aprendido localmente. En el caso de una regresión lineal, estos parámetros son sus coeficientes, que indican la importancia y dirección de influencia de cada característica en la predicción de una muestra concreta. Por tanto, un coeficiente positivo indica que la característica contribuye a aumentar la salida analizada del modelo, mientras que un coeficiente negativo indica que contribuye a reducirla.

2.2.3. Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping) es una técnica de explicabilidad “post-hoc” utilizada principalmente en modelos basados en capas convolucionales, por lo que se considera “model-agnostic”. Su objetivo es identificar qué regiones de la entrada, o de una representación intermedia del modelo, han contribuido en mayor medida a la clasificación de una muestra concreta (2017gradcam).

A diferencia de otras técnicas, Grad-CAM utiliza la información interna de las capas convolucionales del modelo. Estas capas generan mapas de activación que contienen los patrones detectados por el modelo en la entrada analizada. Para una clase concreta, Grad-CAM calcula cómo cambiaría la salida del modelo si variasen dichos mapas de activación. De esta forma, estima cuáles de estos mapas han tenido mayor influencia en la predicción final.

El proceso comienza seleccionando una clase objetivo c , generalmente la predicha por el modelo. A continuación, se utiliza el modelo para obtener la puntuación asociada a dicha clase. Posteriormente, para una capa convolucional seleccionada, se calcula el gradiente de esa puntuación respecto a los mapas de activación, cada uno de los cuales está formado por múltiples valores internos correspondientes a posiciones espaciales dentro del mapa. Estos gradientes indican la sensibilidad de la salida del modelo respecto a cada activación individual A_{ij}^k , es decir, cómo cambiaría la puntuación de la clase objetivo si se alterase una posición concreta.

Para obtener la importancia de cada mapa de activación, Grad-CAM calcula α_k^c mediante el promedio espacial de los gradientes:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}$$

donde y^c representa la salida del modelo para la clase c , A^k es el mapa de activación k de la capa convolucional seleccionada, y Z es el número total de posiciones espaciales.

Una vez calculados estos pesos, se combinan los mapas de activación mediante una suma ponderada y se aplica una función ReLU:

$$L_{\text{Grad-CAM}}^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right)$$

La función ReLU permite conservar únicamente las regiones que contribuyen positivamente a la clase analizada. El resultado es un mapa de calor que señala las zonas más relevantes para la predicción del modelo.

En el contexto de la detección de audios generados, Grad-CAM puede aplicarse sobre representaciones tiempo-frecuencia, como espectrogramas, o sobre mapas de activación internos obtenidos a partir de capas convolucionales del modelo. De este modo, permite identificar qué regiones temporales o frecuenciales han influido más en la clasificación de una muestra. Por ejemplo, en un espectrograma, las zonas con mayor intensidad en el mapa resultante indicarían las regiones que el modelo ha considerado más relevantes para tomar su decisión.

2.2.4. Integrated Gradients

Integrated Gradients (IG) es una técnica de carácter local, de interpretabilidad “post-hoc” y “model-class-specific”, ya que requiere acceder a los gradientes del modelo sobre el que se aplica (**2017integratedgradients**). Se basa en integrar los gradientes de la función del modelo a lo largo de una trayectoria que conecta una entrada de referencia (“baseline”) x' con la entrada real x . Este método permite capturar cómo varía la sensibilidad del modelo respecto a cada característica a lo largo del recorrido, evitando problemas como la saturación del gradiente en el punto de entrada.

Siendo f la función del modelo, i una característica específica que se está observando, x' el “baseline” y α que parametriza el camino continuo entre x y x' :

$$IG_i(x) = (x_i - x'_i) \times \int_{\alpha=0}^1 \frac{\partial f(x' + \alpha(x - x'))}{\partial x_i} d\alpha$$

Tres propiedades clave de IG que justifican teóricamente su uso son:

- **Sensibilidad:** si x y x' difieren únicamente en una característica y la predicción varía, entonces esa característica debe tener una atribución no nula.
- **Invariancia de implementación:** si dos modelos implementan la misma función aunque varíen las arquitecturas o la organización de parámetros, entonces las atribuciones de IG serán iguales.

- **Completitud:** la suma de las atribuciones de todas las características es igual a la diferencia entre la predicción del modelo para x y x' .

En audio, el “baseline” se representa como ruido blanco o como silencio.

2.2.5. DICE

DiCE (Diverse Counterfactual Explanations) es una técnica de explicabilidad post-hoc, generalmente considerada “model-agnostic”, basada en la generación de explicaciones contrafactuales (**2020dice**). A diferencia de otros métodos, DiCE busca identificar qué cambios mínimos en una muestra concreta modificarían la predicción del modelo hacia una clase objetivo diferente.

La idea principal de las explicaciones contrafactuales consiste en responder a preguntas del estilo de: “¿qué tendría que cambiar en la entrada para que el modelo produjera otra predicción?”. De este modo, para una muestra original clasificada en una determinada clase, DiCE genera nuevas muestras similares a ella, denominadas contrafactuales, que son clasificadas por el modelo como una clase alternativa elegida.

En cuanto a las muestras contrafactuales, DiCE cumple que estas sean cercanas a la muestra original, de forma que la explicación resultante no requiera modificaciones excesivas. Además, una característica importante de esta técnica es que busca generar varias alternativas contrafactuales con el objetivo de conseguir explicaciones diversas.

En el contexto de la detección de audios generados, DiCE puede aplicarse sobre características acústicas previamente extraídas, como MFCC o pitch. Por ejemplo, si una muestra es clasificada como “spoof”, DiCE puede generar ejemplos contrafactuales que indiquen qué modificaciones en dichas características harían que el modelo la clasificase como “bona fide”. De esta forma, la explicación no se limita a señalar qué variables son importantes, sino que muestra qué cambios concretos en la representación de entrada podrían alterar la decisión del modelo.

Teniendo en cuenta lo explicado, DiCE resulta especialmente útil cuando se desea obtener una explicación accionable de una predicción, ya que permite analizar posibles modificaciones de la entrada que llevarían a un resultado diferente.

2.2.6. SincNet

SincNet es una arquitectura “post-hoc” y “model-specific”, ya que la explicabilidad se incorpora directamente al diseño del propio modelo (**2018sincnet**). Sustituye las capas de convolución estándar por filtros de banda basados en la función Sinc, una función matemática utilizada para construir filtros pasa-banda definidos por sus frecuencias de corte inferior y

superior f_1 y f_2 . En lugar de aprender todos los coeficientes de los filtros, el modelo solo aprende estas frecuencias de corte, permitiendo una interpretación física directa de los rangos espectrales (como formantes o armónicos) que el sistema utiliza para discriminar la voz real de la sintética.

La principal ventaja de SincNet desde el punto de vista de la explicabilidad es que los filtros aprendidos pueden analizarse en términos de bandas de frecuencia. Esto permite estudiar qué regiones espectrales utiliza el modelo en las primeras etapas del procesamiento. En el contexto de la detección de audios generados, dichas bandas podrían relacionarse con patrones acústicos relevantes para distinguir entre audio “bona fide” y audio “spoof”, como alteraciones en determinadas zonas frecuenciales, variaciones en la estructura armónica o diferencias en regiones asociadas a la energía vocal.

Este método opera directamente sobre la onda de audio cruda por lo que no requiere espectrogramas ni características acústicas adicionales. Esto favorece que se use para una interpretación global de los resultados.

2.2.7. ProtoPNet

ProtoPNet (Prototypical Part Network) es una arquitectura de interpretabilidad “ante-hoc” y “model-specific”, ya que la explicación forma parte del propio mecanismo de decisión del modelo (**2019protopnet**). El mecanismo de decisión de este modelo consiste en comparar regiones de la representación interna de una muestra con un conjunto de prototipos aprendidos durante el entrenamiento. Con esta idea, proporciona explicaciones locales de la forma “esta parte de la entrada se asemeja a cierta región de este prototipo”.

Además de proporcionar explicaciones locales para muestras concretas, ProtoPNet también permite obtener una interpretación más global del modelo mediante el análisis de los prototipos asociados a cada clase. Al observar qué prototipos utiliza el modelo para reconocer una determinada categoría, es posible identificar qué patrones considera representativos de dicha clase.

En el contexto de la detección de audios generados, una arquitectura inspirada en ProtoPNet podría aplicarse sobre representaciones tiempo-frecuencia, como espectrogramas, o sobre representaciones internas aprendidas por una red neuronal. En este caso, los prototipos podrían corresponder a patrones acústicos recurrentes, como determinadas estructuras espectrales, regiones temporales o zonas tiempo-frecuencia relevantes para distinguir entre audios “bona fide” y “spoof”. La explicación tendría entonces la forma: “esta región del audio se parece a este patrón prototípico aprendido para una clase”.

2.2.8. Occlusion

Occlusion, propuesta por Zeiler y Fergus, es una técnica de interpretabilidad “model-agnostic”, “post-hoc”, que ofrece explicaciones locales (**2014occlusion**). Analiza la sensibilidad del modelo ocultando sucesivamente pequeñas regiones de la entrada y observando la variación de su salida. En imágenes, este análisis se realiza desplazando un cuadrado gris sobre la imagen; cuando la puntuación de la clase disminuye de forma notable, la región “tapada” se considera relevante para la decisión del clasificador. A diferencia de otros métodos, Occlusion solo necesita acceder a las predicciones del modelo.

En audio, adoptamos la misma idea pero aplicada sobre ventanas temporales o sobre regiones de un espectrograma, permitiendo identificar qué zonas de la señal contienen la información más importante a la hora de clasificar muestras.

2.2.9. RISE

RISE (Randomized Input Sampling for Explanation) es un método de explicabilidad “post-hoc”, de carácter local y “model-agnostic,” basado en perturbaciones aleatorias de la entrada (**2018rise**). Su funcionamiento consiste en generar múltiples versiones enmascaradas de la señal mediante máscaras aleatorias que ocultan total o parcialmente distintas regiones de la entrada. La importancia de cada segmento se estima a partir de la media del *score* del modelo sobre todas aquellas entradas enmascaradas en las que dicho segmento permanece visible, es decir, en las que el valor de la máscara en esa posición es distinto de cero. Aunque al igual que con Occlusion, esta técnica fue propuesta originalmente para imágenes, en este trabajo se ha adaptado su planteamiento al dominio de audio.

La importancia de cada segmento puede expresarse mediante la siguiente aproximación:

$$S_{I,f}(\lambda) \approx \frac{1}{N \cdot \mathbb{E}[M]} \sum_{i=1}^N f(I \odot M_i) M_i(\lambda)$$

donde I es la entrada original, N es el número total de máscaras generadas, $\mathbb{E}[M]$ representa la fracción media de posiciones visibles en cada máscara, y M_i es la máscara aleatoria i -ésima. Además, $f(I \odot M_i)$ corresponde al *score* asignado por el modelo a la entrada enmascarada, mientras que $M_i(\lambda)$ indica si el segmento λ permanece visible en dicha máscara.

2.3. Conclusiones

Una vez revisados los modelos de clasificación considerados en este trabajo, puede observarse que no todos responden al mismo grado de especialización respecto a la tarea de detección de audio falsificado. Por un lado, se han estudiado modelos de carácter más general, ampliamente utilizados en problemas de clasificación de distinta naturaleza, como **Random Forest**, **k-NN**, **LightGBM**, **SVM** o la **Regresión Logística**. Por otro lado, se han incluido enfoques más específicos para el ámbito del procesamiento de señal y al análisis de voz, como el **Perceptrón Multicapa**, **Wav2Vec2** o **AASIST**, este último diseñado para tareas de “**anti-spoofing**”. Esta diversidad permite abordar el problema desde perspectivas complementarias, permitiendo comparar soluciones más generales y fácilmente interpretables con otras más específicas y de mayor complejidad.

De manera paralela, el bloque dedicado a explicabilidad muestra que la interpretación de modelos en detección de “deepfakes” de audio tampoco puede reducirse a una única estrategia. Las técnicas post-hoc, como **SHAP**, **LIME**, **Grad-CAM**, **Integrated Gradients**, **Occlusion** o **RISE**, permiten analizar el comportamiento de modelos complejos una vez entrenados, identificando qué regiones, variables o patrones han resultado más influyentes en la decisión. Frente a ellas, enfoques ante-hoc o intrínsecamente interpretables, como **SincNet** o **ProtoPNet**, incorporan la explicabilidad en la propia arquitectura del modelo. Asimismo, herramientas como **DiCE** añaden el análisis del ámbito contrafactual, permitiendo estudiar qué cambios mínimos en la entrada podrían modificar la predicción.

En conjunto, ambos bloques, clasificación y explicabilidad, constituyen la base sobre la que se desarrolla el proyecto. En este sentido, los modelos de clasificación y las técnicas XAI no deben entenderse como bloques independientes, sino como componentes complementarios de un mismo sistema de análisis. Mientras que los primeros aportan distintas capacidades predictivas y distintos niveles de adaptación al problema, los segundos permiten interpretar y justificar en que evidencias técnicas se basan estas predicciones.

A partir de este marco teórico, en el siguiente capítulo se explica el desarrollo práctico del trabajo, incluyendo tanto la metodología seguida como el análisis de los resultados obtenidos. En la siguiente sección, se concreta también la aportación propia del proyecto, como una propuesta orientada a adaptar, combinar y analizar distintas técnicas de IA y XAI para avanzar en la tarea de detección de “spoof”, con el objetivo, no únicamente de construir un clasificador capaz de distinguir entre audios naturales y generados, sino también disponer de mecanismos que permitan comprender a nivel humano que características o patrones han determinado la clasificación.

3. Metodología

En este capítulo se va a hablar del desarrollo en sí del proyecto, así como de los resultados obtenidos. En primer lugar se va a comentar sobre la elección de un dataset (ver sección [Dataset](#)). Tras esto, se explican las distintas características que se pueden extraer de un audio (ver sección [Extracción de características](#)). Acto seguido, se mencionan las métricas de evaluación que usarán, por lo general, los modelos (ver [Métricas de evaluación](#)). A continuación, se va a definir el proceso común de experimentación que se llevará a cabo en ciertos modelos, para así no explicar lo mismo varias veces (ver [Proceso común de experimentación](#)). Después, se pasa a explicar cómo hemos definido cada modelo: hiperparámetros usados, pruebas realizadas, etc., sin entrar en resultados (ver [Modelos](#)). Tras definir los modelos, se mencionan los resultados obtenidos, haciendo algunos comentarios sobre estos de manera individual (ver [Resultados](#)). Después de ver los resultados, se hace una discusión más extensa de los mismos, comparando entre modelos (ver [Discusión de los resultados](#)). Finalmente, se hace un cierre del capítulo con las conclusiones (ver [Conclusiones](#)).

3.1. Dataset

Para encontrar un dataset público que tuviera voces tanto reales como clonadas y que además fueran por pares (i.e. un audio de voz real cuya transcripción es "Hoy he comido en un restaurante chino" y un audio clonando dicha voz diciendo lo mismo) ha sido complicado. Hemos echado un ojo a bastantes datasets. Vamos a comentar los que hemos tenido en cuenta.

1. **HABLA 3tamayoflorez23_interspeech**: Este dataset cuenta con una gran cantidad de audios tanto reales, como generados (con técnicas como TTS) y voces clonadas (con técnicas como Cycle-GAN). Lo hemos descartado para una primera iteración del proyecto, pero no lo descartamos para una próxima iteración.
2. **audioMNIST (4BECKER2024)**: Este dataset cuenta con 30 mil audios de los números del 0 al 9 en inglés. Lo hemos descartado porque todos los audios son voces reales, por lo que no nos sirve para nuestra tarea.

3. ASVSpooof2021 ([asvspoof2021__zenodo](#)): Este dataset forma parte de una serie de desafíos internacionales organizados por la comunidad de verificación automática de audio cuyo objetivo es evaluar y mejorar los sistemas capaces de detectar voz manipulada. El dataset se compone de tres grupos de audio, todos ellos con voz real y generada mezclada, logical access con grabaciones transmitidas mediante canales de voz con efectos de codificación y transmisión, physical access con grabaciones de voz auténticas y ataques de reproducción en espacios físicos reales y speech deepfake con grabaciones generadas sintéticamente y comprimidas de formas distintas. Este dataset lo hemos descartado por no contar con pares de audio aunque se pueden rescatar algunos audios en inglés sobre todo para probar los modelos que probemos más adelante.
4. Mozilla Common Voice: Este dataset contenía voces de personas reales en varios idiomas, lo cual daba muchas posibilidades, pero no tenía las respectivas voces generadas por IA por lo que decidimos descartarlo para buscar otro dataset más acorde a lo que necesitábamos.
5. Wavefake (??): Este dataset de Kaggle cuenta con gran cantidad de audios generados. Como tal, nos sirvió para probar el primer prototipo de extracción de características y buscar librerías como librosa o speechRecognition. Este dataset, al no tener los audios reales, no nos sirve para luego desarrollar los modelos.
6. In-the-wild:[] este dataset busca condiciones realistas recopilando audios reales y falsos desde fuentes públicas como redes sociales y plataformas relacionadas. Cuenta con 38 horas de audios en inglés repartidas más o menos equitativa entre reales y falsos.

Finalmente, el dataset por el que nos decantamos fue [ODSS](#). Hemos usado este conjunto de audios porque ya tiene creado un par para cada audio, lo que nos ahorra tiempo de generar nuestros propios audios. A esta ventaja se le suma que ODSS ya contenía audios en inglés y en español, ambos idiomas son sumamente hablados por lo que es interesante centrarse en ellos.

Análisis del dataset

De este subconjunto de audios extraídos sacaremos todas sus características que explicaremos más adelante junto con su espectrograma, su duración, su categoría (natural o generado) y el audio. Esto hace un total de 47 columnas por cada audio ya que de características como las formantes y los mfcc's que no solo se saca su valor promedio sino que también se sacará su distribución a lo largo del audio. Un resumen de los tipos de datos que encontramos en nuestro dataset sería:

- **Datos tabulares:** La mayoría de los datos sobre las características del audio.

- **Series temporales:** Muestran la evolución de las formantes o los mfcc's a lo largo del audio.
- **Datos de audio:** Se corresponde con el archivo de audio (en la práctica es la ruta al archivo de audio).
- **Datos de texto:** Se corresponde con las transcripciones de los audios.

3.2. Extracción de características

Para poder clasificar los audios en reales y generados separamos los datos que podemos proporcionar a dichos modelos en 3 grupos:

1. Características tabulares. Podemos extraer características de la voz como el tono fundamental, los formantes, las pausas, y otros. Con estos datos, los modelos pueden aprender las relaciones entre los dos grupos de audios.
2. Gráficas. Podemos extraer gráficas de la voz como la forma de onda, el espectrograma de MEL, y otros. Con estas gráficas las “Convolutional Neural Networks” pueden aprender los puntos clave que diferencian a los audios reales de los generados.
3. Audio crudo. Existen modelos llamados [Wav2Vec2](#) los cuales aprenden directamente de los audios, sacando un vector denso de características latentes en los audios, para después hacer ajuste fino o *fine-tuning* de un modelo para clasificar los audios. Por otro lado, hay modelos como [AASIST](#) que también utilizan la onda cruda y aprenden representaciones discriminativas durante el entrenamiento.

A continuación, estudiamos que características de cada tipo podemos extraer.

Características

Podemos extraer características de la voz como el tono fundamental, los formantes, las pausas, y otros. Con estos datos, los modelos pueden aprender las relaciones entre los dos grupos de audios. Cabe destacar que las características de este tipo extraídas son mayormente arrays en el que cada componente es un valor en un momento determinado. Esto se debe a que la voz tiene una componente temporal, por lo que hay que dividir cada audio en marcos temporales (frames). Además, los modelos que aplicaremos no entrenan con arrays, por lo que los promediaremos con la media y/o desviación típica. Vamos a explicar cada característica que extraemos de los audios:

1. **Valor cuadrático medio (RMS):** Mide la energía promedio de una señal acústica (i.e. la intensidad o volumen general del audio). Se extrae dividiendo la señal en frames, para cada frame se calcula la raíz cuadrada del promedio de los valores de onda al cuadrado y, por último, se promedian esos valores. En voz humana natural, el RMS tiende a tener variaciones más orgánicas, mientras que audios sintéticos pueden mostrar patrones más uniformes.
2. **Tasa de cruces por cero (ZCR):** Mide el número de veces que la señal cambia de signo por segundo, lo que nos permite distinguir entre los sonidos tonales (periódicos) y los ruidosos (no periódicos). Se extrae detectando los puntos donde la señal cambia de signo, se cuentan dichos puntos por unidad de tiempo y se normaliza por la longitud del frame. Los sonidos sordos (como /s/, /f/) tienen ZCR alto, mientras los sonoros (/a/, /m/) tienen ZCR bajo.
3. **Centroide espectral:** Mide el centro de masa del espectro de frecuencias, lo que nos dice, de forma ponderada, si la energía de un sonido se concentra en las frecuencias graves (bajas) o en las frecuencias agudas (altas). Se obtiene calculando la Transformada Rápida de Fourier (FFT) para cada frame para obtener el espectro de frecuencias, se ponderan las frecuencias por su magnitud (energía de dicha frecuencia) y se calcula la media de estas. Cabe mencionar que las voces femeninas tienden a tener centroides más altos que las masculinas.
4. **Ancho de banda espectral:** Mide qué tan dispersas están las frecuencias alrededor del centroide, para ver si el sonido es “puro” como un silbido o una sílaba mantenida, o es “ruidoso” como el ruido blanco o un aplauso. Se extrae hallando el centroide espectral y se calcula la desviación típica ponderada en vez de la media ponderada.
5. **Punto de caída espectral (spectral rolloff):** Mide la frecuencia por debajo de la cual se concentra un determinado porcentaje de la energía total del espectro, es decir, la frecuencia límite para un porcentaje de energía. Normalmente, dicho porcentaje es del 85 %, aunque puede variar entre 75 % a 90 %, y funciona como un percentil. Se obtiene calculando el centroide espectral y sumando la energía progresivamente hasta que lleguemos a una frecuencia con la que nos pasamos de dicho porcentaje. Es útil para distinguir entre sonidos con energía concentrada en bajas frecuencias, en un amplio espectro, o en altas frecuencias.
6. **Planitud espectral:** Mide cómo de plano es el sonido. Varía entre 0 y 1 siendo 0 un sonido con picos pronunciados y 1 un sonido con pocos picos. Se obtiene calculando el espectro como antes (FFT) y dividiendo la media geométrica del espectro entre la media aritmética del mismo.

7. **Tono fundamental (F0):** Es la frecuencia más baja y periódica de un sonido. En el contexto de la voz, es la frecuencia a la que vibran las cuerdas vocales. El tono fundamental puede verse en, por ejemplo, las emociones, las preguntas frente a afirmaciones, las notas musicales, entre otras, por lo que nos da mucha información acerca de algo o alguien. Hay varias formas de calcularlo, y ninguna es 100 % precisa. Nosotros nos decantamos por la función PYIN, la cual primero obtiene las frecuencias fundamentales mediante el algoritmo YIN y después aplica el algoritmo probabilístico de Viterbi para obtener la cadena F0 más probable. Estos son algoritmos más sofisticados que combinan autocorrelación, análisis espectral y probabilidades para evitar errores comunes como la octava errónea (detectar 200Hz por ruido en vez del tono real de 100Hz), zonas no voceadas (unvoiced) y ruido de fondo.
8. **Formantes:** Son las frecuencias de resonancia del tracto vocal (la boca, la garganta y la nariz). Estas frecuencias se amplifican de forma natural cuando hablamos, dependiendo de la forma que le demos a la boca.
- a) F1 (Primer formante): Relacionado con la abertura de la mandíbula (qué tan abierta está la boca).
 - b) F2 (Segundo formante): Relacionado con la posición de la lengua (adelante o atrás).
 - c) F3 (Tercer formante): Relacionado con la posición de la punta de la lengua y tiene mucha importancia en las vocales “r” y en la identidad del hablante.

Para extraerlos hay varias alternativas, nosotros usamos el método de Burg para estimar las formantes, después se muestrean 100 puntos a lo largo del audio y se calcula la media.

9. **Relación Armónico-Ruido:** Mide la proporción entre energía periódica (armónicos) y energía aperiódica (ruido). Esta relación es una indicadora directa de la eficiencia fonatoria y la salud vocal: puede indicar la existencia de patologías como voz ronca, o afonía; una fatiga vocal; o, en algunos casos, emociones. Hay varias formas de calcularla: como el método de autocorrelación o el de análisis espectral. Nosotros usamos la función `to_harmonicity` de `parselmouth.Sound`, la cual la calcula con el método de autocorrelación: primero halla el F0 de los frames; después aplica la función de autocorrelación normalizada $r(\tau) = \frac{\int x(t) \cdot x(t+\tau) dt}{\int x^2(t) dt}$; luego busca el máximo de la función de autocorrelación en el intervalo alrededor del período fundamental esperado, este máximo corresponde a la correlación entre la señal y su versión desplazada un período; tras esto calcula el HNR en decibelios $HNR = 10 \cdot \log_{10} \left(\frac{r_{\max}}{1 - r_{\max}} \right)$ y, por último, aplica interpolación parabólica para obtener valores más precisos del máximo y realiza correcciones para compensar el efecto de la ventana de análisis. Es interesante saber que las voces humanas naturales suelen oscilar entre los 10 a 25 dB, por lo que unos valores extremos pueden indicar voz sintética.

10. **Proporción de silencio:** Mide el porcentaje del tiempo o de frames donde no hay voz activa, es decir, hay silencio o ruido de fondo hasta cierto umbral. La forma de calcularla es muy simple: se establece un umbral de energía debajo del cual una energía es considerada silencio, se calcula el rms y se le aplica el filtro del umbral. Finalmente se divide el número de frames considerados silencio entre el número total de frames. Esto, pese a su simpleza, puede ser determinante en el análisis de una voz ya que las IAs pueden generar patrones de pausas poco naturales o demasiado regulares.
11. **Coefficientes Cepstrales en Frecuencia de Mel (MFCC):** Son una representación compacta del espectro de frecuencias que imita cómo percibe el sonido el oído humano. En lugar de trabajar con frecuencias lineales (Hz), trabajan con una escala llamada escala Mel, que es más sensible a los cambios en frecuencias bajas que en altas (como nuestro oído). Son un total de 13 coeficientes: el primero representa la energía promedio del espectro (similar al RMS, pero en el dominio cepstral); mientras que el resto representan la forma detallada del espectro: los picos, los valles, las pendientes... Cada coeficiente añade un nivel de detalle diferente. Estos coeficientes son muy importantes ya que son, con diferencia, la característica más utilizada en: reconocimiento de voz (Speech-To-Text), reconocimiento de hablante (Identificación), clasificación de géneros musicales, entre muchas otras aplicaciones. Hoy en día son un estándar en reconocimiento de voz. Se extraen mediante un proceso más laborioso: primero se hace un pre-énfasis, amplificando las frecuencias altas, ya que estas tienen poca energía. Posteriormente, se calcula el espectro de frecuencias de cada frame; tras esto se aplica un banco de filtros triangulares espaciados según la escala Mel, donde por debajo de los 1000 Hz se aplican filtros lineales (nuestro oído distingue bien los cambios en frecuencias bajas) y por encima de los 1000 Hz se aplican filtros logarítmicos (nuestro oído distingue peor los cambios en frecuencias altas), con la fórmula $Mel(f) = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right)$ y, para terminar, aplica la Transformada Coseno Discreta (DCT), la cual decorrelaciona las bandas de Mel, que suelen estar correlacionadas entre sí, compacta la información y devuelve los 13 primeros coeficientes (pueden aparecer entre 20 y 40, pero no son muy representativos).
12. **Asimetría (skewness):** Mide hacia qué lado se inclina la distribución de la energía en un conjunto de datos. Si hay una asimetría positiva, la energía se concentra en valores bajos, con una cola larga hacia los valores altos. En cambio si hay una asimetría negativa, la energía se concentra en valores altos, con una cola larga hacia los valores bajos. Si no hay asimetría, la energía está equilibrada alrededor del centro (como una campana de Gauss). Se calcula restando la media a cada muestra y elevando al cubo, luego hallando el promedio de estas desviaciones cúbicas, y, finalmente, dividiendo por la desviación

estándar al cubo para estandarizar. Cabe mencionar que unas distribuciones de amplitud asimétricas pueden indicar características anómalas en voces sintéticas.

13. **Curtosis:** Mide qué tan concentrados están los valores alrededor de la media y qué tan pesadas son las colas de la distribución. En audio, la curtosis nos dice qué tan “espigado” o “suave” es un sonido en términos de cómo se distribuyen sus amplitudes (en el tiempo) o su energía (en la frecuencia). Se obtiene con la siguiente fórmula
$$\text{Curtosis} = \frac{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^4}{\left(\sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2} \right)^4}.$$
 Adicionalmente, se puede restar 3 a la curtosis para que la mitad (mesocúrtica) se halle en 0. Podemos destacar que una curtosis alta (distribución picuda) puede aparecer en audios con compresión excesiva o artefactos de síntesis.
14. **Transcripción:** Para la transcripción de los audios, utilizamos modelos de [Vosk](#). Esta elección se basó en que tiene modelos para los dos idiomas que vamos a investigar y además, dos versiones dentro de cada lenguaje. El que usamos para las pruebas es el small ya que es más rápido y por lo tanto más útil para esta primera fase pero dejamos en consideración el modelo full para trabajo futuro ya que tiene mayor precisión. Sobre la implementación, cabe mencionar que primero probamos Whisper, el modelo de reconocimiento automático del habla desarrollado por OpenAI, pero tras varios problemas durante su implementación, lo descartamos y escogimos vosk en su defecto para esta tarea. Es interesante tener en cuenta la transcripción de un audio ya que en ocasiones, si no podemos sacar texto en claro, pero se sabe que hay voces, puede ser un primer indicio de que el audio es generado.

3.3. Métricas de evaluación

Antes de entrar a hablar de los modelos, vamos a ver qué métricas de evaluación usaremos en estos, ya que son la pieza clave para entender qué tan bien está funcionando un modelo concreto. Métricas de evaluación hay muchas, e incluso distintas según distintos problemas (regresión, clasificación, etc.). Las principales y más conocidas son la exactitud (*accuracy*), f1, y Error Cuadrático Medio (RMSE), aunque hay muchas más como hemos mencionado. Vamos a listar todas las métricas que usamos, qué es lo que evalúan y en qué modelos las usamos (ya que algunas son tan importantes que se usan en casi todos los modelos). Es importante mencionar que, ya que nuestro problema es de clasificación, no usaremos algunas métricas de regresión como el RMSE, pues no podemos calcularlas ni tiene sentido.

Matriz de confusión

Vamos a introducir también la matriz de confusión y el concepto de aciertos y fallos, ya que es fundamental para entender mejor las métricas. Un modelo puede hacer dos cosas, acertar y fallar, y, a su vez, los aciertos y los fallos pueden deberse a que la instancia fuera negativa y positiva. Con esto tenemos los siguientes conceptos:

- **Verdaderos Positivos (VP):** Instancias positivas predichas como positivas.
- **Verdaderos Negativos (VN):** Instancias negativas predichas como negativas.
- **Falsos Positivos (FP):** Instancias negativas predichas como positivas. También llamado Error tipo I.
- **Falsos Negativos (FN):** Instancias positivas predichas como negativas. También llamado Error tipo II.

Las matrices de confusión toman esos cuatro valores y los distribuyen en: eje y los valores reales, eje x los valores predichos y se forma una matriz $N \times N$, siendo N el número de clases que definamos. En nuestro caso, y en la mayoría de casos, se da una matriz 2×2 . También es importante entender que la diagonal principal siempre guarda los valores verdaderos (los aciertos de predicción). Con la siguiente imagen queremos ilustrar cómo se ve una matriz de confusión 2×2 . Normalmente se pone la clase negativa arriba/a la izquierda de la clase positiva, aunque podrían estar al revés, pero siempre manteniendo lo que comentamos de la diagonal principal.

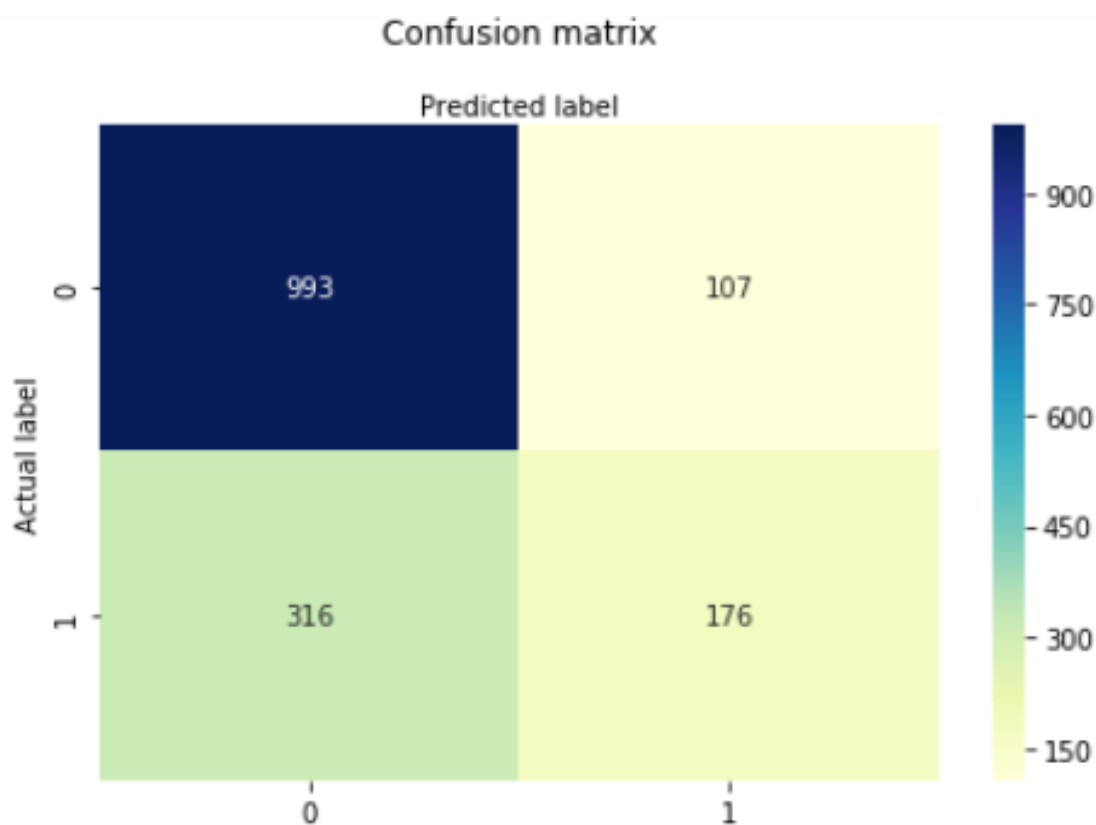


Figura 3.1: Ejemplo real de matriz de confusión.

Métricas usadas

Veamos entonces con lo que ya sabemos qué métricas usamos y qué miden.

- **Exactitud (*accuracy*):** Mide la proporción total de predicciones correctas (tanto positivas como negativas) sobre el total de casos. Es útil cuando las clases están balanceadas, pero puede ser engañosa si hay un desbalance (ej. 95 % de una clase y 5 % de otra, predecir siempre la mayoritaria daría un 95 % de *accuracy*). Su fórmula es: $\frac{VP+VN}{VP+VN+FP+FN}$. Lo usamos en Random Forest, LightGBM, SVM, ...
- **Precisión:** Mide la relación de las clases predichas como positivas frente al total de valores reales positivos. Es una métrica que se enfoca en controlar los Falsos Positivos (FP). Es útil cuando el costo de un FP es alto (por ejemplo, marcar un correo legítimo como spam). Su fórmula es: $\frac{VP}{VP+FP}$. Podría calcularse la precisión para la clase negativa también. Lo usamos en Random Forest, LightGBM, SVM, ...

- **Exhaustividad (*recall*)**: Mide la cantidad de valores positivos que el modelo logró predecir correctamente. Es una métrica que se enfoca en minimizar los Falsos Negativos (FN). Es útil cuando el costo de un FN es alto (por ejemplo, no detectar una enfermedad grave). Su fórmula es: $\frac{VP}{VP+FN}$. Podría calcularse el recall de la clase negativa también. Lo usamos en Random Forest, LightGBM, SVM, ...
- **F1-Score**: Es la media armónica entre la Precisión y el Recall. Busca un equilibrio entre ambas métricas. Es una métrica más robusta que la Accuracy cuando hay clases desbalanceadas, ya que penaliza los extremos (una precisión alta con recall bajo, o viceversa). Es útil cuando quieres una sola métrica que resuma el rendimiento en la clase positiva. Su fórmula es: $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$. Lo usamos en Random Forest, LightGBM, SVM, ...
- **Área bajo la curva ROC (ROC-AUC)**: Mide la capacidad discriminativa del modelo, independientemente del umbral de decisión elegido. La curva ROC (Receiver Operating Characteristic) grafica la Tasa de Verdaderos Positivos (Recall) frente a la Tasa de Falsos Positivos. La fórmula de la curva es: $\frac{FP}{FP+VN}$. El AUC (Area Under the Curve) es el área bajo esta. Lo usamos en Random Forest, LightGBM, SVM, ...
- **Coeficiente de Correlación de Matthews (MCC)**: Es esencialmente un coeficiente de correlación entre las etiquetas reales y las predicciones. A diferencia de las métricas anteriores (que a veces solo se enfocan en la clase positiva), el MCC considera los cuatro cuadrantes de la matriz de confusión (VP, VN, FP, FN). Se considera una métrica equilibrada incluso con conjuntos de datos muy desbalanceados. Su valor oscila entre -1 y 1 siendo -1 una discrepancia total entre predicción y realidad y 1 una predicción perfecta. Su fórmula es: $\frac{VP \cdot VN - FP \cdot FN}{\sqrt{(VP+FP)(VP+FN)(VN+FP)(VN+FN)}}$. Lo usamos en Random Forest, LightGBM, SVM, ...

3.4. Proceso común de experimentación

En este apartado vamos a hablar de los procesos comunes que vamos a usar en varios (dos o más) modelos, para evitar repetir explicarlos múltiples veces.

3.4.1. División de los datos

Sobre el dataset que ya hemos comentado, hemos tomado un subset de 100 audios de cada idioma. La mitad son audios naturales y la otra mitad audios generados. De estos 100 audios entrenaremos con 80 audios y probaremos los modelos con 20, de cambiar la partición lo

mencionaremos explícitamente, pero un 80-20 será lo normal. Más adelante, probaremos a usar más cantidad de audios y compararemos el rendimiento.

3.4.2. Validación Cruzada y Grid Search

En algunos modelos aplicamos Validación Cruzada Estratificada y/o Grid Search para ver la estabilidad de un modelo y buscar los mejores parámetros de un modelo, respectivamente.

La Validación Cruzada Estratificada, también llamada *stratified k-fold*, lo que hace es dividir los datos en k conjuntos o *folds*. Entonces entrena el modelo que le hayamos pasado con $k-1$ conjuntos, y lo evalúa con el conjunto restante según una métrica que le hayamos pasado a la función de validación cruzada. Hace esto k veces, una por cada conjunto (cada conjunto pasa por validación exactamente 1 vez y por entrenamiento exactamente $k-1$ veces). Al acabar los k entrenamientos se promedian los resultados y se calcula la desviación típica de los mismos, devolviendo estos valores para ver el rendimiento del modelo. En general, si la desviación típica es baja, significa que el modelo es estable en los distintos conjuntos que ha creado de los datos. No son resultados absolutos, ya que hay muchos más conjuntos posibles, pero es una mejor aproximación que un único conjunto de entrenamiento y test.

El Grid Search lo que hace es buscar la mejor combinación de parámetros de un modelo. Tenemos que definir previamente un diccionario o *grid* donde las claves son los parámetros del modelo y los valores arrays con los posibles valores que dicho parámetro puede tomar. Esto entrena un modelo con cada posible combinación de parámetros, y devuelve el mejor modelo (mejor combinación) según una métrica que le hayamos definido previamente. El Grid Search funciona con validación cruzada (se puede reutilizar la que hayamos creado para la validación persé). Además, hay que pasarle una métrica a optimizar, como hemos mencionado, que en nuestro caso será el F1-Score.

Hay que tener en cuenta que estos procesos toman bastante tiempo, ya que tienen que entrenar múltiples veces, por lo que es posible que solo los apliquemos a modelos cuyo entrenamiento base no tarde tanto, como Random Forest, o LightGBM

3.4.3. Funciones de explicación

Para algunos modelos, los que usan datos tabulares hemos diseñado unas funciones generales de XAI las cuales generan la explicación sin saber cual es el modelo. Esto funciona gracias a que los modelos de XAI que usamos aquí (SHAP, LIME, DiCE) no son específicas para un modelo, es decir, son *model agnostic*, por lo que funcionan para varios. También hemos hecho una función que permite eliminar ciertas características de la lista de características de entrenamiento

además de generar una explicación de cualquier modelo de XAI que hemos mencionado en este apartado. Para esto es necesario reentrenar el modelo, por lo que es importante que sea un modelo de “entrenamiento rápido”, como los mencionados en el apartado anterior. Además, ya que volvemos a entrenar el modelo, podemos volver a ver el rendimiento de este si así lo queremos para ver qué tal lo hace sin algunas características.

3.4.4. Estado aleatorio

Un parámetro muy importante si queremos que nuestros experimentos sean reproducibles es el estado aleatorio o *random state*. Este parámetro establece una semilla generadora, dicha semilla sirve para generar los valores que sean aleatorios como en separación y mezcla de datos o en modelos que se inicializan aleatoriamente como KNN. Cuando declaramos el parámetro estamos forzando al modelo/algoritmo a usar dicha semilla, no cualquiera. Por este motivo si queremos que nuestros resultados sean reproducibles, es decir, sean iguales cada vez que lo ejecutamos, necesitamos definir el estado aleatorio. Lo hemos establecido, trivialmente, al valor 1221182

3.5. Modelos

En esta subsección vamos a analizar qué modelos hemos usado para resolver el problema de detección de voz generada por IA. Es importante recalcar que distintos modelos pueden usar distintos tipos de datos (tabulares, imágenes, etc.). Además, un mismo modelo puede usar varios tipos de datos a la vez para entrenar. Antes de describir los modelos que hemos usado, cómo los hemos aplicado y cuáles han sido sus resultados, en la siguiente lista, se indica el o los tipos de datos que usa cada modelo.

- **Random Forest:** Datos tabulares.
- **LightGBM:** Datos tabulares.
- **SVM:** Datos tabulares.
- **AASIST:** Onda de audio cruda.
- **MLP:** Datos tabulares.
- **Wav2Vec2:** Onda de audio cruda.
- **Regresión Logística:** Datos tabulares.
- **KNN:** Datos tabulares.

3.5.1. Random Forest

Como hemos comentado sobre los Random Forest en la [Subsección 2.1.3](#), estos construyen un conjunto o “bosque” de árboles de decisión donde cada árbol se entrena de forma ligeramente diferente, introduciendo aleatoriedad controlada.

Justificación del modelo

Se seleccionó Random Forest como uno de los clasificadores base debido a su robustez frente al sobreajuste, su capacidad para manejar características heterogéneas y su interpretabilidad mediante el análisis de importancia de variables.

Configuración experimental

El proceso experimental para Random Forest se estructuró en tres fases: (1) establecimiento de un modelo baseline, (2) validación cruzada para evaluar robustez, (3) optimización de hiperparámetros mediante búsqueda sistemática.

División de datos

El conjunto de datos se particionó en entrenamiento (80 %) y prueba (20 %) utilizando muestreo estratificado para mantener la proporción de clases en ambos conjuntos. Esta estratificación es crucial cuando existe desbalance entre voces reales y generadas.

Modelo baseline

Se estableció una configuración inicial con hiperparámetros por defecto para obtener una referencia de rendimiento:

- `n_estimators`: 100.
- `max_depth`: 10.
- `min_samples_split`: 5.
- `min_samples_leaf`: 2.
- `max_features`: `sqrt`.
- `class_weight`: `balanced`.

Validación cruzada estratificada

Para evaluar la robustez del modelo y evitar que los resultados dependan de una única partición de los datos, se realizó validación cruzada estratificada con 5 folds, como se comentó en la [Sección 3.4](#).

Optimización de hiperparámetros

Se realizó una búsqueda sistemática de hiperparámetros mediante `GridSearchCV` con validación cruzada de 5 folds, optimizando el F1-score. El espacio de búsqueda incluyó:

- **n_estimators**: Probamos los valores 50, 100 y 200.
- **max_depth**: Probamos los valores 5, 10, 15.
- **min_samples_split**: Probamos 2, 5 y 10.
- **min_samples_leaf**: Probamos 1, 2 y 4.
- **max_features**: Probamos “sqrt” y “log2”. Este último toma el logaritmo en base 2 del número de características.

3.5.2. LightGBM

Como hemos visto en la [Subsección 2.1.4](#), a diferencia de Random Forest, que construye árboles de forma independiente y paralela, LightGBM los construye secuencialmente, donde cada nuevo árbol se especializa en corregir los errores de los anteriores.

Justificación del modelo

Hemos usado LightGBM debido a que presenta ventajas significativas para la tarea de detección de deepfakes de audio, como pueden ser su eficiencia computacional para grandes volúmenes de datos o su regulación integrada con los parámetros L1 y L2.

Configuración experimental

El proceso experimental para LightGBM se estructuró en tres fases: (1) establecimiento de un modelo baseline, (2) validación cruzada para evaluar robustez, y (3) optimización de hiperparámetros mediante búsqueda sistemática.

Preparación de datos

A diferencia de otros algoritmos, LightGBM puede manejar directamente datos sin normalización, aunque requiere que no existan valores nulos.

Modelo baseline con LightGBM

Se estableció una configuración inicial utilizando la interfaz nativa de LightGBM, que ofrece un control más fino sobre el proceso de entrenamiento. Los parámetros que usamos para el modelo baseline fueron los siguientes.

- **objective:** binary.
- **metric:** binary logloss.
- **boosting_type:** gbdt.
- **num_leaves:** 31.
- **learning_rate** η : 0,05.
- **feature_fraction:** 0,9.
- **bagging_fraction:** 0,8.
- **bagging_freq:** 5.
- **random_state:** 12.
- **is_unbalanced:** True.

El parámetro `is_unbalance=True` activa la compensación automática para clases desbalanceadas, ajustando los pesos de forma inversamente proporcional a la frecuencia de cada clase.

Tras ello, entrenamos el modelo con los parámetros base, los datos de entrenamiento sin escalar, un set de validación, y los siguientes parámetros:

- **num_boost_round:** Es el número máximo de iteraciones (árboles) que se entrenarán. En este caso, 1000 es el límite superior, pero el `early_stopping` puede detener el proceso antes si no hay mejoras. Este último permite prevenir el sobreajuste, ahorrar tiempo computacional y determinar automáticamente el número óptimo de iteraciones.
- **callbacks:** funciones que se ejecutan durante el entrenamiento para controlar el proceso. En este caso mostramos información del entrenamiento cada 100 iteraciones y paramos si no mejora por 50 iteraciones.

Validación cruzada estratificada

Para LightGBM se utilizó la interfaz compatible con scikit-learn (`LGBMClassifier`) que permite integrarse con las herramientas de validación cruzada estándar. El modelo que usamos es el mismo que el baseline (en cuanto a parámetros). Tras esto aplicamos la validación cruzada tal y como hemos visto en la [Sección 3.4](#)

Optimización de hiperparámetros

LightGBM cuenta con un conjunto de hiperparámetros específicos, diferentes a los de Random Forest. La búsqueda se centró en aquellos que más influyen en el rendimiento, según lo visto en la [Subsección 2.1.4](#). Estos difieren de los que declaramos en el modelo baseline, esta decisión es debido a que vamos a optimizar estos hiperparámetros por ser más influyentes.

- `num_leaves`: Probamos 15, 31 y 63.
- `learning_rate`: Probamos 0,01, 0,05 y 0,1.
- `n_estimators`: Probamos 100, 200 y 300.
- `min_child_samples`: Probamos 20, 50 y 100.
- `subsample`: Probamos 0,6, 0,8 y 1.
- `colsample_bytree`: Probamos 0,6, 0,8 y 1.
- `reg_alpha`: Probamos 0, 0,1 y 0,5.
- `reg_lambda`: Probamos 0, 0,1 y 0,5.

3.5.3. Support Vector Machine (SVM)

Como ya hemos visto sobre los SVM en la [Subsección 2.1.5](#), estos se fundamentan en la búsqueda del hiperplano óptimo que maximiza el margen de separación entre clases, lo que proporciona una base teórica sólida y garantías de generalización.

Justificación del modelo

Usamos SVM ya que presenta características especialmente relevantes para la detección de deepfakes de audio que hemos comentado antes, como la eficacia en espacios de alta dimensión (como puede ser el mundo del lenguaje natural) o su fundamento teórico robusto.

Importancia de la normalización

Una característica crítica de SVM es su sensibilidad a la escala de las características. Las variables de audio presentan escalas muy diferentes:

- **Formantes:** Del orden de cientos de Hz (ej. 300-3000 Hz)
- **MFCC:** Valores típicamente en el rango $[-200, 200]$
- **ZCR** (Zero Crossing Rate): Valores normalizados entre 0 y 1
- **HNR** (Harmonics-to-Noise Ratio): Valores en decibelios (dB)

Sin normalización, las características con mayor magnitud dominarían el cálculo de distancias, sesgando el modelo. Por ello, se aplicó `StandardScaler` para estandarizar las características a media cero y desviación típica uno.

Es crucial aplicar el escalado *después* de la división train/test para evitar *data leakage* (aportar información extra al conjunto de prueba, proveniente del conjunto de entrenamiento).

Configuración experimental

El proceso experimental para SVM se estructuró en cuatro fases: (1) modelo baseline con kernel lineal, (2) modelo con kernel RBF, (3) validación cruzada, y (4) optimización de hiperparámetros mediante Grid Search.

Modelo baseline: kernel lineal

Se estableció un primer modelo con kernel lineal como referencia simple:

- **kernel:** linear.
- **C:** 1.
- **probability:** True.
- **random_state:** 12.
- **class_weight:** balanced.

Modelo baseline: kernel RBF

Dada la naturaleza compleja de las señales de audio, se implementó un modelo con kernel RBF para capturar relaciones no lineales:

- **kernel:** RBF.
- **C:** 1.

- **gamma**: scale.
- **probability**: True.
- **random_state**: 12.
- **class_weight**: balanced.

El parámetro `gamma='scale'` establece automáticamente $\gamma = 1/(n_{\text{características}} \cdot \text{Var}(X))$, un valor inicial razonable.

Validación cruzada estratificada

Para garantizar la robustez de las evaluaciones y evitar dependencias de una única partición, se implementó validación cruzada estratificada con 5 folds, como explicamos en la [Sección 3.4](#). Se utilizó un pipeline que integra el escalado y el clasificador para asegurar que la normalización se realice correctamente en cada fold.

Optimización de hiperparámetros

Para buscar la mejor configuración de hiperparámetros para SVM nos hemos centrado en el kernel RBF, ya que es más usado que el kernel lineal, como vimos en la [Subsección 2.1.5](#). SVM con kernel RBF tiene dos hiperparámetros críticos que determinan su rendimiento (C y γ). La búsqueda sistemática se realizó mediante Grid Search con validación cruzada sobre dichos parámetros:

- **C**: Probamos 0,1, 1, 10 y 100.
- **gamma**: Probamos 0,001, 0,01, 0,1, 1, “scale” y “auto”.
- **kernel**: Probamos únicamente el kernel no lineal “rbf”.

3.5.4. AASIST

Como vimos en la sección [2.1.6](#), AASIST es un modelo creado con la motivación de combinar pistas espectrales y temporales para detectar artefactos de spoofing y conseguir robustez frente a una gran variedad de ataques.

Justificación del modelo

La elección de AASIST se justifica por tratarse de un modelo desarrollado específicamente para la detección de “spoof” en audio. Su arquitectura está diseñada para explotar de forma conjunta información temporal y espectral, lo que resulta especialmente útil en un problema

donde las diferencias entre señales naturales y generadas pueden ser sutiles y estar distribuidas. Además, su buen desempeño en tareas de “*anti-spoofing*” lo convierte en una referencia sólida para ser utilizado en este trabajo.

Aplicación del modelo

Lo explicado en esta subsección de aplicación se corresponde con lo desarrollado en este “[notebook](#)”.

Lo primero, al ser un modelo que produce una predicción a partir del audio crudo, el primer paso es definir las rutas a los audios. Posteriormente, se lee el contenido del **archivo .conf** del modelo. Para ello, hemos creado una función que busca el elemento pasado por parámetro y devuelve el bloque superior que contiene ese elemento junto con los elementos al mismo nivel. Esto nos permite construir una variable, que vamos a llamar `args`, con los siguientes campos:

- **first_conv**: configuración del **kernel** de la primera capa de convolución, es decir, cuántas muestras de audio se utilizan para cada filtro. En este caso, 128.
- **flts**: es un array de elementos siendo el primero el número de **filtros** en la primera capa de convolución. Los siguientes elementos son pares para cada bloque residual con el número de filtros de entrada y de salida.
- **gat_dims**: tamaño del **embedding** que representa cada nodo. Tiene dos valores que hacen referencia al tamaño al construir los grafos de atención y al tamaño en las fases de “HS-GAL” + “pooling” respectivamente.
- **pool_ratios**: se usa en la técnica de “graph pooling” y significa el **ratio de reducción de nodos** en cada capa. Los dos primeros valores se usan para reducir los grafos espectral y temporal, respectivamente, antes de combinarlos en el grafo heterogéneo. El siguiente valor se usa para reducir el grafo heterogéneo después de cada capa HS-GAL. Aunque existe un cuarto valor, en el modelo final no se usa.
- **temperatures**: controla lo suave o agresiva que es la **atención** en cada fase. Cuanto más bajo es este parámetro, mayor peso se concentra en pocos nodos vecinos.

Con `args` ya creado, instanciamos el modelo. Posteriormente, se usa el checkpoint para cargar los pesos preentrenados.

El siguiente paso es definir las funciones de inferencia. Para esto, nos apoyamos primero en dos funciones auxiliares, una que convierte los audios a un **formato estándar**, un solo canal y 16 kHz, y otra que crea una **ventana** para recorrer audios largos. Esta última es importante ya que AASIST está diseñado para recibir segmentos de longitud de unos 4 segundos. Con esto

preparado, creamos las tres funciones de inferencia para ver si conseguíamos mejores resultados alterando un poco el proceso. Las tres funciones se describen a continuación:

- **average_windows**: esta función devuelve la media de sumar las predicciones de cada ventana calculadas de forma independiente.
- **score_without_windows**: esta función devuelve solamente la puntuación del segmento inicial del audio en archivos largos o del audio en su totalidad en el caso contrario.
- **average_logits**: esta función fusiona todas las ventanas sumando sus “logits” antes de calcular una única probabilidad.

Finalmente, se aplica el modelo y se vuelcan los resultados en un csv con el nombre del audio, la probabilidad de que sea “bonafide” y su duración. Para esto, se recorre cada audio del dataset, se le aplica una de las funciones de inferencia y se escribe la información en el csv.

Explicación de los modelos XAI aplicados a AASIST

Para explicar esta subsección, nos apoyamos en lo desarrollado en este [“notebook”](#).

Enfoque de explicabilidad ante-hoc

Hemos usado las propias estructuras del modelo para empezar a explicar qué es lo que le empuja a decidir una clase u otra. Esto nos sirve además para enseñar mejor cómo funciona la arquitectura de AASIST. Lo primero que mostramos es el formato de la salida que es una tupla con dos valores, la probabilidad de que el audio sea “spoof” y de que sea “bona fide”, y la estructura de los grafos que nos permite ver que se crean 23 nodos espectrales y 29 temporales. También, simulamos cómo funciona un “pooling” mostrando los nodos de cada tipo con mayor puntuación.

Después, ejemplificamos cuáles son los valores que se pasan a la capa de salida y cuánto afectan al cálculo de cada uno de los valores de la salida. Estos valores son los resultados de las operaciones “max” y “average” de cada subgrafo y el resultado del stack node. Con estas muestras podemos empezar a ver qué ha sido más significativo a la hora de responder si un audio es “spoof” o no.

En relación a filtros de frecuencia, primero calculamos la banda de frecuencias aproximada de cada uno de los 70 filtros con una transformada rápida de Fourier y las mostramos en una [gráfica](#). Con ella podemos afirmar que se captan todas las frecuencias de 0 a 8000 Hz y que en las frecuencias bajas hay más solapamiento entre filtros. Esto es importante ya que cuando extraemos cuáles son los filtros que más se activan durante el procesamiento del audio, no

es raro que salgan algunos consecutivos ya que si dos filtros están observando una frecuencia muy activa, ambos responderán parecido.

Posteriormente, representamos en una gráfica los filtros que más activación han tenido en el primer procesado, de dos formas: en la primera [imagen](#), viendo qué frecuencias cubren, y en la [segunda](#), viendo su activación a lo largo del audio.

Con la primera gráfica, se observa que los 5 filtros más excitados se pueden separar en dos grupos por su consecución, mientras que en la segunda, se aprecia cómo tienen comportamientos muy parecidos.

En relación con las frecuencias, simulamos cómo sería la salida de la capa “**sinc-convolution**” que podemos asemejar a un espectrograma. Para conseguir este mapa filtro-tiempo capturamos la salida de la capa durante la inferencia y la representamos en un “**heatmap**” en cuyo eje Y se colocan los filtros de menor a mayor banda de frecuencia.

Hemos creado dos mapas filtro-tiempo, el [primero](#) sería el más fiel al modelo, mientras que el [segundo](#) normaliza la activación de cada filtro por su máximo. Esta combinación nos permite determinar qué regiones de frecuencias se excitan más pero sin ocultar completamente a las menos determinantes.

Con estos mapas, podemos ver qué filtros, es decir, bandas de frecuencia se activan más y cuándo se excitan más. Asimismo, es importante mencionar que se está representando la magnitud absoluta de atribución, no signo.

Implementación de Occlusion

A continuación, vamos a aplicar Occlusion, técnica explicada en la sección [2.2.8](#), tanto al ámbito espectral como al temporal. Esta técnica consiste en ir “tapando” fragmentos de la entrada y ver cómo cambia el “logit” final con estas variaciones para ver qué tramos han afectado más. Para tapar, en el ámbito temporal hemos puesto silencio en el tramo objetivo y en el ámbito frecuencial, hemos puesto silencio en la banda de frecuencia objetivo.

Además, vamos a utilizar dos medidas complementarias. Por un lado, el cambio en el “**logit**” de la clase objetivo nos permite representar cuánto contribuye cada región a la evidencia bruta de dicha clase. Por otro, medimos el **margen** o diferencia entre los “logits” de ambas clases; esta métrica nos permite analizar separación entre clases durante el audio. Mientras que la primera aporta información importante sobre cierta clase, el margen resulta más valioso para entender la decisión final de AASIST ya que nos muestra si realmente una región ayuda a la clase objetivo o ayuda a ambas por igual.

A la hora de llevar esta idea a código, hemos creado dos funciones separadas para cada dominio pero su idea es la misma. Primero, sacan los “logits” iniciales, seguidamente, con un bucle vamos tapando segmentos y sacando nuevos “logits” que nos permiten ver cómo ha variado el resultado al obviar ese segmento. Ambas funciones tienen un parámetro que permite decidir qué clase observar y en caso de no especificar, se mira la clase con la que se clasificó el audio. La única diferencia entre estas funciones, es que la del plano espectral necesita hacer uso de una FFT para poder cambiar entre ámbitos y así seleccionar correctamente qué banda de frecuencia silenciar.

Las caídas en la probabilidad las mostramos en una gráfica y nos permite ver qué instantes del audio son más relevantes para el modelo. Hemos usado instantes de 10 ms ya que hemos concluido que es un punto donde ya podemos ver cambios finos sin que los picos en tramos pequeños se compensen y obtengamos aportación nula.

Tenemos como resultado las gráficas temporales de [“logit”](#) y de [margen](#). Aunque con ellas todavía no sabemos qué fenómenos son los determinantes, sí nos permite saber en qué momento ocurren.

Con las gráficas frecuenciales, podemos ver en qué frecuencias ocurren. Asimismo, tenemos las mismas dos gráficas, una de [“logit”](#) y otra de [margen](#).

Implementación de Integrated Gradients

La siguiente técnica que vamos a aplicar es la explicada en la sección [2.2.4](#). Para medir el error usando esta técnica usamos la métrica “delta” que devuelve la propia clase. Este valor mide la diferencia entre la variación real de la salida del modelo entre el input y el baseline y la suma de las atribuciones obtenidas. Un valor cercano a 0 significa buena convergencia. Primero, vamos a aplicar IG sobre los mapas filtro-tiempo que acabamos de ver. Para ello, usamos la clase **LayerIntegratedGradients** porque analizamos solo una capa concreta del modelo y nos devuelve la salida en el mismo formato que la entrada, lo que nos permite representarla de la misma manera. Esta clase usa los siguientes parámetros:

- **inputs**: el tensor del propio audio.
- **baselines**: utilizamos un vector silencio ya que la entrada es un audio.
- **target**: la clase objetivo.
- **n_steps**: número de saltos entre el baseline y la entrada real.
- **return_convergence_delta**: si es “True”, nos devuelve la métrica delta.
- **attribute_to_layer_input**: si es “False”, las atribuciones se calculan sobre las activaciones que salen de esta capa.

A diferencia de los mapas filtro-tiempo mostrados anteriormente, aquí representamos las zonas que más **contribuyen** al “logit” final. Sin embargo, igual que antes, representamos una gráfica [normalizada](#) para apreciar más detalle y una [sin normalizar](#) para ver el mapa puro.

Para mostrar esto de manera más visual, generamos una [gráfica](#) donde vemos los filtros en el eje x, proporcionando información sobre los más excitados.

Siguiendo con IG, ahora lo aplicamos sobre el **audio crudo**, usando, a diferencia de con los mapas, la clase **IntegratedGradients**. Esta clase recibe los mismos parámetros que la usada antes a excepción de “attribute_to_layer_input”.

Como resultado, obtenemos una [gráfica](#) de importancia temporal indicando lo importante que es cada instante según esta técnica.

Implementación de RISE

A continuación aplicamos RISE, sección [2.2.9](#), y obtenemos los resultados en forma de gráfica temporal. Para transformar la idea de esta técnica a audio, lo primero es generar una función que genere las máscaras aleatorias. Esta función genera una primera máscara de tamaño “grid_size” formada solo por 1 o 0. Cada posición de esta primera máscara tiene una probabilidad “p_keep” de tener valor 1, es decir, conservar sin alteraciones esa parte de la señal. Entre cada uno de estos puntos, se realiza un escalado suave hasta tener una máscara del mismo tamaño que la señal original, evitando cambios bruscos que provoquen discontinuidades artificiales.

En cuanto a la función principal, podemos realizar los cálculos de dos formas ya mencionadas anteriormente, el “logit” de clase objetivo o el margen entre clases. Para estos cálculos, primero creamos dos vectores de acumulación y después iteramos “n_mask” veces. En cada iteración, generamos una nueva máscara, la aplicamos y, una vez tenemos el audio normalizado, recalculamos resultados con el modelo. Posteriormente, calculamos ambas métricas y las sumamos, multiplicadas por la máscara, a sus vectores de acumulación correspondientes. Esta multiplicación es la que va a ir discriminando qué instantes de audio son los que favorecen más a cada métrica.

Por último, con el bucle acabado, aplicamos a los acumuladores una media sobre el número de máscaras multiplicado por el valor “p_keep”.

Mostramos las gráficas de ambas métricas, “[logit](#)” y [margen](#) para poder compararlas como se ha mencionado anteriormente.

Implementación de SHAP

El último modelo de XAI que hemos aplicado en AASIST es SHAP, sección 2.2.1. Sin embargo, no lo hemos podido aplicar directamente debido a dos razones principales. En primer lugar, el modelo recibe como entrada una onda de audio cruda, lo que se traduce en muchísimas variables, algo poco manejable para esta técnica. En segundo lugar, la arquitectura de AASIST, con técnicas como el bloque MGO, no es compatible de forma natural con los explicadores de SHAP. Incluso obviando estas razones, como resultado de aplicar SHAP sobre AASIST, obtendríamos la importancia para miles de muestras individuales del audio y no características interpretables, como es el objetivo.

Para abordar este problema, hemos usado dos modelos surrogate: “**XGBRegressor**” y “**XGBClassifier**”. Se optó por utilizar ambos ya que al no poder aplicar directamente SHAP sobre AASIST, resulta útil comparar ambas aproximaciones y obtener resultados más fiables. La diferencia principal entre ambos es que intentan aproximar distintos comportamientos de nuestro modelo de clasificación. El regresor se planteó con la idea de reproducir la **salida continua**, es decir, el logit final, aportando una visión más detallada de la respuesta de AASIST. En cambio, el clasificador solo pretende imitar la **clasificación binaria** final, por lo que su entrenamiento resulta más sencillo, aunque aporta información más limitada.

Antes de mostrar los resultados, vamos a explicar el preprocesado de los datos común a ambos modelos. En primer lugar, combinamos el csv de características de los audios y el csv de resultados de AASIST. Para ello, hacemos un merge de ambos archivos poniendo especial atención en que las rutas coincidan y para ello creamos una función de normalización que unifica su formato.

Cabe mencionar que transformamos el score del segundo csv a “logit” con la intención de intentar mejorar la imitación del modelo ya que los valores en “logit” crecen de manera más lineal y no están cerrados entre 0 y 1.

Posteriormente, eliminamos del conjunto resultante aquellas columnas de características que no queremos que se usen en la entrada del surrogate como el nombre de cada archivo. Asimismo, hemos eliminado las columnas no numéricas ya que ambos modelos requieren una entrada de solo datos numéricos. Estos datos son pitch, f1, f2, f3 y los distintos MFCC. Aunque estas ausencias son importantes a la hora de interpretar los resultados, en el conjunto de datos sí se encuentran derivadas de estas características como la media, por lo que el modelo si recibe información de ellas. Finalmente, realizamos la división del dataset con la función “train_test_split”.

Por un lado, para estimar la fiabilidad del clasificador, hemos usado las métricas: precision, recall y F1. Como puede observarse en la salida de la celda donde utilizamos el modelo, ninguna

de las métricas descende de 0,92 al calcular la media entre ambas clases. En conjunto, estos datos sugieren que este primer modelo surrogate constituye una aproximación razonable al comportamiento de AASIST.

Por otro lado, para medir la precisión del regresor a la hora de imitar correctamente usamos la métrica R^2 . En este contexto, R^2 mide qué tan bien el modelo del surrogate reproduce correctamente la salida original de AASIST:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

donde y_i es el valor real, $\hat{y}_i$ el valor predicho por el surrogate, $\bar{y}$ la media de los valores reales.

Esta fórmula se puede presentar de forma más intuitiva como:

$$R^2 = 1 - \frac{\text{error del surrogate}}{\text{variabilidad total de la salida}}$$

Por tanto, cuanto más cercano sea R^2 a 1, mejor será la capacidad del surrogate para imitar el comportamiento de AASIST. En cambio, un valor cercano a 0 indica que el modelo apenas mejora una predicción basada simplemente en la media.

Como se puede observar en la salida de la celda donde se entrena el surrogate, obtenemos un R^2 al testear de 0,66 lo que indica que sí consigue capturar una parte importante del comportamiento de AASIST aunque no de forma completamente precisa. Pese a esto, consideramos útil utilizar sus resultados para compararlos con los del clasificador ya que aportan una visión complementaria.

Como modelos, hemos escogido **XGBRegressor** y **XGBClassifier**. Además de por ser compatible con SHAP, esta elección se sustenta en tres motivos:

- Presentan una gran flexibilidad de ajuste, ya que permiten controlar la complejidad del modelo con los hiperparámetros que se explican más adelante.
- Están basados en “**gradient boosting**”, una técnica que les permite capturar relaciones no lineales entre variables. Esta técnica consiste en entrenar sucesivos árboles de decisión que corrigen los errores cometidos por los anteriores. De este modo, combina estimadores simples para aproximar relaciones complejas entre variables.
- No requieren un preprocesado excesivo de los datos acústicos, lo que nos permite utilizar directamente los archivos “.csv” de características ya calculados para otros modelos.

En lo referido a los parámetros de entrenamiento ambos modelos reciben los mismos:

- **n_estimators**: número de árboles del modelo. Aumentarlo puede mejorar la capacidad de ajuste, aunque también puede favorecer el sobreajuste.
- **max_depth**: profundidad máxima de cada árbol. Controla la complejidad de las reglas que puede aprender el modelo.

- **learning_rate**: peso o contribución de cada árbol nuevo al resultado final. Valores bajos hacen el aprendizaje más gradual.
- **subsample**: porcentaje de audios o muestras que utiliza cada árbol durante el entrenamiento. Ayuda a introducir variedad y a reducir el sobreajuste.
- **colsample_bytree**: porcentaje de características que usa cada árbol. Permite que no todos los árboles dependan exactamente de las mismas variables.
- **reg_lambda**: término de regularización L_2 . Penaliza pesos grandes y ayuda a controlar el sobreajuste.
- **reg_alpha**: término de regularización L_1 . Penaliza pesos pequeños y favorece modelos más simples.
- **min_child_weight**: controla la creación de nodos nuevos, evitando divisiones poco relevantes.
- **gamma**: coste mínimo necesario para realizar una partición en un nodo. Cuanto mayor es, más conservador es el crecimiento del árbol.
- **random_state**: semilla de aleatoriedad, que permite reproducir siempre los mismos resultados.

Por último, aplicamos SHAP sobre los modelos para generar las gráficas en forma “waterfall”. Para agrupar las contribuciones locales de las gráficas anteriores, usamos una gráfica tipo “beeswarm”. En esta técnica, cada punto representa una muestra, su posición horizontal indica el valor SHAP asociado y su color refleja el valor original de la característica.

3.5.5. Perceptrón multicapa

Los perceptrones multicapa son modelos de redes neuronales empleados para la aproximación de funciones complejas, entre las que se incluyen tareas de clasificación binaria.

Justificación del modelo

El motivo de probar un MLP para esta tarea de clasificación fue, sobre todo, explorar qué posibles opciones tenemos a la hora de abordarla.

La idea inicial fue comprobar qué conseguíamos con los modelos más simples, ya que contamos con un dataset muy reducido.

Planificación del trabajo

El tratamiento del perceptrón se divide en varias etapas:

1. Tratamiento de los datos: En esta etapa se adaptarán los audios a los requisitos del modelo y se dividirán en conjuntos de entrenamiento y prueba.
2. Creación y entrenamiento del modelo: Se creará y configurará el modelo, que posteriormente será entrenado en la tarea de clasificación.
3. Obtención y análisis de resultados: En esta fase se analizarán los resultados obtenidos por el modelo.

Preprocesamiento de los datos

El proceso de tratamiento de datos comienza extrayendo del dataset las características globales de los audios, es decir, los que representan una media. También se obtendrán los espectrogramas de cada audio.

A continuación, emplearemos un MLP y una CNN de PyTorch, que reciben como entrada tensores de datos.

El objetivo de estas dos redes auxiliares es obtener vectores de *embeddings* a partir de los audios.

Embeddings de los espectrogramas

Para sacar los *embeddings* de los espectrogramas emplearemos una CNN de la librería de *Pytorch.nn* y *Pytorch.nn.functional*.

Una CNN es una red neuronal profunda basada en la operación de convolución y se usan para el procesamiento de datos no estructurados como imágenes y audio.

Esta CNN recibirá el vector y lo convertirá en un tensor 2D de 128 dimensiones. Para esta tarea la red se definirá de la siguiente manera:

- **Primera capa convolucional:** `Conv2d(in_channels = 1 , out_channels = 32, kernel_size=3, padding=1)`.
 - **in_channels = 1:** La imagen tiene un único canal
 - **out_channels = 32:** Genera 32 mapas de características, es decir, aprende 32 filtros distintos.
 - **Kernel_size = 3:** Indica que cada filtro es de tamaño 3x3

- **padding = 1:** Añade un borde de un píxel alrededor de la imagen para mantener su tamaño espacial

Durante la ejecución de esta capa cada uno de los 32 filtros se deslizará por la imagen haciendo multiplicaciones elemento a elemento y sumando los resultados produciendo al final 32 imágenes nuevas.

- **Primera capa de normalización:** BatchNorm2d(32). Para cada uno de los 32 canales de entrada que espera, calculará:

- **Media y varianza**
- **Normalizará los valores:** $x_{\text{norm}} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$
- **Aplica una transformación aprendible:** $y = \gamma x_{\text{norm}} + \beta$

- **Primera capa de pooling:** MaxPool2d(kernel_size = 2, stride = 2).

- **kernel_size = 2:** Ventana de tamaño 2x2
- **stride = 2:** La ventana se desplaza de 2 en 2 bloques sin solaparse.

Esta capa reduce cada mapa de características en bloques de 2x2 donde cada bloque se queda con el valor máximo. Esta capa no cambia el número de canales, lo que hace es reducir el alto y el ancho a la mitad.

- **Segunda capa convolucional:** Conv2d(in_channels = 32, out_channels = 64, kernel_size=3, padding=1).
- **Segunda capa de normalización:** BatchNorm2d(64).
- **Segunda capa de pooling:** MaxPool2d(kernel_size = 2, stride = 2).
- **Tercera capa convolucional:** Conv2d(in_channels = 64, out_channels = 128, kernel_size=3, padding=1).
- **Tercera capa de normalización:** BatchNorm2d(128).
- **Capa de pooling adaptativo:** AdaptiveAvgPool2d(1). Reduce cada mapa de características a un tamaño fijo, en este caso 1x1. Esta capa toma cada canal completo (toda la imagen) y calcula su valor promedio colapsando toda la información espacial en un solo número por canal en este caso.
- **Capa totalmente conectada:** Linear(in_features = 128, out_features = 128). Toma un vector de *in_features* y lo transforma en otro de *out_features*. Esto lo hace aplicando la siguiente fórmula: $y = Wx + b$ donde
 - **y:** Vector de salida
 - **W:** Matriz de pesos
 - **x:** Vector de entrada

- **b**: Vector de sesgo

Además definimos la función de avance aplicando tras cada conjunto de capa convolucional + normalización la función de activación relu (Convierte los valores negativos en 0) para dejar pasar solo los importante. Luego aplicamos la capa de pooling. Hacemos esto hasta que lleguemos a la tercera capa convolucional donde en lugar de aplicarle la capa de pooling, le aplicaremos la capa adaptativa y la capa totalmente conectada.

Al concluir todo este proceso tendremos el embedding de 128 dimensiones final que represente al espectrograma del audio.

Embeddings de las características

Para las características definiremos el siguiente perceptrón multicapa:

- **Primera capa totalmente conectada:** `Linear(in_features = 29, out_features = 128)`.
 - **Primera capa relu:** `ReLU()`.
 - **Capa de normalización:** `BatchNorm1d(128)`.
 - **Segunda capa totalmente conectada:** `Linear(in_features = 128, out_features = 64)`.
 - **Segunda capa relu:** `ReLU()`.

Este proceso acabará devolviendo un *embedding* de características de 64 dimensiones.

Embedding combinado

A continuación, se concatenan ambos vectores en un único vector de *embeddings* que combina características y espectrograma. Estos vectores resultantes constituirán los datos de entrada del perceptrón encargado de clasificar los audios.

Por último, se divide el dataset en conjuntos de entrenamiento (80 %) y prueba (20 %).

Creación y entrenamiento del modelo

En esta fase se construirá el modelo encargado de la clasificación. Para ello, se debe tener en cuenta que no se busca una decisión binaria (0 o 1), sino un grado de pertenencia a la clase (real o generada) en un entorno continuo. Para ello, se empleará un MLP con coeficientes de regularización inspirados en lógica difusa.

Teniendo esto en cuenta, el modelo constará de las siguientes partes. Durante el entrenamiento, también se le proporcionarán al modelo las etiquetas correspondientes a la clasificación de los audios.

1. Entrada - Primera capa lineal - `Linear(input_dim, 128)`
2. Capa ReLU - `ReLU()`
3. Capa de normalización - `LayerNorm(128)`
4. Segunda capa lineal - `Linear(128, 64)`
5. Capa ReLU - `ReLU()`
6. Capa de normalización - `LayerNorm(64)`
7. Última capa lineal - `Linear(64, 1)`
8. Salida: Aplicación de la función de pertenencia difusa

El modelo también presenta los siguientes parámetros difusos:

1. Centro
2. Temperatura

Su función *forward* devuelve el grado de pertenencia difuso como predicción probabilística del modelo, el logit y la temperatura.

Por último, se define una función de pérdida que ajusta los coeficientes *reg_center* y *reg_temperature*.

En cuanto al entrenamiento, se emplea la siguiente configuración:

1. Batch size = 10
2. Num_batches = 100
3. Epochs = 10
4. Criterion = `MSELoss`
5. Optimizer = Adam (Algoritmo empleado para entrenar redes neuronales combinando el momentum y tasas de aprendizajes adaptativas).

Enlace al notebook: “[notebook](#)”

3.5.6. Wav2Vec2

El modelo elegido fue Wav2Vec2, un modelo de aprendizaje profundo para el procesamiento de voz que permite obtener representaciones acústicas de alta calidad directamente a partir de la señal de audio cruda, sin necesidad de extraer características manualmente.

Justificación del modelo

El propósito de realizar un *fine-tuning* de este modelo para adaptarlo a nuestra tarea se basa en que, al contar con un dataset de audio muy reducido, es necesario partir de un modelo previamente entrenado con grandes volúmenes de datos. Aunque dicho entrenamiento previo no esté orientado a clasificación, permite reutilizar el conocimiento aprendido y adaptar el modelo a nuestra tarea mediante su reentrenamiento.

Planificación del trabajo

El proceso de *fine-tuning* de este modelo se divide en varias etapas:

1. Preprocesamiento de los datos: Conversión de los datos al formato requerido por el modelo.
2. Configuración y entrenamiento del modelo: Adaptación y reentrenamiento del modelo con los nuevos datos.
3. Análisis de resultados: Evaluación de los resultados obtenidos.

Antes de comenzar con el preprocesamiento de los datos, es necesario considerar las librerías empleadas. Este modelo se utiliza a través de la librería *transformers*, de la cual también se importan las clases *Trainer*, *TrainingArguments*, *Wav2Vec2ForSequenceClassification* y, especialmente relevante en esta etapa, *Wav2Vec2FeatureExtractor*. Asimismo, se emplea la librería *Datasets*, que no debe confundirse con las estructuras de datos de la librería *Pandas*.

Preprocesamiento de los datos

Para el preprocesamiento de los audios se definen dos constantes globales: la frecuencia de muestreo (*sampling rate*), fijada en 16000 Hz (ya que el modelo fue entrenado con esta frecuencia), y la duración máxima de los audios, establecida en 10 segundos.

El *dataframe* utilizado consta de dos columnas: una con las rutas a los audios y otra con las etiquetas correspondientes. Este conjunto de 200 audios, que incluye muestras tanto en español como en inglés, se divide en tres subconjuntos: entrenamiento (80%), validación (10%) y prueba (10%).

Posteriormente, se cargan los audios y se convierten las etiquetas al tipo *long*, que es el formato esperado por el modelo.

A continuación, los datos se procesan mediante una serie de funciones para adaptarlos a los requisitos del modelo:

1. Extractor de características: define cómo se introduce el audio en el modelo, recibiendo señal cruda, normalizándola y aplicando *padding* para convertirla al formato esperado.
2. Función de preprocesamiento: recibe el audio, la longitud máxima, la frecuencia de muestreo y el extractor de características, y devuelve los tensores listos para el modelo.
3. Colector de datos: recoge todos los datos del dataframe (valores de entrada y etiquetas) y devuelve un diccionario de esos datos transformados a tensores de PyTorch.

Configuración y entrenamiento del modelo

En esta fase se emplean las clases restantes de la librería *transformers*, comenzando con *Wav2Vec2ForSequenceClassification*.

Esta clase permite cargar el modelo base *Wav2Vec2*, configurar el número de salidas y definir la tarea. En nuestro caso, el modelo se configura para clasificación binaria (*single-label classification*) con dos salidas.

Para el entrenamiento de este modelo se han usado los 200 audios del dataset (100 en español y 100 en inglés) divididos como se mencionó anteriormente (80-10-10). Dado que son pocos audios para efectuar esta tarea, optamos por congelar el extractor de características para evitar el sobreajuste de los filtros del modelo.

A continuación, se definen las métricas de evaluación. En este caso, debido al posterior postprocesado de las salidas, se emplean la precisión (*accuracy*), el *F1-score* y el área bajo la curva ROC (ROC-AUC).

Posteriormente, se utiliza la clase *TrainingArguments* para configurar el entrenamiento del modelo:

- `output_dir = ./wav2vec2-deepfake`: Carpeta que guarda los checkpoints, logs y el mejor modelo.
- `eval_strategy = steps`: Evaluación cada x pasos.
- `save_strategy = steps`: Guardado cada x pasos.
- `logging_steps = 50`: Cada 50 pasos calcula el loss, la tasa de aprendizaje y las métricas si toca evaluar.
- `save_steps = 500`: Evalúa cada 500 pasos.
- `eval_steps = 500`: Guarda cada 500 pasos.
- `learning_rate = 1e-5`: Tasa de aprendizaje. Es tan baja ya que se trata de una tarea de fine-tuning.
- `per_device_train_batch_size = 4`: Batch-size del entrenamiento.

- `per_device_eval_batch_size = 4`: Batch-size de la evaluación.
- `gradient_accumulation_steps = 2`: Acumula gradientes durante 2 pasos antes de actualizar los pesos.
- `num_train_epochs = 5`: Épocas de entrenamiento.
- `fp16 = True`: Precisión de 16 bits para optimizar memoria y mejorar la velocidad de entrenamiento.
- `metric_for_best_model = auc`: Métrica que determinará cuál es el mejor modelo.
- `load_best_model_at_end = True`: Carga el modelo con mejor métrica AUC.
- `report_to = none`: Desactiva la integración con herramientas externas.

Por último, se utiliza la clase *Trainer* para ensamblar todo el pipeline y definir el proceso de entrenamiento:

- `model = model`
- `args = training_args`
- `train_dataset = train_dataset["train"]`
- `eval_dataset = train_dataset["eval"]`
- `data_collator = data_collator`
- `processing_class = feature_extractor`
- `compute_metrics = compute_metrics`

Una vez configurados todos los componentes, se procede a iniciar el entrenamiento del modelo.

Enlace al notebook: “[notebook](#)”

Implementación de modelos de XAI para la interpretabilidad del modelo Wav2Vec2 fine-tuneado

Ahora que tenemos el resultado del modelo (el Wav2Vec2 fine-tuneado), vamos a explicar que características y que parte del espectrograma de Mel del audio son las más influyentes.

Sin embargo ahora nos encontramos con el problema principal de este modelo que es que como entrada recibe un embedding, entonces, no podemos aplicar técnicas de XAI directamente al modelo. Otro problema que tenemos es que el código que tenemos de extracción de características saca muchísimas características y de ciertas características no solo saca un valor escalar sino que también saca un vector con sus valores en cada instante.

Para solucionar estos problemas hemos realizado dos preprocesamientos. La primera es que agruparemos los atributos del dataset en cinco grupos en función de sus características y lo que representan. Con esto nos quedan las siguientes agrupaciones:

1. Energía: rms, silence ratio, zcr, skewness, kurtosis. Este grupo son las características que se centran en la energía del audio.
2. Espectrales: spectral centroid, spectral bandwidth, spectral rolloff, spectral flatness, hnr. Este grupo son las características que miden la distribución de la energía en el ámbito de la frecuencia.
3. Pitch: pitch mean, pitch std. Este grupo son las características que miden el comportamiento de la frecuencia fundamental del audio.
4. Formantes: f1 mean, f2 mean, f3 mean. Este grupo son las características que describen la estructura vocal.
5. MFCC: MFCC mean del 1 al 13. Este grupo de características capturan patrones del espectro del audio.

El segundo preprocesamiento que se ha hecho ha sido desarrollar modelos surrogate que imiten el comportamiento del modelo. Para esto hemos implementado seis modelos distintos. Los cinco primeros modelos serán regresiones Ridge (añadir referencia) de la clase *sklearn*. Usamos esta regresión en lugar de una regresión lineal o una regresión logística ya que con los pocos datos del conjunto de entrenamiento, es seguro que el modelo va sufrir de sobreajuste y esta regresión ataja este problema.

Ahora pasamos al sexto surrogate model, una CNN muy simple que recibirá como entrada los espectrogramas de mel de los audios en forma de tensores de *pytorch*. Esta se compone solo de una capa convolucional y una capa de pooling. Además se simplificarán los espectrogramas agrupando todas las bandas de frecuencia en 8 superbandas. Esto se hace para evitar que se produzca un sobreajuste ya que tenemos muy pocos audios.

Sobre este surrogate model aplicaremos Shap, Lime, Integrated gradients y Grad cam.

Enlace al notebook: “[notebook](#)”

3.5.7. Regresión Logística

La Regresión Logística es un algoritmo de aprendizaje supervisado utilizado para problemas de clasificación binaria. En este proyecto, el modelo se emplea para determinar la probabilidad de que una muestra de audio pertenezca a una de dos categorías: **Real (1)** o **Generado/Fake (0)**. A pesar de su nombre, es un clasificador lineal que utiliza la función sigmoide para mapear

cualquier valor real en un rango de 0 a 1, facilitando la interpretación de los resultados como probabilidades.

Justificación del modelo

Se ha seleccionado la Regresión Logística como modelo base (*baseline*) por las siguientes razones extraídas del desarrollo experimental:

- **Eficiencia y Simplicidad:** Es computacionalmente ligero, lo que permite un entrenamiento casi instantáneo con el dataset de 100 muestras.
- **Interpretabilidad Directa:** Al ser un modelo lineal, permite analizar los coeficientes asignados a cada característica acústica para entender su relevancia.
- **Idoneidad para Clasificación Binaria:** El problema presenta clases bien balanceadas (50 muestras reales y 50 sintéticas), donde la regresión logística suele converger de manera óptima.

Planificación del trabajo

El flujo de trabajo documentado en el cuaderno sigue estas fases:

1. **Carga y Fusión:** Consolidación de los archivos `spoof_features.csv` y `bonafide_features.csv`.
2. **Preprocesamiento:** Selección de variables numéricas y normalización estadística.
3. **Entrenamiento:** División del dataset (80/20) y ajuste de hiperparámetros (`random_state=42`).
4. **Evaluación y XAI:** Análisis de métricas y generación de explicaciones locales con SHAP, LIME y DiCE.

Tratamiento de los datos

El proceso de ingeniería de datos incluyó:

- **Dataset:** Un total de 100 muestras de audio.
- **Características (Features):** Se utilizaron 34 variables acústicas, incluyendo *duration*, *rms*, *zcr*, *spectral centroid*, *hnr*, y los 13 primeros coeficientes *MFCC*.
- **Normalización:** Se aplicó `StandardScaler` para estandarizar las características (*mean=0*, *std=1*), paso crítico para asegurar que todas las variables contribuyan equitativamente al modelo lineal.

Creación y entrenamiento del modelo

El proceso de construcción del modelo se basó en la implementación de *Scikit-Learn*, siguiendo una configuración diseñada para la estabilidad en datasets pequeños:

- **Instanciación:** Se utilizó la clase `LogisticRegression` configurando un `random_state=42` para asegurar la consistencia de los coeficientes en diferentes ejecuciones.
- **Algoritmo de Optimización:** Se empleó el optimizador *liblinear*, el cual es especialmente robusto para conjuntos de datos de dimensiones moderadas (100 muestras) y problemas de clasificación binaria.
- **Proceso de Ajuste:** El modelo fue entrenado utilizando el método de máxima verosimilitud, ajustando los pesos (w) para cada una de las 34 características acústicas. Este proceso busca minimizar la función de pérdida de entropía cruzada (*Log-Loss*):

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))] \quad (3.1)$$

- **Validación:** El entrenamiento se validó inicialmente sobre el 80 % de los datos, verificando que la convergencia del gradiente fuera óptima antes de pasar a la fase de test.

Resultados de la Regresión Logística

Tras la evaluación con el conjunto de prueba (20 muestras), el modelo alcanzó un rendimiento perfecto:

- **Precisión Global (Accuracy):** 100 %.
- **Reporte de Clasificación:** Tanto para la clase *Fake* como *Real*, se obtuvieron valores de 1.00 en precisión, *recall* y F1-score.
- **Matriz de Confusión:** Se clasificaron correctamente las 10 muestras de cada clase sin falsos positivos ni falsos negativos.

Conclusión: La Regresión Logística demuestra que el conjunto de características acústicas seleccionado es altamente discriminativo para este problema, permitiendo una separación lineal perfecta de las clases en el entorno de prueba. Por otro lado es extraño que den resultados tan perfectos, pero al escuchar algunos de los audios generados descubrimos que son muy dicriminatorios y por tanto los puede identificar fácilmente.

Explicabilidad de la Regresión Logística

Se aplicaron tres técnicas para auditar las decisiones del modelo:

- **SHAP:** SHAP permite identificar qué características usa para clasificar cada instancia en una clase u otra, en este caso vemos que la más discriminativa es el spectral flatness seguido del f3 mean y el mfcc 1 mean.
- **LIME:** Se analizó el conjunto entero y se descubrió que los datos que más discriminan para una clase u otra son el spectral flatness y el spectral bandwidth por lo que confirma lo mostrado por shap en una instancia.
- **DiCE:** Se generaron contraejemplos donde vemos que para que una instancia cambie de clase hay que modificar el valor de spectral flatness siguiendo la línea de las otras explicaciones.

Conclusión XAI: como podemos observar las tres técnicas coinciden en que el dato más discriminante para clasificar es el spectral flatness. Al verlo en las tres técnicas confirmamos que no es algo puntual y que el dataset entero se clasifica en base a ese dato.

3.5.8. KNN

El algoritmo K-Nearest Neighbors (KNN) se ha seleccionado como una alternativa no paramétrica para la clasificación de audios. A diferencia de la Regresión Logística, KNN no asume una relación lineal entre las características, sino que clasifica cada muestra basándose en la proximidad geométrica de sus vecinos más cercanos en el espacio de características.

Justificación del modelo

La elección de KNN se fundamenta en:

- **Naturaleza No Lineal:** Es capaz de capturar fronteras de decisión complejas que un modelo lineal podría pasar por alto.
- **Simplicidad Conceptual:** No requiere una fase de entrenamiento pesada, ya que "memoriza" el conjunto de datos, lo que lo hace útil para conjuntos de datos de tamaño moderado.
- **Sensibilidad Local:** Es muy eficaz cuando las muestras de una misma clase tienden a presentar firmas acústicas muy similares entre sí.

Planificación del trabajo

El desarrollo del clasificador KNN siguió estas etapas:

1. **Carga y Limpieza:** Integración de los vectores de características de audios *bonafide* y *spoof*.

2. **Optimización de Hiperparámetros:** Uso de validación cruzada para determinar el número óptimo de vecinos (k).
3. **Entrenamiento y Evaluación:** Ajuste del modelo con la métrica de distancia euclídea y evaluación de su capacidad de generalización.

División de datos El conjunto de datos se particionó en entrenamiento (80 %) y prueba (20 %) utilizando muestreo estratificado para mantener la proporción de clases en ambos conjuntos. Esta estratificación es crucial cuando existe desbalance entre voces reales y generadas.

Modelo baseline Se estableció una configuración inicial con hiperparámetros por defecto para obtener una referencia de rendimiento:

- **n_estimators:** 100.
- **max_depth:** 10.
- **min_samples_split:** 5.
- **min_samples_leaf:** 2.
- **max_features:** sqrt.
- **random_state:** 12.
- **class_weight:** balanced.

Tratamiento de los datos

Debido a que KNN se basa en distancias, el tratamiento de datos fue crítico:

- **Escalado:** Se aplicó `StandardScaler` a las 34 características acústicas. Sin este paso, las variables con rangos mayores (como *spectral_rolloff*) dominarían la métrica de distancia sobre otras como el *RMS*.
- **División Estratificada:** Se mantuvo la proporción de clases en el 80 % de entrenamiento y 20 % de test para evitar sesgos por cercanía.

Creación y entrenamiento del modelo (KNN)

A diferencia de los modelos paramétricos, la creación del modelo KNN se centró en la optimización del espacio de búsqueda y la métrica de proximidad:

- **Búsqueda del Hiperparámetro k :** Se implementó un ciclo de iteración para evaluar el rendimiento del modelo con valores de k entre 1 y 20. Se utilizó la técnica de **Validación**

Cruzada (*Cross-Validation*) con 5 iteraciones ($cv=5$) para garantizar que el valor de k elegido no fuera fruto del azar.

- **Selección de K Óptimo:** Basándose en la puntuación media de precisión, se seleccionó $k = 5$ (o el valor que refleje tu gráfica de error), ya que ofrecía el mejor equilibrio entre sesgo y varianza.
- **Métrica de Distancia:** Se configuró el modelo para utilizar la **distancia Euclídea**, calculada en el espacio n -dimensional de características normalizadas:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (3.2)$$

- **Almacenamiento:** El modelo final (`KNeighborsClassifier`) se entrenó con el set completo de entrenamiento, indexando las 80 muestras para permitir una búsqueda rápida de vecinos durante la fase de evaluación.

Resultados del KNN

Tras realizar una búsqueda del hiperparámetro óptimo, se determinó que el modelo alcanza su mejor rendimiento con un valor de $k = 5$. Los resultados obtenidos en el conjunto de prueba de 20 muestras son:

- **Precisión Global (Accuracy):** 0.95.
- **Métricas Detalladas:** El modelo alcanzó una puntuación F1 de 0.95 para ambas clases (*Real* y *Fake*).
- **Matriz de Confusión:** Se observó una clasificación perfecta, con 10 verdaderos positivos y 9 verdaderos negativos, dejando solo un falso positivo.

Conclusión: Al igual que la Regresión Logística, KNN obtiene unos números casi perfectos en este dataset, lo que sugiere que al igual que en el apartado anterior discrimina bien los audios a excepción de uno, lo que suponemos que significa que ese audio generado está más logrado.

Explicabilidad del KNN

Para entender el comportamiento de un modelo basado en distancias como KNN, se aplicaron las siguientes técnicas de IA explicable:

- **Análisis de Contribución (SHAP):** Aunque KNN no tiene coeficientes, SHAP permite identificar qué características usa para clasificar cada instancia en una clase u otra, en

Modelo	Accuracy	Precision	Recall	F1-Score	AUC-ROC	MCC
RF	1	1	1	1	1	1
LGBM	1	1	1	1	1	1
SVM	1	1	1	1	1	1
KNN	1	1	1	1	1	1
RL	1	1	1	1	1	1

Tabla 3.1: Métricas principales de los modelos baseline de datos tabulares

este caso vemos que la más discriminativa es el spectral flatness seguido del f3 mean y el mfcc 3 mean.

- **Explicación Local (LIME):** Se analizó el conjunto entero y se descubrió que los datos que más discriminan para una clase u otra son el spectral flatness y el spectral bandwidth por lo que confirma lo mostrado por shap en una instancia.
- **Contrafactuales (DiCE):** Se generaron contraejemplos donde vemos que para que una instancia cambie de clase hay que modificar el valor de spectral flatness siguiendo la línea de las otras explicaciones.

Conclusión XAI: como podemos observar las tres técnicas coinciden en que el dato más discriminante para clasificar es el spectral flatness. Al verlo en las tres técnicas confirmamos que no es algo puntual y que el dataset entero se clasifica en base a ese dato.

3.6. Resultados

3.6.1. Modelos de datos tabulares

En este apartado vamos a agrupar y revisar los resultados de los modelos de datos tabulares, los cuales se mencionan en 3.5, sin incluir los resultados de IA explicable. Para comodidad en las tablas, los modelos los hemos acortado por sus siglas y, de ser exactamente iguales los resultados, la tabla de resultados mostrará todos los resultados compactados.

Modelos baseline

Como vemos en las tablas 3.1 3.2 3.3, estos modelos clasifican correctamente entre reales y generados.

Métrica-modelo	Real (1)	IA (0)	accuracy	macro avg	weighted avg
Precisión	1	1		1	1
Recall	1	1		1	1
F1	1	1	1	1	1
Nº Instancias	10	10	20	20	20

Tabla 3.2: Classification report de los modelos baseline de datos tabulares

Modelo	Verdaderos Negativos	Verdaderos Positivos	Falsos Positivos	Falsos Negativos
modelos	10	10	0	0

Tabla 3.3: Análisis de errores de los modelos baseline de datos tabulares

Validación Cruzada Estratificada

Los resultados revelan que los modelos son estables en los 5 folds y que clasifican bien, como se ve en [3.4](#).

Grid Search

En primer lugar veamos cuáles son los mejores parámetros del grid que hemos definido en los distintos modelos.

Para Random Forest podemos ver que los parámetros difieren ligeramente a lo que hemos definido en el modelo baseline (ver [Tabla 3.5](#)). Teniendo que los valores de `max_depth`, `min_samples_split` y `min_samples_leaf` han disminuido.

Como se mencionó en [Subsección 3.5.2](#), el Grid Search de LightGBM usaba parámetros distintos. Los resultados podemos verlos en la [tabla 3.6](#).

Métrica	Media	Desviación típica
Accuracy	1	+/- 0
F1-Score	1	+/- 0
AUC-ROC	1	+/- 0

Tabla 3.4: Resultados de Validación Cruzada con 5 folds de los modelos baseline de datos tabulares

Hiperparámetro	Valor
n_estimators	100
max_depth	5
min_samples_split	2
min_samples_leaf	1
max_features	sqrt

Tabla 3.5: Mejores hiperparámetros para Random Forest con Grid Search y CV de 5 folds

Hiperparámetro	Valor
num_leaves	15
learning_rate	0.01
n_estimators	300
min_child_samples	20
subsample	0.6
colsample_bytree	0.6
reg_alpha	0
reg_lambda	0

Tabla 3.6: Mejores hiperparámetros para LightGBM con Grid Search y CV de 5 folds

Hiperparámetro	Valor
C	1
gamma	0.01
kernel	rbf

Tabla 3.7: Mejores hiperparámetros para SVM con Grid Search y CV de 5 folds

El Grid Search del modelo SVM puede parecer más simple, pero es interesante ver los valores que devuelve (ver Tabla 3.7. En primer lugar el valor de C es el mismo que el que definimos, 1. Por otro lado, gamma ahora vale 0,01, que, como se explicó en [Subsección 2.1.5](#), un valor bajo hace que los límites de decisión sean más suaves, previniendo el sobreajuste.

3.6.2. Random Forest

En este apartado vamos a mostrar los resultados sobre los métodos de explicación aplicados a Random Forest.

Como podemos observar en las gráficas 3.2 y 3.3, las cuatro características más influyentes en una predicción son las mismas y en el mismo orden. Además encontramos que tienen valores similares en un ambos sentidos de la explicación, pero con signo opuesto.

Como se muestra en la gráfica 3.4, la importancia de las características en la explicación global de SHAP concuerda con las que vimos antes, en mismo orden y magnitud. Además, en colores podemos ver mejor, por ejemplo en spectral flatness, que si el valor para una instancia es bajo (azul), su peso es positivo, mientras que si es alto (rojo) su peso es negativo.

Ya que hemos visto que “spectral flatness” es la característica más influyente con diferencia, hemos decidido eliminarla y entrenar un modelo, cuyos resultados podemos ver en las tablas 3.8, 3.9 y 3.10. Aquí vemos que el modelo deja de clasificar correctamente, debido a que, al perder la característica más influyente, el modelo pierde rendimiento en clasificación.

Tras quitar la característica “spectral flatness” del modelo, la explicación que genera la podemos ver en las gráficas 3.5 y 3.6. Aquí vemos que otras características adquieren peso, ya que la falta de una característica hace que las importancias se reestructuren. En particular vemos que “spectral bandwidth” mantiene una alta importancia, mientras que “mfcc 1” la mantiene solo en la predicción de clase negativa.

Con la nueva distribución de importancias, hemos decidido quitar, además de “spectral flatness”, “spectral bandwidth” y “mfcc 1”. Los resultados del modelo los encontramos en las

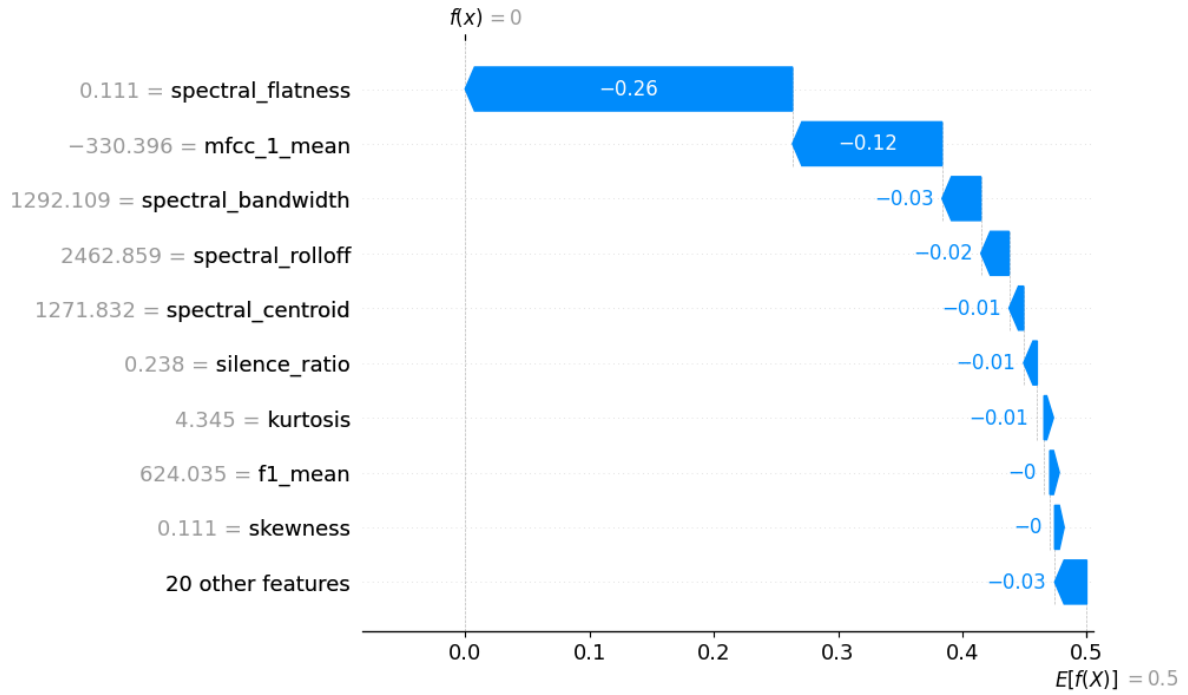


Figura 3.2: Explicación local de Random Forest con SHAP

Métrica	Valor
Accuracy	0.9000
Precision	0.9000
Recall	0.9000
F1-Score	0.9000
AUC-ROC	0.9900
MCC	0.8000

Tabla 3.8: Métricas principales de Random Forest sin spectral flatness

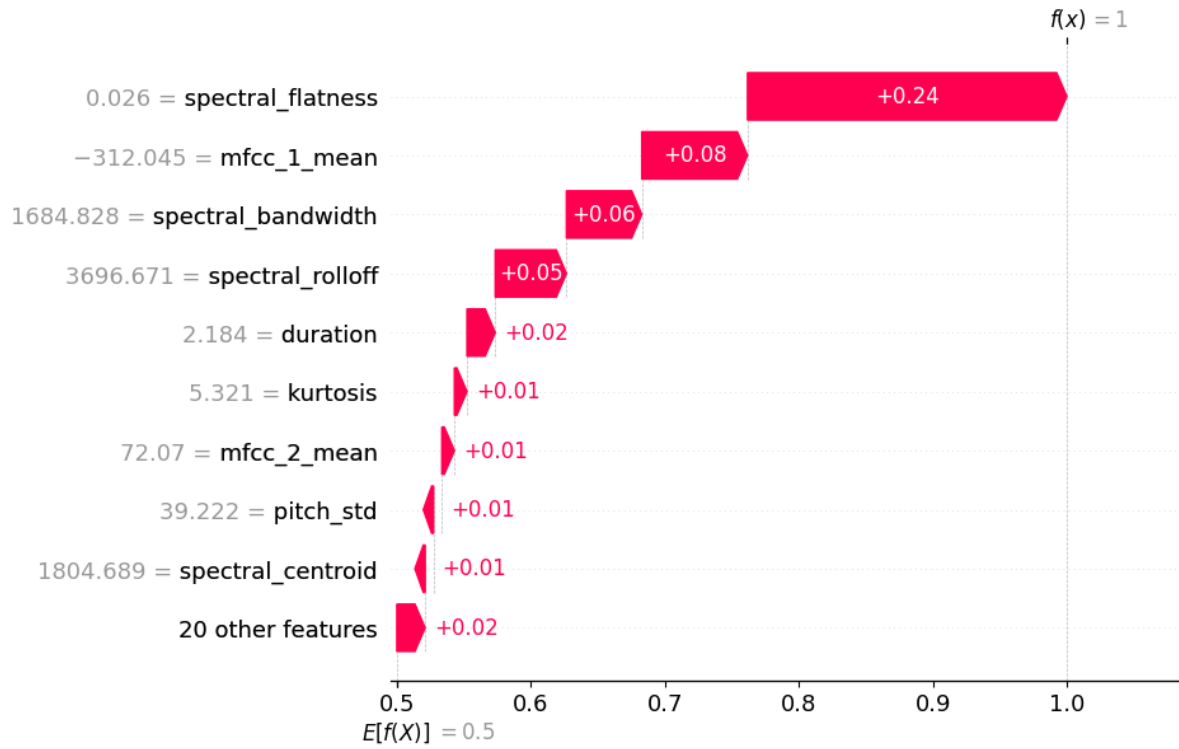


Figura 3.3: Explicación local de Random Forest con SHAP

Dato	Precisión	Recall	F1-Score	Nº Instancias
Real (1)	0.9000	0.9000	0.9000	10
IA (0)	0.9000	0.9000	0.9000	10
accuracy			0.9000	20
macro avg	0.9000	0.9000	0.9000	20
weighted avg	0.9000	0.9000	0.9000	20

Tabla 3.9: Classification report de Random Forest sin spectral flatness

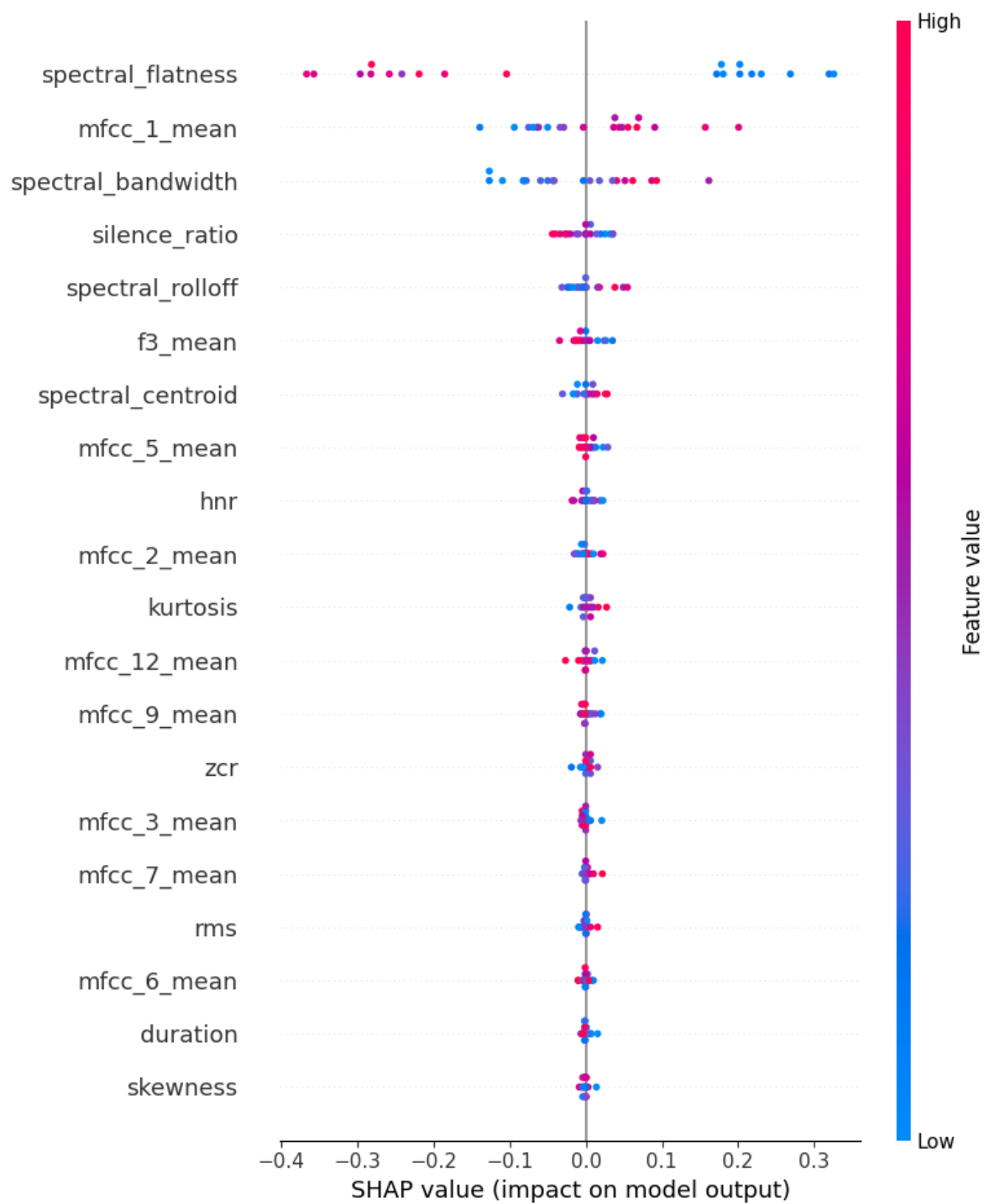


Figura 3.4: Explicación global de Random Forest con SHAP

Tipo de predicción	Cantidad
Verdaderos Negativos	9
Verdaderos Positivos	9
Falsos Positivos	1
Falsos Negativos	1
Error tipo I	10 %
Error tipo II	10 %

Tabla 3.10: Análisis de errores de Random Forest sin spectral flatness

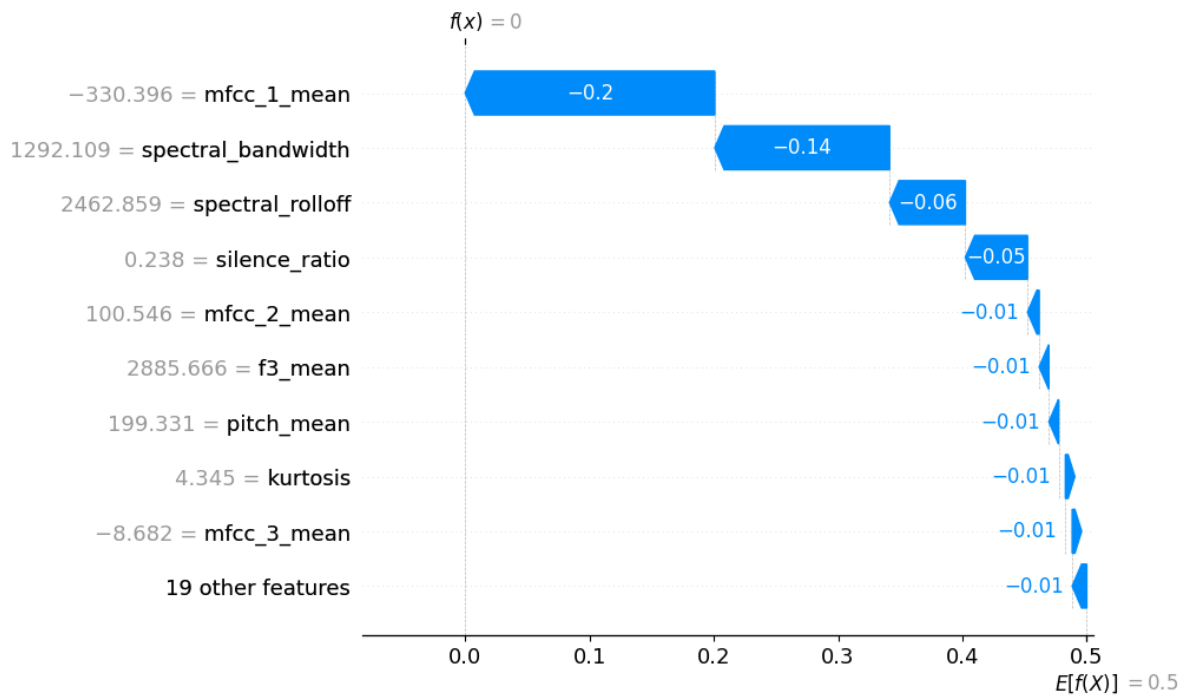


Figura 3.5: Explicación local de Random Forest con SHAP tras quitar spectral flatness

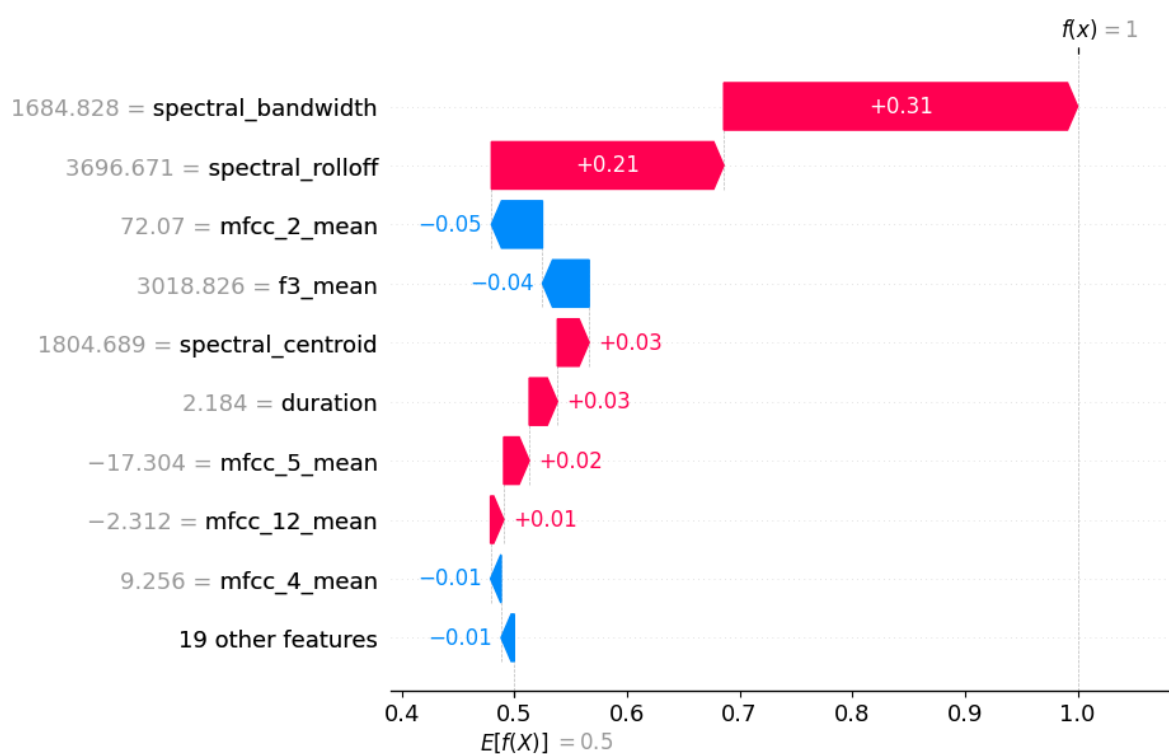


Figura 3.6: Explicación local de Random Forest con SHAP tras quitar spectral flatness

Métrica	Valor
Accuracy	0.7500
Precision	0.7778
Recall	0.7000
F1-Score	0.7368
AUC-ROC	0.8600
MCC	0.5025

Tabla 3.11: Métricas principales de Random Forest sin spectral flatness, mfcc1 y spectral bandwidth

Dato	Precisión	Recall	F1-Score	Nº Instancias
Real (1)	0.7273	0.8000	0.7619	10
IA (0)	0.7778	0.7000	0.7368	10
accuracy			0.7500	20
macro avg	0.7525	0.7500	0.7494	20
weighted avg	0.7525	0.7500	0.7494	20

Tabla 3.12: Classification report de Random Forest sin spectral flatness, mfcc1 y spectral bandwidth

tablas 3.11, 3.12 y 3.13. Estos son algo inferiores a los previos, y es consecuente con que al eliminar características, el modelo pierde rendimiento.

Como se puede observar en las gráficas 3.7 y 3.8, las importancias vuelven a redistribuirse, ganando más en este caso “silence ratio”. Es interesante ver que ahora la segunda predicción, que hasta ahora predecía como positiva, ahora la predice como negativa (falso negativo), debido a que con las características que tiene no es capaz de clasificarla correctamente.

Como vemos en la imagen 3.9, la explicación que genera LIME tiene “spectral flatness” como característica insignia, aunque también aparecen, con menor relevancia, “mfcc 1” y “spectral bandwidth”.

Los cambios (counterfactuals) que nos devuelve Dice para Random Forest nos muestran, como ya hemos visto, que “spectral flatness” y “mfcc 1” son importantes, pues en las tres instancias deberían cambiar de valor, como observamos en la tabla 3.14.

Tipo de predicción	Cantidad
Verdaderos Negativos	8
Verdaderos Positivos	7
Falsos Positivos	2
Falsos Negativos	3
Error tipo I	20 %
Error tipo II	30 %

Tabla 3.13: Análisis de errores de Random Forest sin spectral flatness, mfcc1, y spectral bandwidth

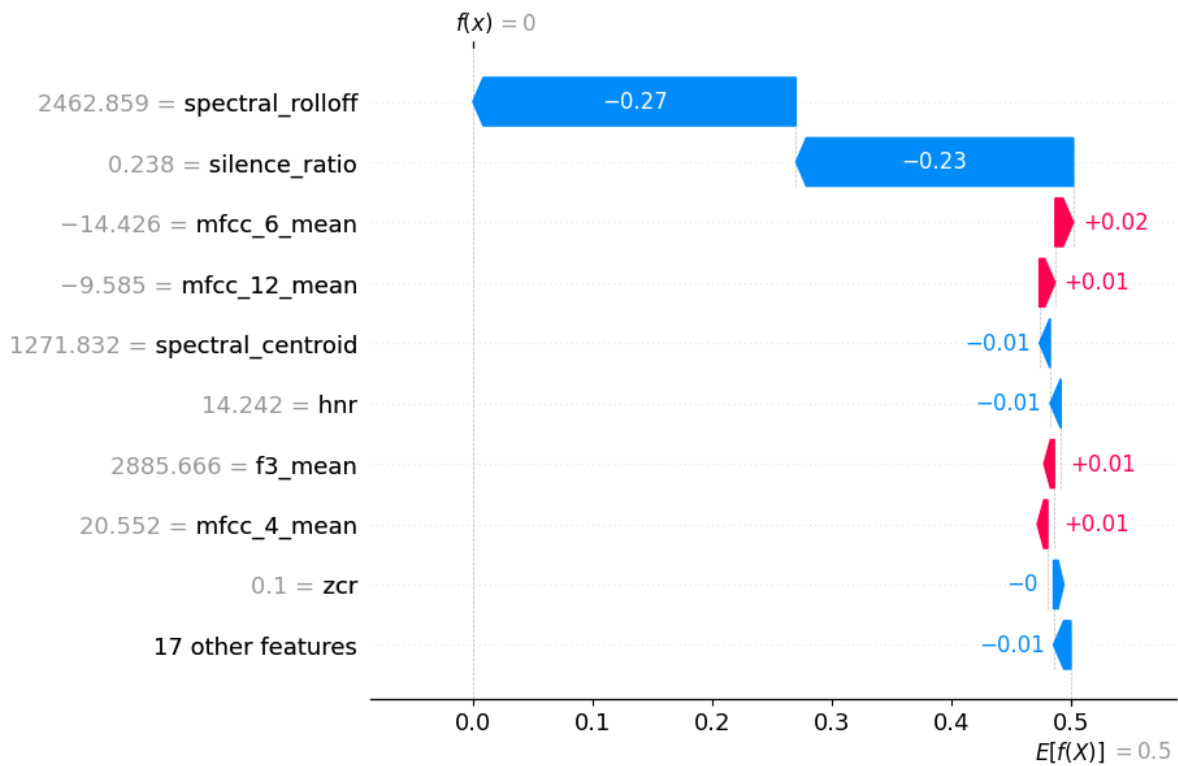


Figura 3.7: Explicación local de Random Forest con SHAP tras quitar spectral flatness, mfcc 1 y spectral bandwidth

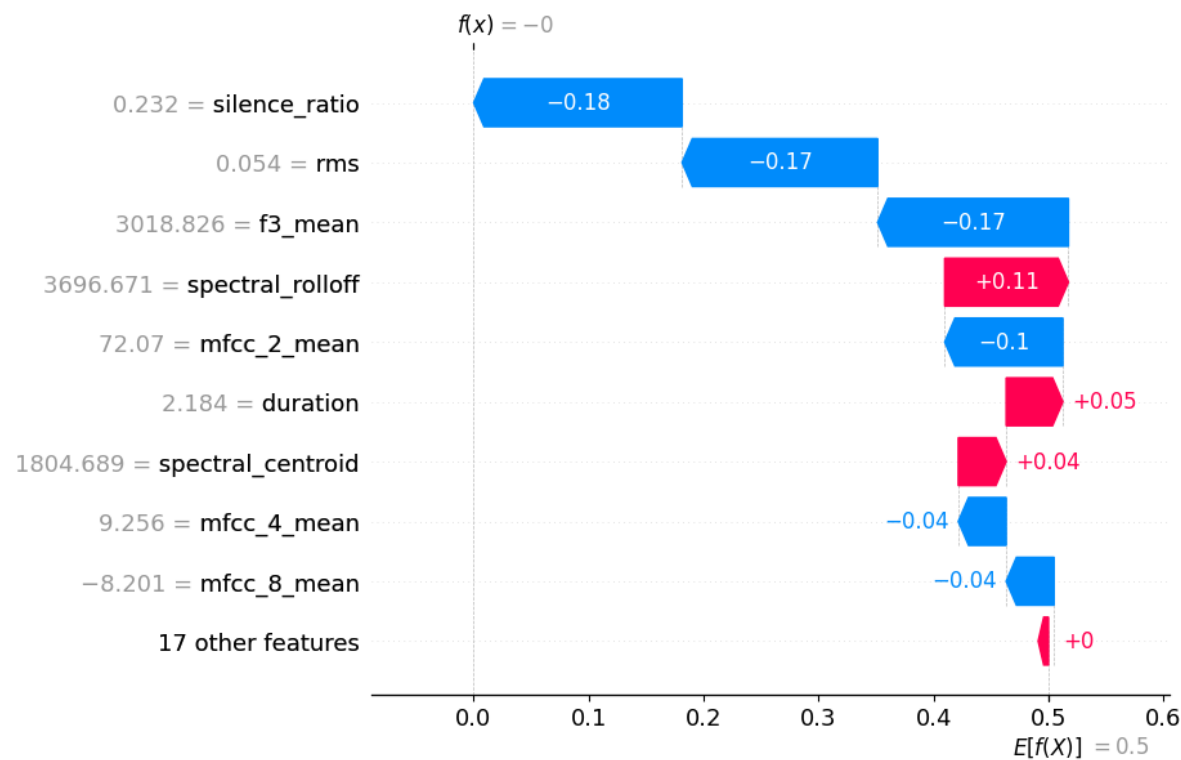


Figura 3.8: Explicación local de Random Forest con SHAP tras quitar spectral flatness, mfcc 1 y spectral bandwidth

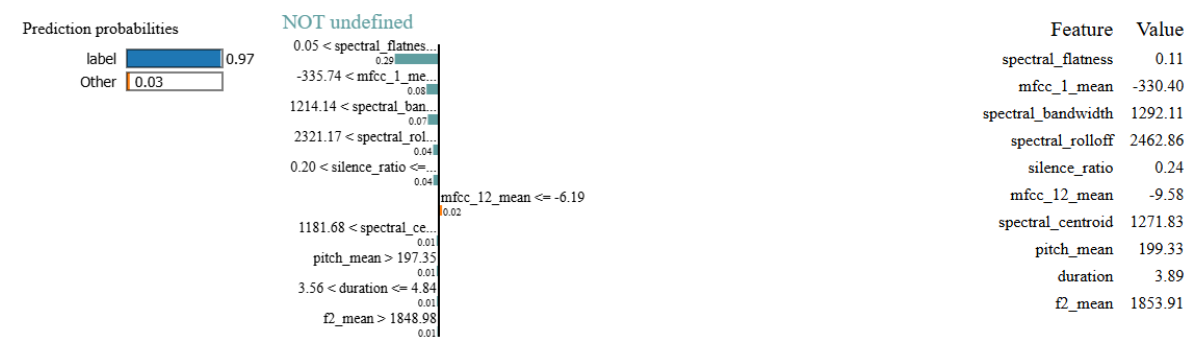


Figura 3.9: Explicación local de Random Forest con LIME

Instancia	spectral flatness	mfcc 1	mfcc 3	mfcc 8	Etiqueta
Explicada	0.11064	-330.39648	-8.68196	-9.58195	0
Counter factual 1	0.02891	-270.65508			1
Counter factual 2	0.02891	-270.65508		-7.92017	1
Counter factual 3	0.02891	-270.65508	14.76117	-7.92017	1

Tabla 3.14: Counter factuales generados por DiCE para Random Forest

Métrica	Valor
Accuracy	0.9000
Precision	0.9000
Recall	0.9000
F1-Score	0.9000
AUC-ROC	0.9900
MCC	0.8000

Tabla 3.15: Métricas principales de LightGBM sin spectral flatness

3.6.3. LightGBM

En este apartado vamos a mostrar los resultados sobre los métodos de explicación aplicados a LightGBM.

Como se puede ver en las gráficas 3.10 y 3.11, el modelo clasifica únicamente con “spectral flatness”. Esto nos dice que, pese a que antes no observamos un sobreajuste, el modelo realmente lo tiene con esta característica.

La importancia de características en la explicación global es congruente con las explicaciones locales: la característica “spectral flatness” tiene mucho peso, mientras que las otras tienen peso cercano a 0.

Hemos quitado dicha característica para ver cómo se comporta el modelo, lo que podemos ver en las tablas 3.15, 3.16 y 3.17. Como era de esperar al ver el sobreajuste del modelo el rendimiento ha bajado, aunque mantiene valores de predicción muy altos.

Para ver ahora la importancia de cada característica veamos las gráficas 3.13 y 3.14. Podemos ver que en la instancia que clasifica como negativa los pesos se redistribuyen razonablemente bien, mientras que en la instancia que clasifica como positiva encontramos que “spectral flatness” recibe casi todo el peso.

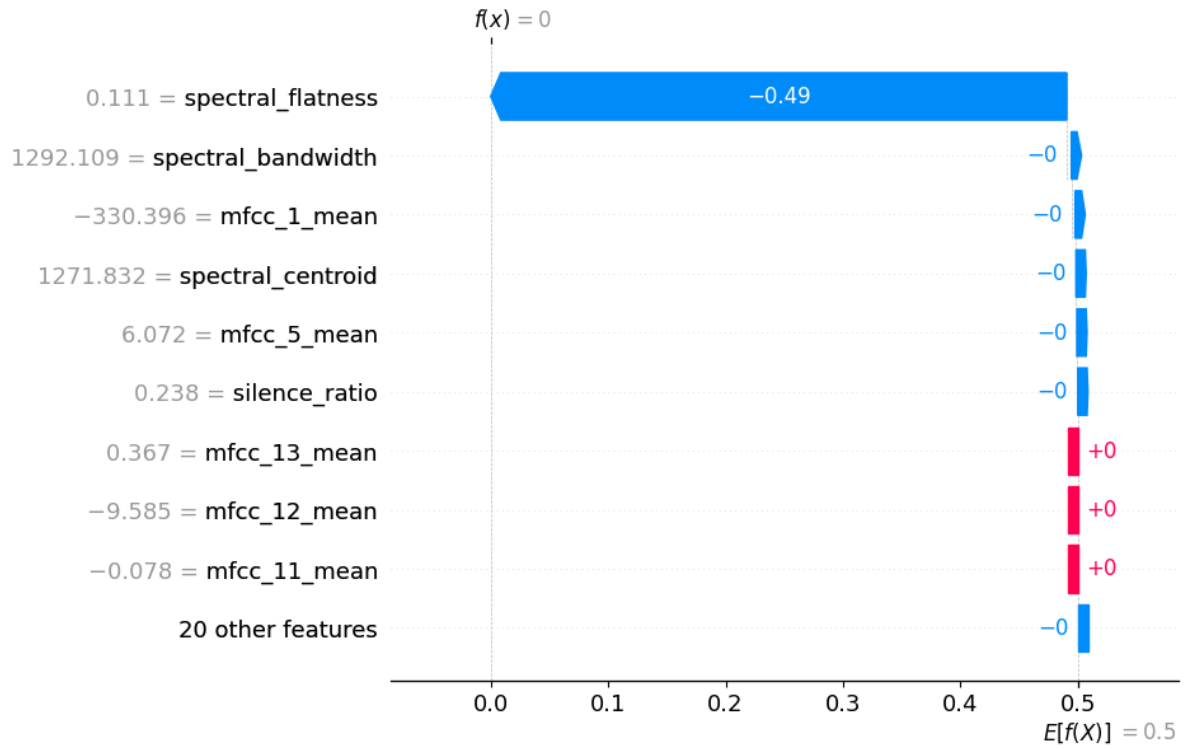


Figura 3.10: Explicación local de LightGBM con SHAP

Dato	Precisión	Recall	F1-Score	Nº Instancias
Real (1)	0.9000	0.9000	0.9000	10
IA (0)	0.9000	0.9000	0.9000	10
accuracy			0.9000	20
macro avg	0.9000	0.9000	0.9000	20
weighted avg	0.9000	0.9000	0.9000	20

Tabla 3.16: Classification report de LightGBM sin spectral flatness

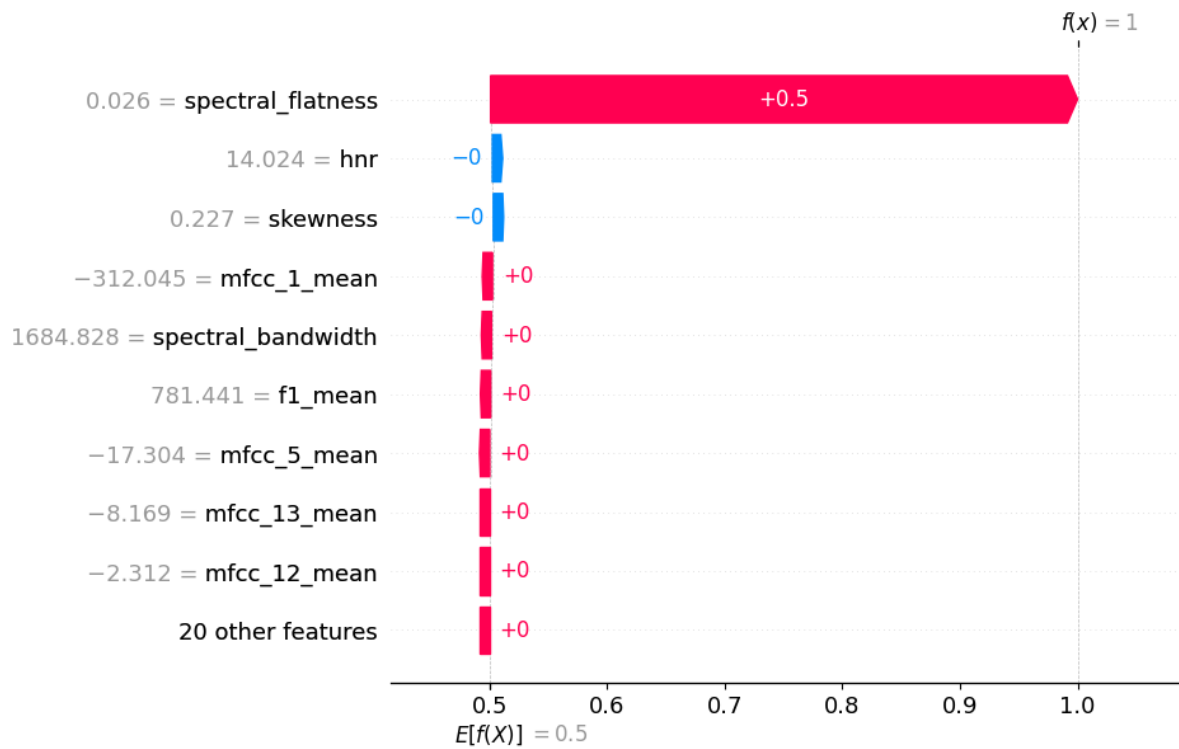


Figura 3.11: Explicación local de LightGBM con SHAP

Tipo de predicción	Cantidad
Verdaderos Negativos	9
Verdaderos Positivos	9
Falsos Positivos	1
Falsos Negativos	1
Error tipo I	10 %
Error tipo II	10 %

Tabla 3.17: Análisis de errores de LightGBM sin spectral flatness

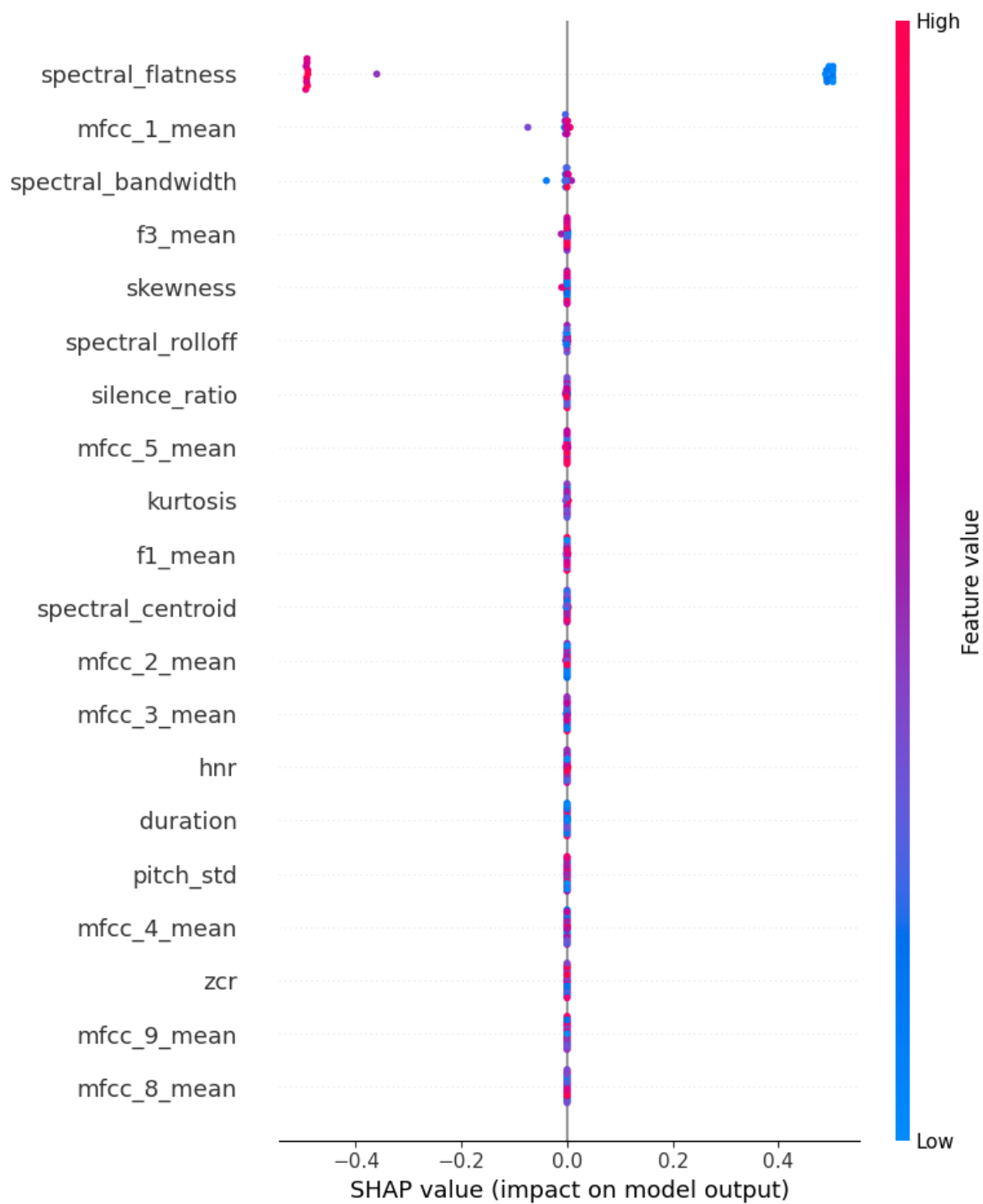


Figura 3.12: Explicación global de LightGBM con SHAP

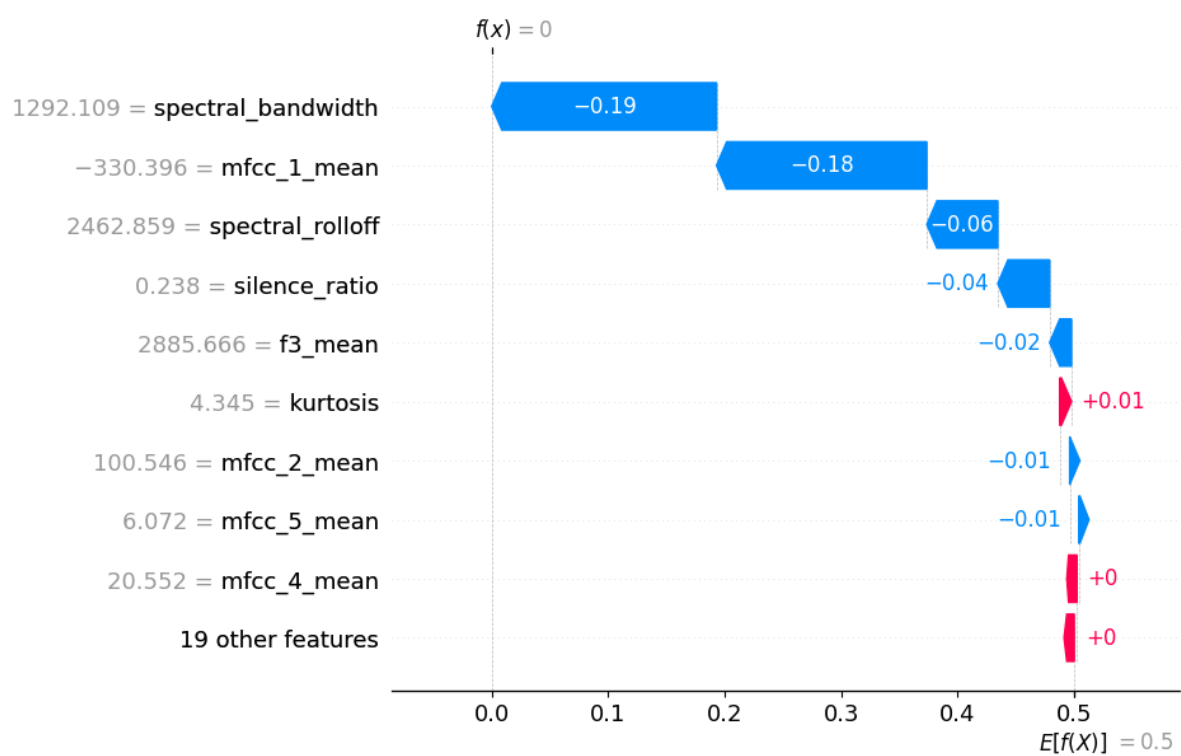


Figura 3.13: Explicación local de LightGBM con SHAP tras quitar spectral flatness

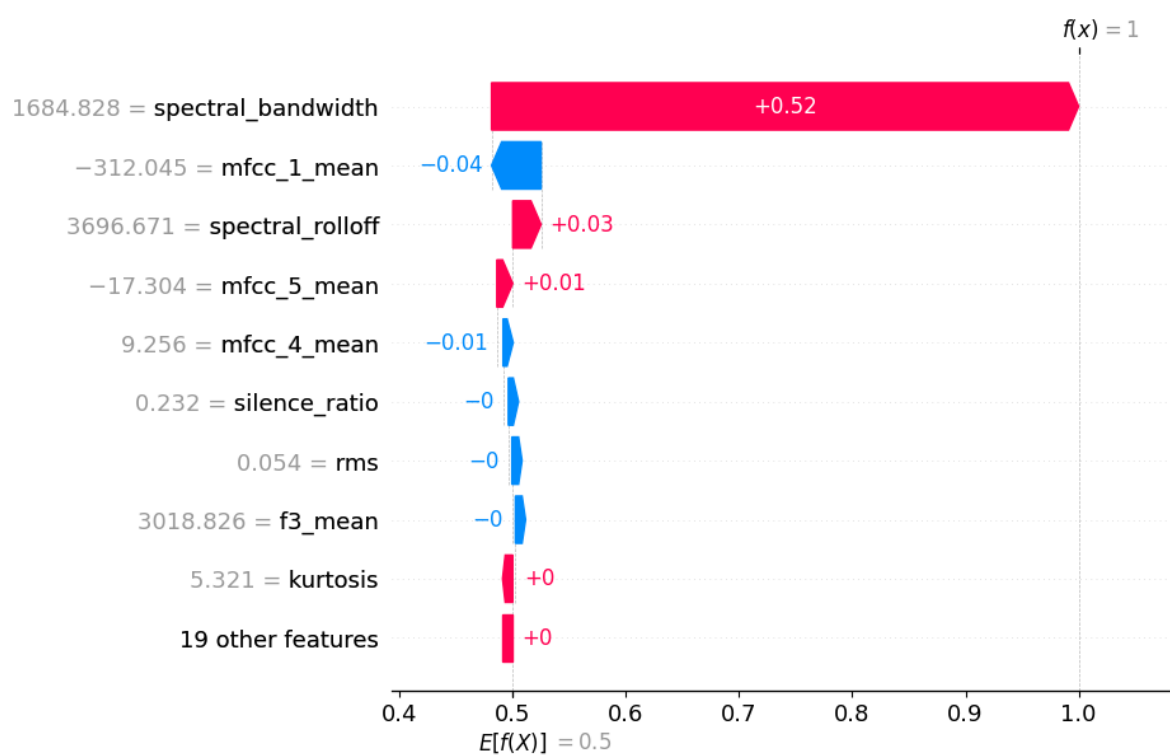


Figura 3.14: Explicación local de LightGBM con SHAP tras quitar spectral flatness

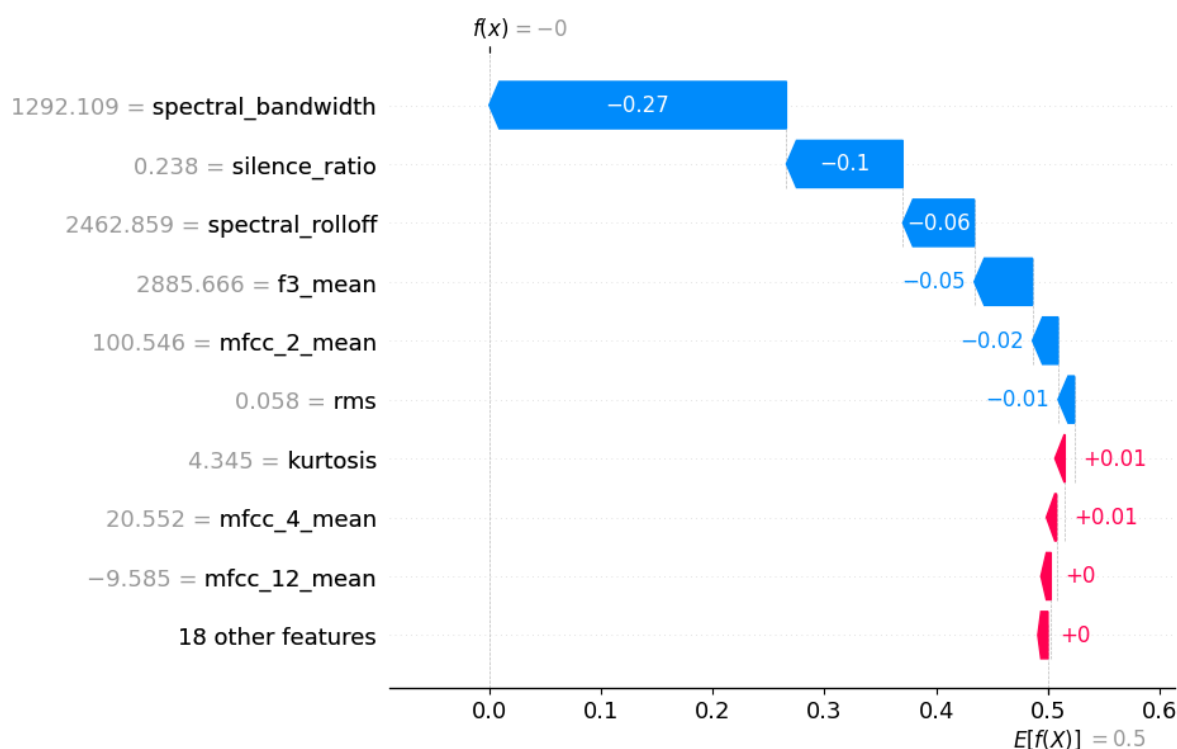


Figura 3.15: Explicación local de LightGBM con SHAP tras quitar spectral flatness y mfcc 1

Tras quitar “spectral flatness” y “mfcc 1” encontramos que la característica que más explica es “spectral bandwidth”, característica que, como ya hemos visto, tiene bastante importancia, como vemos en las gráficas 3.15 y 3.16. Se ve que la importancia de dicha característica es muy alta, el resto de características no tienen mucho peso en la clasificación.

Los resultados de este modelo siguen siendo buenos, como vemos en las tablas 3.18, 3.19 y 3.19. Podemos ver que el rendimiento no ha bajado demasiado pese a quitar dichas características y que mantiene una baja tasa de falsos positivos y falsos negativos.

Como vemos en la imagen 3.17, la explicación que genera LIME también tiene “spectral flatness” como principal característica a la hora de clasificar. También encontramos otras que hemos visto antes como “mfcc 1” o “spectral bandwidth” con pesos relevantes.

La explicación que genera DiCE para LGBM nos muestra tres instancias con alteraciones de, principalmente, “spectral flatness”, aunque también se ven modificaciones de “mfcc 1” y “mfcc 5”, como vemos en la tabla 3.21.

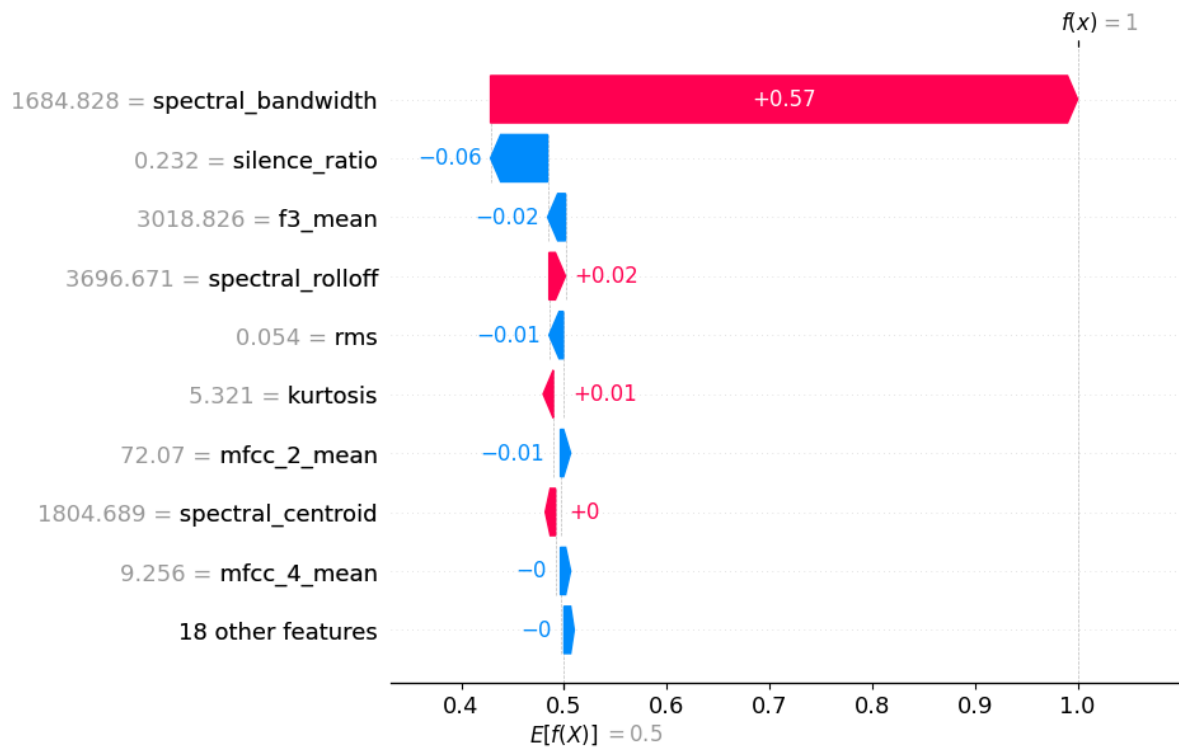


Figura 3.16: Explicación local de LightGBM con SHAP tras quitar spectral flatness y mfcc 1

Métrica	Valor
Accuracy	0.8500
Precision	0.8889
Recall	0.8000
F1-Score	0.8421
AUC-ROC	0.9200
MCC	0.7035

Tabla 3.18: Métricas principales de LightGBM sin spectral flatness y mfcc1

Dato	Precisión	Recall	F1-Score	Nº Instancias
Real (1)	0.8182	0.9000	0.8571	10
IA (0)	0.8889	0.8000	0.8421	10
accuracy			0.8500	20
macro avg	0.8535	0.8500	0.8496	20
weighted avg	0.8535	0.8500	0.8496	20

Tabla 3.19: Classification report de LightGBM sin spectral flatness y mfcc 1

Tipo de predicción	Cantidad
Verdaderos Negativos	10
Verdaderos Positivos	9
Falsos Positivos	0
Falsos Negativos	1
Error tipo I	0 %
Error tipo II	10 %

Tabla 3.20: Análisis de errores de LightGBM sin spectral flatness y mfcc1

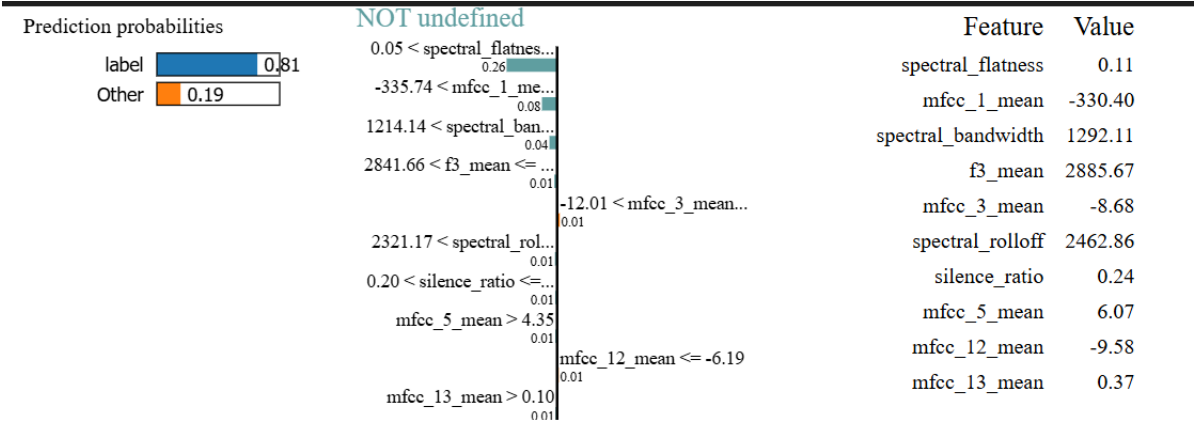


Figura 3.17: Explicación local de LightGBM con LIME

Instancia	spectral flatness	mfcc 1	mfcc 5	Etiqueta
Explicada	0.11064	-330.39648	6.07202	0
Counter factual 1	0.01700			1
Counter factual 2	0.02708	-382.50071		1
Counter factual 3	0.02945		13.43934	1

Tabla 3.21: Counter factuales generados por DiCE para LightGBM

Métrica	Valor
Accuracy	1
Precision	1
Recall	1
F1-Score	1
AUC-ROC	1
MCC	1

Tabla 3.22: Métricas principales de SVM sin spectral flatness

3.6.4. Support Vector Machine

En este apartado vamos a mostrar los resultados y comentaremos un poco al respecto, aunque ahondaremos más en la parte de IA explicable (XAI).

Como podemos ver en las gráficas 3.18 y 3.19, la importancia de las características está bastante bien distribuida. No hay ninguna que supere a las demás por una gran diferencia.

Con estas dos instancias no tenemos una clara idea de cuál es la característica más influyente. Como vemos en la gráfica 3.20, la más influyente es “spectral flatness”, como en los casos anteriores. Aunque en este caso no tiene un peso excesivamente alto, pues hay otras como “spectral bandwidth” o “mfcc 1” que también tienen un peso notable.

Para ver si “spectral flatness” tiene una alta dominancia en la clasificación, entrenamos un modelo sin ella, esto se puede ver en las tablas 3.22, 3.23 y 3.24. Como vemos, el modelo sigue clasificando correctamente aún sin dicha característica.

Para ver ahora la nueva distribución de importancia de características mostramos las gráficas 3.21 y 3.22. Como se puede observar las importancias siguen estando bien distribuidas, aunque sí vemos que “spectral bandwidth” está en la zona de alta importancia en ambas instancias predichas.

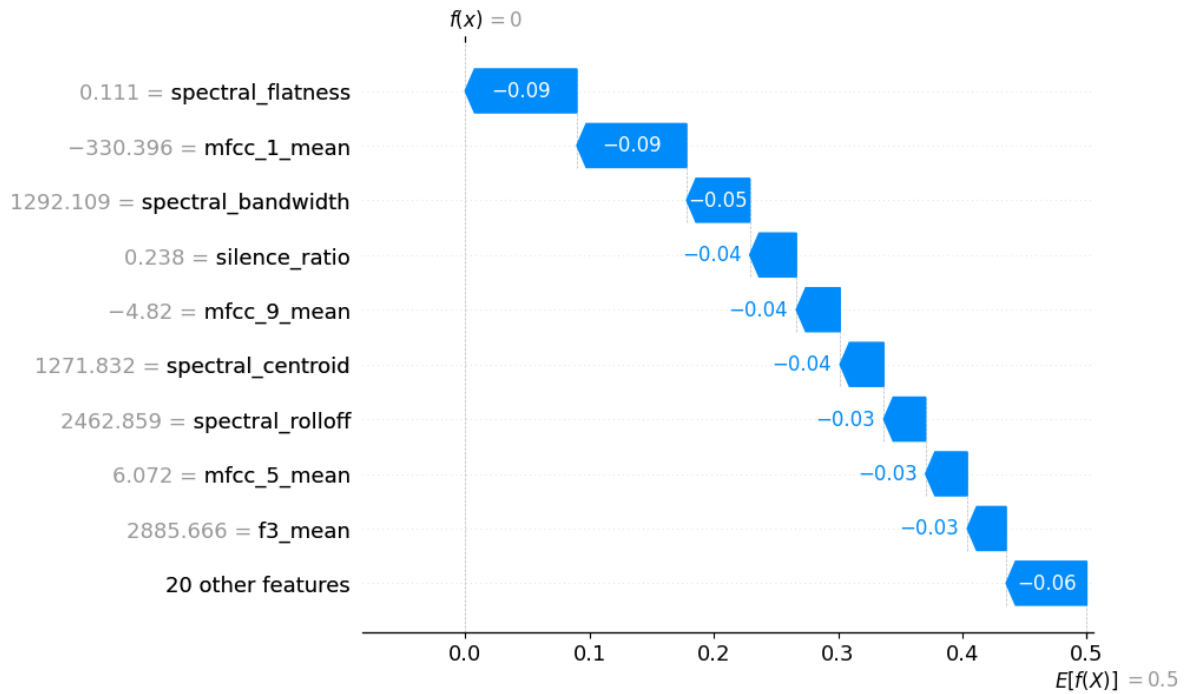


Figura 3.18: Explicación local de SVM con SHAP

Dato	Precisión	Recall	F1-Score	Nº Instancias
Real (1)	1.0000	1.0000	1.0000	10
IA (0)	1.0000	1.0000	1.0000	10
accuracy			1.0000	20
macro avg	1.0000	1.0000	1.0000	20
weighted avg	1.0000	1.0000	1.0000	20

Tabla 3.23: Classification report de SVM sin spectral flatness

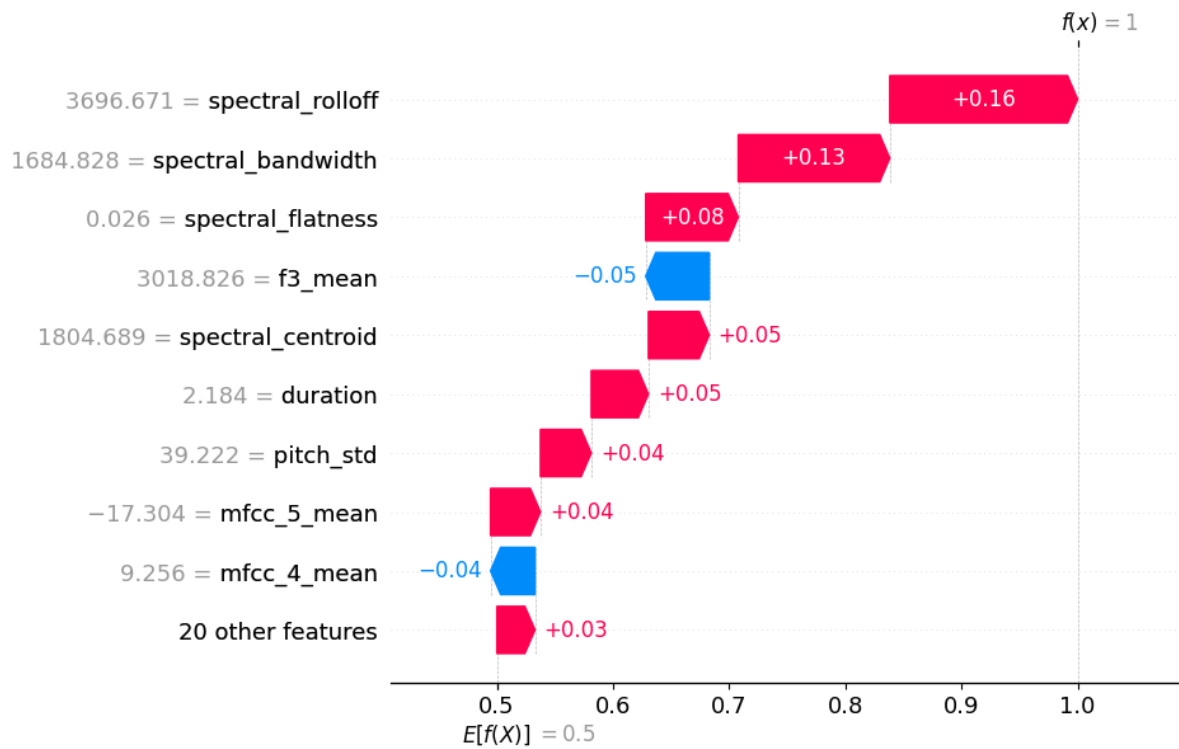


Figura 3.19: Explicación local de SVM con SHAP

Tipo de predicción	Cantidad
Verdaderos Negativos	10
Verdaderos Positivos	10
Falsos Positivos	0
Falsos Negativos	0
Error tipo I	0 %
Error tipo II	0 %

Tabla 3.24: Análisis de errores de SVM sin spectral flatness

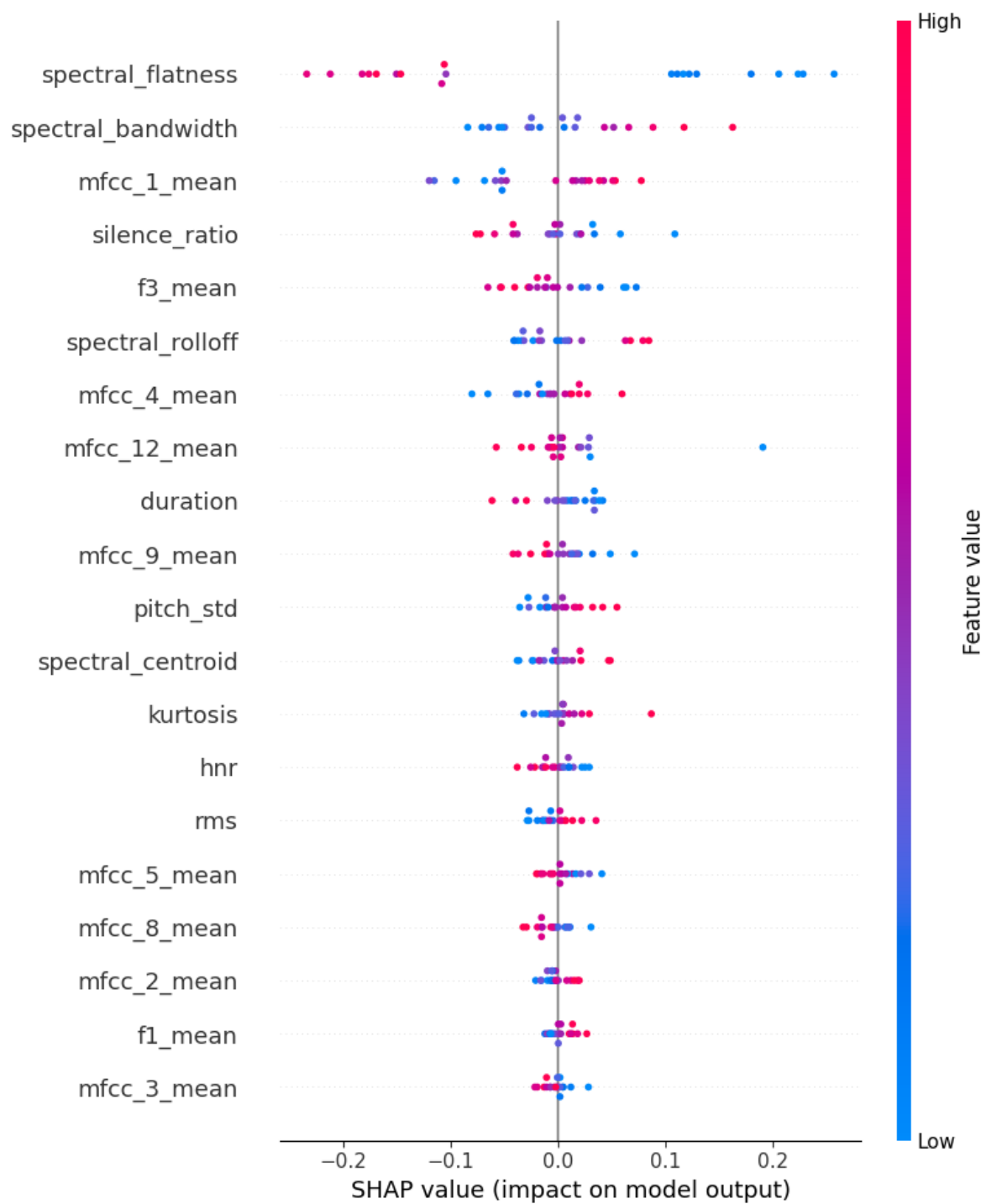


Figura 3.20: Explicación global de SVM con SHAP

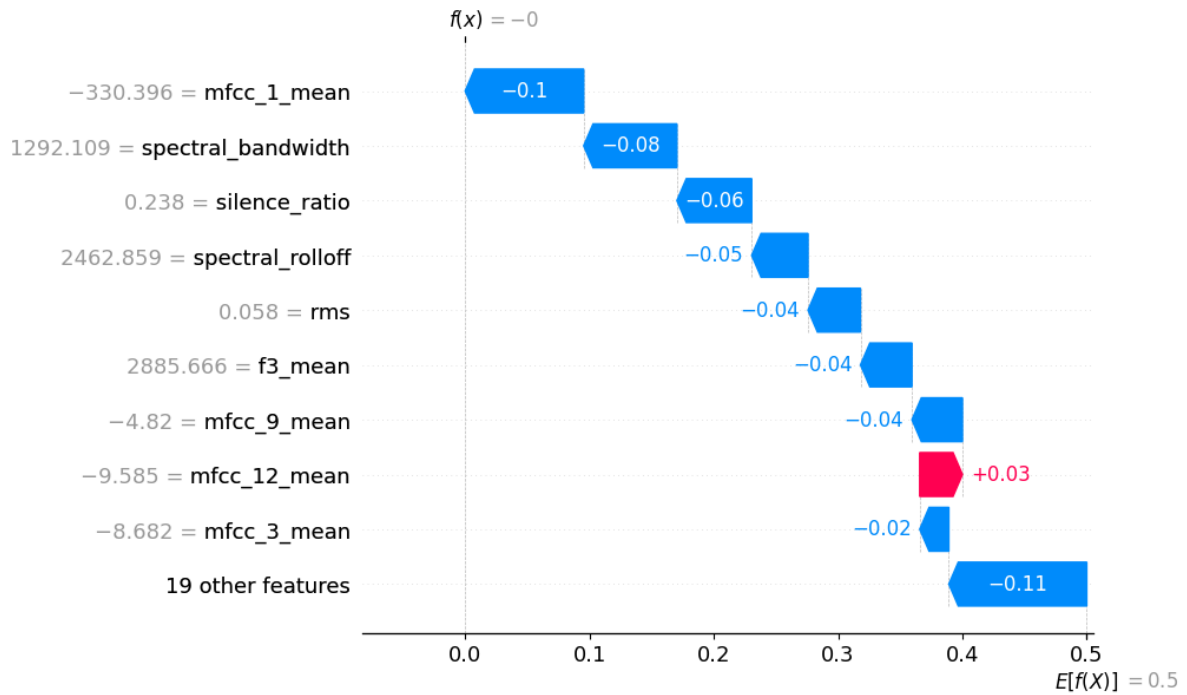


Figura 3.21: Explicación local de SVM con SHAP tras quitar spectral flatness

Ahora probamos a quitar, además de “spectral flatness”, “spectral bandwidth” y “mfcc 1”, resultados en las tablas 3.25, 3.26 y 3.27. Vemos que el modelo sigue clasificando correctamente.

Queremos ahora ver si las importancias se mantienen distribuidas, lo vemos en las gráficas 3.23 y 3.24. Tenemos que la distribución de importancias se mantiene, aunque en la instancia predicha como positiva tenemos que “spectral rolloff” toma un alto valor, pero no deja a 0 el resto de importancias.

Como vemos en la imagen 3.25, la explicación que genera LIME también tiene “spectral flatness” como principal característica a la hora de clasificar.

3.6.5. AASIST

Como hemos mencionado anteriormente, implementamos tres funciones de inferencia distintas para intentar obtener mejores resultados. Tras varias pruebas, alternando las tres funciones, obtuvimos resultados pésimos en los que el modelo tendía a clasificar cada muestra como “bona fide” y alguna como “spoof” pero sin mucho acierto lo que da una sensación de clasificación al azar. Ante estos resultados, decidimos realizar “**fine-tuning**” sobre el propio modelo preentrenado, nuestro propio preentrenamiento con parte del dataset que creamos. Para conseguir esto, primero

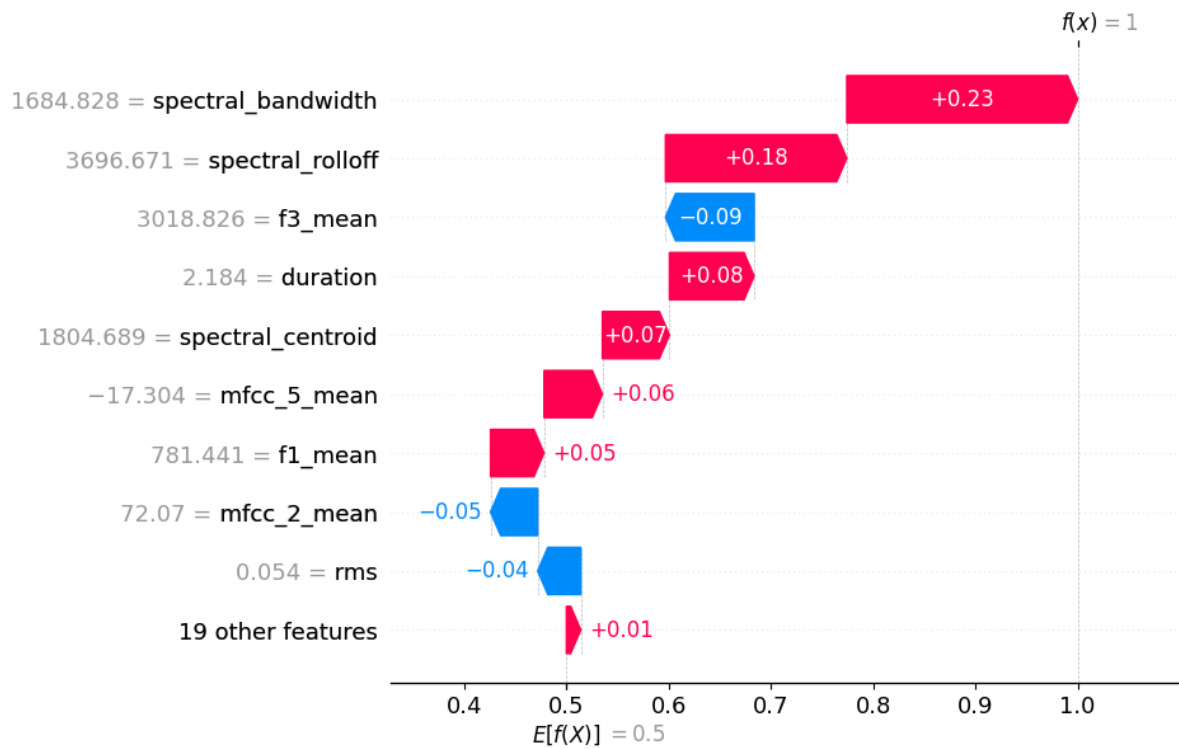


Figura 3.22: Explicación local de SVM con SHAP tras quitar spectral flatness

Métrica	Valor
Accuracy	1
Precision	1
Recall	1
F1-Score	1
AUC-ROC	1
MCC	1

Tabla 3.25: Métricas principales de SVM sin spectral flatness, mfcc1 y spectral bandwidth

Dato	Precisión	Recall	F1-Score	Nº Instancias
Real (1)	1.0000	1.0000	1.0000	10
IA (0)	1.0000	1.0000	1.0000	10
accuracy			1.0000	20
macro avg	1.0000	1.0000	1.0000	20
weighted avg	1.0000	1.0000	1.0000	20

Tabla 3.26: Classification report de SVM sin spectral flatness, mfcc1 y spectral bandwidth

Tipo de predicción	Cantidad
Verdaderos Negativos	10
Verdaderos Positivos	10
Falsos Positivos	0
Falsos Negativos	0
Error tipo I	0 %
Error tipo II	0 %

Tabla 3.27: Análisis de errores de SVM sin spectral flatness, mfcc1 y spectral bandwidth

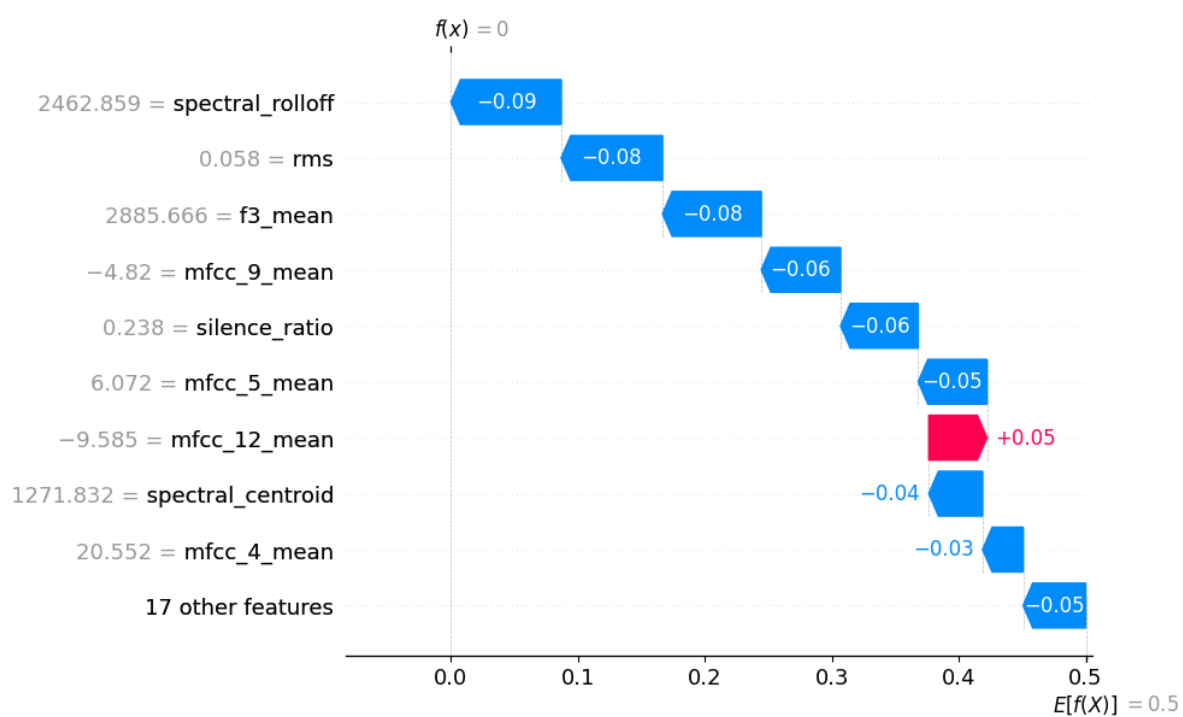


Figura 3.23: Explicación local de SVM con SHAP tras quitar spectral flatness, mfcc 1 y spectral bandwidth

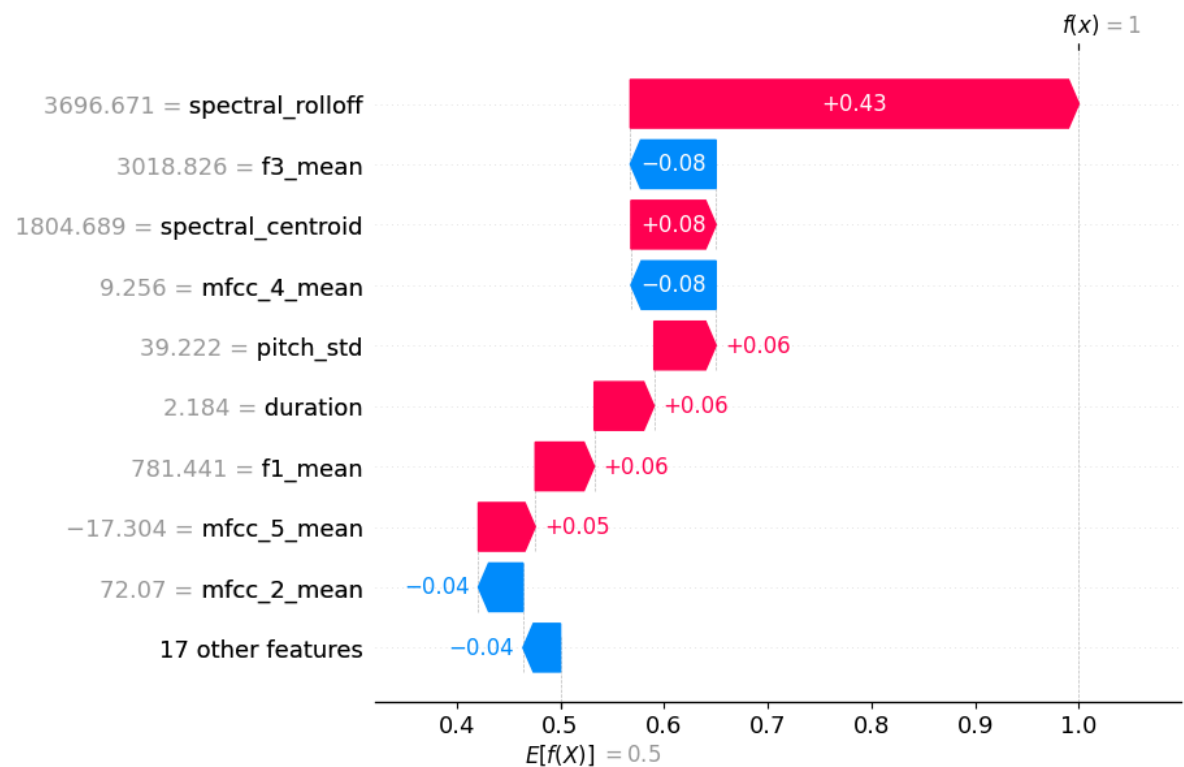


Figura 3.24: Explicación local de SVM con SHAP tras quitar spectral flatness, mfcc 1 y spectral bandwidth

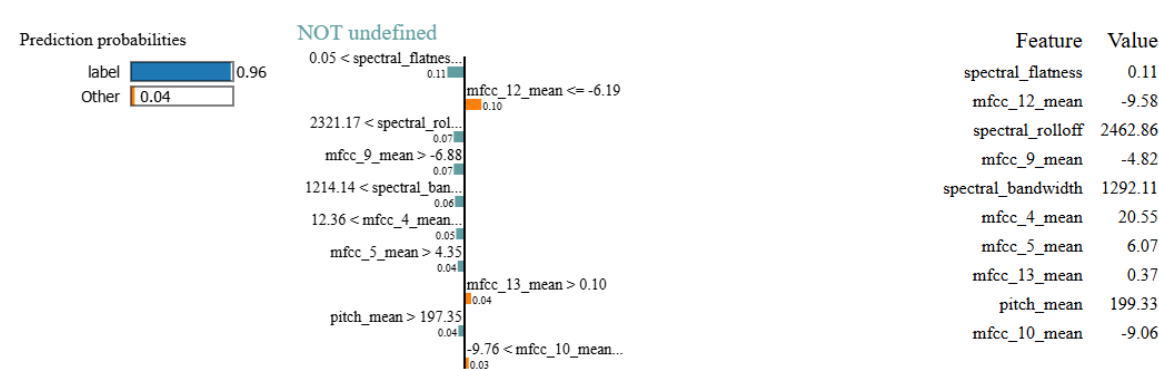


Figura 3.25: Explicación local de SVM con LIME

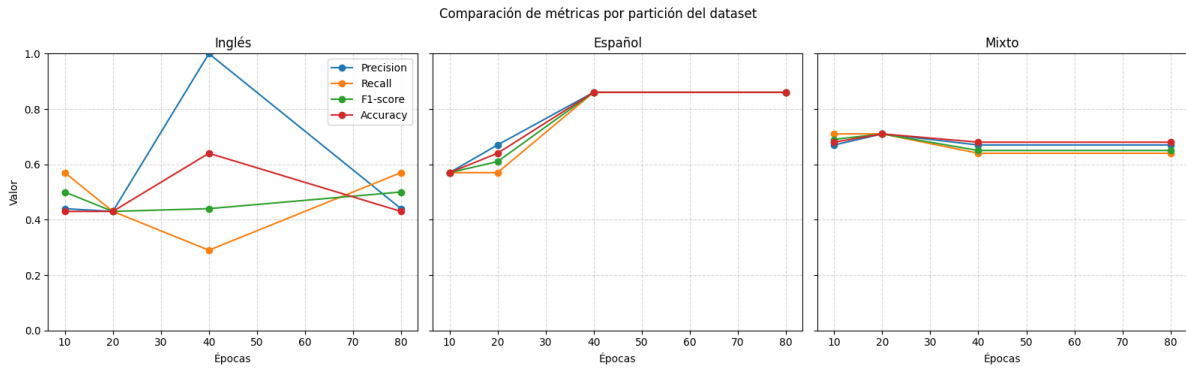


Figura 3.26: Resultados del preentrenamiento en AASIST.

separamos el dataset en entrenamiento, evaluación y validación. Para cada fase, generamos los archivos de “protocols” de donde el modelo va leyendo el nombre y el tipo de los audios de cada partición. Adicionalmente, tuvimos que hacer ciertos ajustes en el .conf y en especial, ajustar el número de épocas.

Para las pruebas, las hemos separado en tres tipos: audios en inglés por separado, español por separado y ambos idiomas juntos. Dentro de cada escenario, hay cuatro pruebas con distinto número de **épocas: 10, 20, 40 o 80**. También, cabe mencionar que siempre se ha mantenido una proporción en la separación en train, dev y eval de 72 %, 14 % y 14 %, respectivamente. Por último, antes de analizar los resultados, es importante explicar que en la separación manual del dataset hemos separado los distintos speakers que había en cada idioma. Esto se refiere a que en el dataset hay por idioma, 5 speakers distintos con 10 audios “bonafide” y 10 “spoof”, entonces, hemos intentado que en las tres particiones haya audios de las 5 personas.

A la hora de evaluar, hemos usado 4 medidas: accuracy, precision, recall y f1.

Lo primero que podemos destacar sobre las gráficas, es que el modelo clasifica claramente mejor los audios en español ya que todas las métricas, a excepción de precisión en 40 épocas, tienen valores superiores. Esta diferencia explica el por qué la partición mixta lo hace peor que la que solo tiene muestras en inglés y permite deducir que ejecutar ambos idiomas juntos no le ayuda a clasificarlos mejor.

En lo referido a las épocas, lo más destacable es que superar las 40 épocas empeora la predicción por lo que podemos suponer que está ocurriendo un fenómeno de sobreaprendizaje. También, podemos observar que en la partición en inglés, el aumento del valor de este parámetro no equivale en términos generales a una subida en la mejora de la predicción, fenómeno que no ocurre con la partición en español.

Resultados de explicación

Las gráficas y mapas que se analizan en esta subsección son los resultados de la ejecución de este “[notebook](#)”.

Resultados

Antes de interpretar los resultados, es importante detallar la composición del conjunto de audios utilizado. Para ello, se seleccionaron 20 muestras, distribuidas de forma equilibrada según idioma y clase: 10 audios en español y 10 en inglés, de los cuales 10 son naturales y 10 generados.

En esta primera [Figura 1](#), vemos cómo ayuda cada tramo a los logits de los audios de cada clase. En lo referido a la clase “bona”, podemos destacar tres fases:

- Entre **0,3 y 0,9** segundos, donde predominan valores negativos, es decir, no aporta evidencia útil al logit de la clase e incluso podría introducir información ambigua.
- Entre **1,0 y 1,8** segundos, donde se encuentran valores muy por encima del resto de la gráfica. Podemos deducir que es la zona donde el modelo encuentra más información importante para la clasificación de esta clase.
- Entre **3,7 y 3,95** segundos, vuelve a aparecer una región claramente positiva. Sin embargo, como rellenamos los audios más cortos hasta 4 segundos, esta última franja puede estar parcialmente contaminada.

Sobre la clase “spoof”, tenemos una gráfica bastante distinta con 4 regiones:

- Entre **0,15 y 0,3** segundos, un pico positivo muy superior al resto de la zona.
- Entre **0,3 y 1,0** segundos, al igual que con la clase “bona”, una zona con valores claramente negativos que indican la misma conclusión.
- Entre **1,2-1,4 y 1,5-1,8** segundos, volvemos a ver picos positivos que empujan el logit hacia la clase “spoof”.
- Entre **3,8 y 4,0** segundos, una región negativa claramente diferenciada. Sin embargo, la precaución explicada para la clase “bona” también se aplica aquí.

En resumen, la [Figura 2](#) sugiere que el modelo se apoya en tramos concretos y no en toda la señal. En la clase “bona”, la evidencia está muy localizada en la zona central, mientras que la clase “spoof” usa evidencia menos estable y seguramente más basada en patrones o irregularidades locales.

Para interpretar esta gráfica, conviene recordar que al mostrar la variación del margen entre clases, las zonas positivas son segmentos **pro-spoof** y las negativas, **pro-bona**. Esta aclaración

se aplica para ambas curvas, ya que para ambos casos calculamos el margen como la diferencia entre la salida de la clase “spoof” y la de la clase “bona”.

En cuanto a las regiones más relevantes, podemos señalar dos: la primera, comprendida entre **0,2 y 0,3**, es pro-spoof, especialmente la curva “spoof”. Esto significa que el modelo encuentra evidencia que le empuja la decisión hacia “spoof”. La segunda región se encuentra entre **1,0 y 2,0** segundos, la cual empuja la decisión claramente a “bona”. Sobre el patrón general de la gráfica, podemos intuir que los audios “spoof” parecen naturales la mayor parte del tiempo, ya que la mayoría de valores son negativos. Sin embargo, AASIST debe encontrar patrones muy localizados que le ayudan a clasificar los audios como spoof.

En la [Figura 3](#), podemos observar claramente cómo la banda 0-500 Hz es muy dominante, e incluso el tramo 500-1000 Hz en “bonas”. Esto es coherente con lo que uno espera de la voz humana, ya que en las bajas frecuencias se concentra no solo la frecuencia fundamental, habitualmente situada en torno a 80-300 Hz, sino también una parte importante de la energía global de la señal y de su estructura armónica.

Asimismo, es importante mencionar que, aunque ambas curvas tienen comportamientos bastante planos en general, esto puede no significar que sean poco relevantes. Es posible que contengan información repetida de forma redundante en otras bandas.

En la [Figura 4](#) podemos destacar tres regiones:

- Tramo **0-500** Hz: en esta banda hay valores positivos, lo que significa que favorece el margen hacia spoof. Aunque pueda parecer contradictorio con la gráfica anterior, una banda puede ser importante para una clase, pero, sin embargo, al considerar el margen, favorecer más a la clase contraria.
- Tramo **500-2000** Hz: aquí encontramos curvas negativas, es decir, es una región pro-bona. Esto puede tener sentido, ya que estas frecuencias se consideran muy importantes para la inteligibilidad del habla.
- Tramo **5500-6500** Hz: aparece una contribución ligeramente positiva. Esto podría estar relacionado con algún patrón artificial de altas frecuencias.

Antes de empezar a analizar la [Figura 5](#), cabe recordar que mostramos la magnitud absoluta de atribución, no signo, por lo que solo podemos destacar relevancia espectral, pero no si una región empuja a cierta clase.

Interpretando los comportamientos, por un lado, observamos actividad predominantemente focalizada en los filtros bajos en el mapa de “bonas”. Por otro lado, en el mapa de “spoofs”, aunque también observamos esta predominancia, se observa una actividad algo mayor en las fre-

cuencias altas. Esto podría reforzar la idea mencionada de patrones artificiales o irregularidades espectrales en frecuencias superiores.

Los mapas normalizados explican mejor cómo se reparte la importancia entre filtros y a lo largo del tiempo al reducir el efecto de las bandas con mayor magnitud absoluta.

En ambas gráficas de la [Figura 6](#), seguimos observando actividad casi continua de los filtros bajos en los 4 segundos. Respecto al mapa de “bonas”, destaca la región entre **1 y 2** segundos, donde hay más actividad. Esto coincide con lo mencionado en la gráfica temporal con “logits” de Occlusion.

En el mapa de “spoofs”, vemos una actividad más fragmentada y localizada en eventos temporales concretos, como, por ejemplo, entre los segundos **1 y 1,5**, y entre los segundos **3,6 y 3,9**. Ambas regiones también coinciden con las conclusiones extraídas en Occlusion temporal.

La [Figura 7](#) aporta información más detallada sobre las bandas de frecuencias bajas y nos señala con mayor precisión dentro de este conjunto, cuáles son las más excitadas. En particular, se observan varios picos destacados entre los filtros 5 y 20, lo que sugiere que la contribución del modelo no se distribuye de forma uniforme entre todos los filtros bajos.

En la [Figura 8](#), volvemos a ver el patrón en el que entre los segundos **1 y 1,8** hay una región importante para ambas clases. Posteriormente, entre los segundos **2 y 3,5**, la atribución es menor, y finalmente, sobre el segundo **3,9**, un pico de relevancia de la curva “bona” claramente diferenciable. Este patrón refuerza la interpretación obtenida en otras gráficas temporales.

Analizando la [Figura 9](#), en conjunto, ambas gráficas vuelven a reforzar lo comentado sobre cómo regiones concretas influyen en mayor medida sobre el logit. Sin embargo, algo interesante de las gráficas de esta figura es que hay menos picos que en las gráficas de Occlusion, lo que sugiere que aquí se capta una relevancia más repartida por la señal.

Observando qué curva presenta valores superiores, nos damos cuenta de que en la que representa el “logit”, la clase “bona” tiene mayor fuerza, mientras que en la que representa el “margen”, ocurre lo contrario. Esto se puede deber a que en audios de la clase “bona”, hay una evidencia relativamente estable que sostiene el logit de la clase durante el audio, mientras que en audios de la clase “spoofs”, hay regiones que no necesariamente maximizan el logit de forma tan uniforme, pero que, sin embargo, sí resultan importantes a la hora de separar la muestra respecto a “bona”.

Antes de analizar la [Figura 10](#), conviene recordar que estas figuras no representan explicaciones obtenidas directamente sobre AASIST, sino sobre modelos **surrogate**. Por ello, su interpretación debe entenderse como una explicación complementaria: no describen de forma interna el

funcionamiento de AASIST, pero sí permiten identificar qué rasgos acústicos interpretables resultan más influyentes en modelos que aproximan su comportamiento.

En cuanto a las figuras, la de la izquierda corresponde al modelo clasificador y la de la derecha al modelo regresor. Consideradas en conjunto, ambas muestran un patrón bastante coherente. La característica más influyente, especialmente en el clasificador, es **RMS**, es decir, la energía media del audio. Junto a ella, también destacan dos familias de variables especialmente relevantes: los **formantes** y los **MFCC**. La importancia observada en las frecuencias bajas y bajas-medias parece estar asociada, al menos en parte, a la estructura formántica y a la envolvente espectral de la voz, rasgos estrechamente relacionados con la naturalidad del habla. En este sentido, la importancia de variables como *f1_mean*, *f2_mean* o *f3_mean* refuerza la atribución que otras técnicas XAI ya sugerían para estas regiones del espectro.

Por otro lado, la presencia destacada de varios coeficientes **MFCC** sugiere que la clasificación no depende únicamente de rasgos gruesos del espectro, sino también de detalles más finos de su envolvente. Esto apoya la idea de que existe una textura espectral relevante para distinguir entre audios naturales y generados.

En conjunto, estas gráficas refuerzan las conclusiones obtenidas mediante otras técnicas de explicabilidad: por un lado, la importancia de las frecuencias bajas y bajas-medias; por otro, la existencia de rasgos globales complementarios, dispuestos por todo el audio, que contribuyen a la discriminación entre audios “bona fide” y “spoof”.

Cabe mencionar que la elevada importancia de RMS también aconseja interpretar este resultado con cautela, ya que parte de su capacidad discriminativa podría estar relacionada no solo con la naturalidad de la señal, sino también con diferencias globales de energía entre muestras.

3.6.6. Perceptrón

Como podíamos esperar, se produce un gran sobreajuste en el modelo por lo que no emplearemos ninguna de las métricas típicas para evaluarlo.

Las conclusiones que podemos sacar sobre este modelo a la vista de los resultados son que para crear un modelo de clasificación desde 0 y que sea eficaz en su labor, necesitaríamos muchísimos más audios de los que disponemos, además, en lo que se refiere a la posterior tarea de aplicar elementos de IA explicable al modelo, al usar embeddings como entrada, no podemos aplicarle directamente ninguna de esas técnicas.

3.6.7. Wav2Vec2

Evaluación del modelo

1. Accuracy = 0.90: Porcentaje de aciertos.
2. F1-Score = 0.909: Balance precisión-recall.
3. ROC-AUC = 0.92: Capacidad discriminativa.

Las métricas nos muestran que el modelo tiene un porcentaje de aciertos del 90 % y una capacidad discriminativa del 92 %, lo que nos permite concluir que en general es un modelo bastante preciso.

Salida del modelo

Ahora vamos a procesar la salida del modelo para hacerla más interpretable, ya que actualmente el modelo devuelve una tupla con dos valores de pertenencia a las distintas clases. Lo que se busca es transformar esto en un número entre 0 y 1 que indique el grado de pertenencia del audio a la clase generada. Cuanto más cercano a 1, más pertenece ese audio a la clase generada.

Para lograrlo, se aplica una función sigmoide que mide qué tan mayor es x_2 (clase generada) con respecto a x_1 (clase real):

$$\frac{1}{1 + e^{-(x_2 - x_1)}}$$

donde:

- x_1 : valor de pertenencia devuelto por el modelo a la clase 1.
- x_2 : valor de pertenencia devuelto por el modelo a la clase 2.

Por último, estos nuevos resultados se guardan en un *dataframe* que servirá para aplicar posteriormente modelos de XAI. Este dataframe contiene el nombre del audio, el valor devuelto por el modelo y la categoría binaria a la que pertenece.

Cabe recalcar que en los nombres la primera letra indica el idioma (E: español, I: inglés), el número es su identificador y la última letra indica si es natural (N) o generado (G).

Podemos ver en los resultados de las predicciones cómo el modelo falla clasificando algunos audios naturales como generados. Esto puede deberse a ruido o a otras imperfecciones en los audios.

Name	Score	Label
I_32_N	0.34013117	0
E_47_G	0.77316723	1
I_12_N	0.29058788	0
I_3_N	0.76783361	0
E_7_G	0.77251038	1
I_2_G	0.66493000	1
E_11_N	0.30043659	0
E_23_N	0.26705380	0
E_16_G	0.77921332	1
I_6_G	0.76163065	1
I_49_N	0.27409548	0
I_5_G	0.65449655	1
E_35_G	0.78612722	1
I_41_N	0.27259401	0
E_37_N	0.33313831	0
E_2_N	0.37300991	0
E_40_G	0.73222093	1
E_41_G	0.77680456	1
I_2_N	0.74845369	0
E_43_G	0.76308837	1

Resultados explicación Wav2Vec2

Los resultados de los primeros cinco surrogate models implementados (orientados a datos tabulares) son los siguientes.

Cabe recalcar que para todos los modelos usaremos la instancia I_12_N cuya clase es 0 (audio real) y el modelo Wav2Vec2 predijo 0,2905, es decir, lo clasificó como real.

En cuanto al Shap mostraremos dos gráficas por modelo y la predicción del modelo para ese audio. La primera gráfica nos mostrará la explicación global para ese modelo mientras que la segunda nos mostrará una explicación local de uno de los audios.

Aquí proporcionamos un enlace a la tabla con todas las características que extrajimos del audio: ??

1. Energía: En este modelo 3.27 podemos ver como la kurtosis y el silence ratio son las características que más influyen mientras que el rms tiene una influencia nula en la predicción.

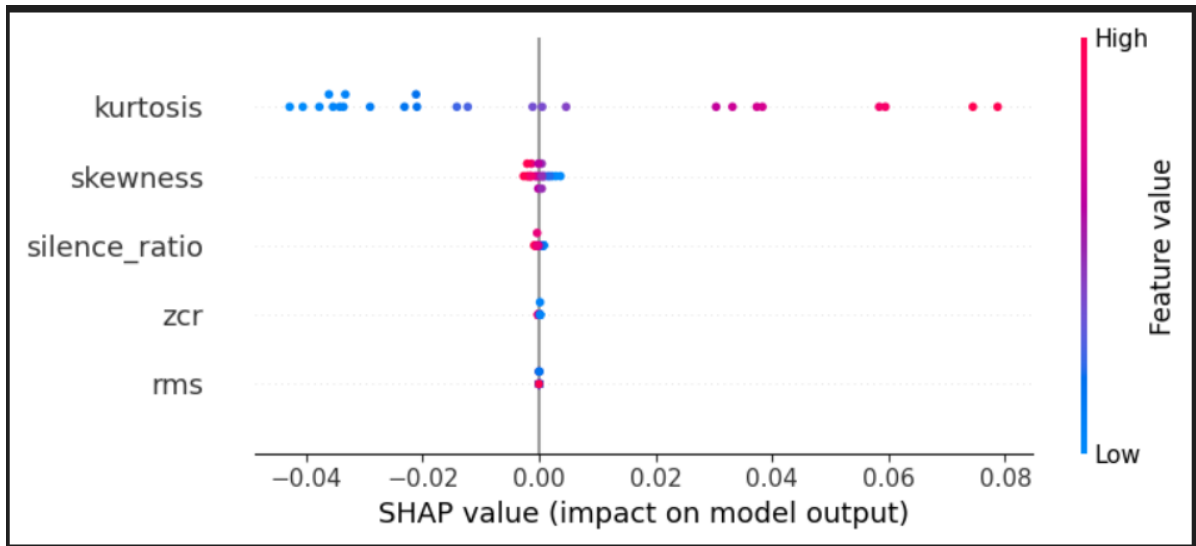


Figura 3.27: Shap global energía.

Antes de pasar con la explicación local debemos detenernos en la predicción del modelo que fue de 0,52822. Esto nos da a entender que ninguna de las características observadas por el modelo son verdaderamente relevantes a la hora de decidir si el audio es real o generado.

Pasando ahora con la explicación local 3.28 podemos ver que ninguna de las características influye demasiado en la predicción pero si que se mantiene el orden global y se puede ver con cuanta fuerza influirían y en que dirección. En este caso y por la fuerza que se le presupone a las características, este audio es más real que generado.

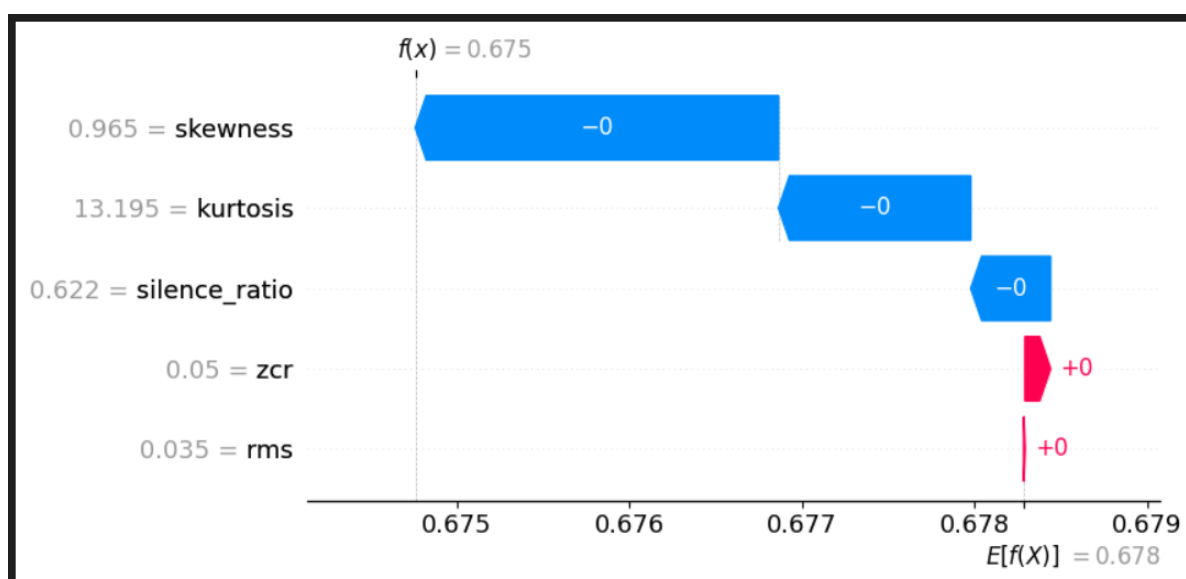


Figura 3.28: Ejemplo concreto.

2. Espectrales: En este modelo las características más influyentes son el spectral_centroid, el spectral_rolloff y el spectral_bandwidth y la predicción del surrogate model de los espectrales fue de 0,49076.

Imágenes del shap para el surrogate model de los espectrales: [11](#)

3. Pitch: En este modelo hay un claro dominio por parte del pitch std en comparación con el pitch mean. La predicción del surrogate model del pitch fue de 0,48127.

Imágenes del shap para el surrogate model del pitch: [12](#)

4. Formantes: En este modelo la f1 y la f3 tienen una influencia similar y muy superior a la de f2. La predicción del surrogate model de las formantes fue de 0,57039.

Imágenes del shap para el surrogate model del pitch: [13](#)

5. MFCC: En este modelo los coeficientes cepstrales de Mel 5,6 y 7 son los que más influyen mientras que los coeficientes 1,13 y 12 son los que menos influyen. La predicción del surrogate model de los mfccs fue de 0,29669.

Imágenes del shap para el surrogate model del pitch: [14](#)

A la vista de todas las predicciones de los surrogate models (Energía: 0,52822, Espectrales: 0,49076, Pitch: 0,48127, Formantes: 0,57039 y MFCC: 0,29669) podemos concluir que los mfccs son el grupo de características más relevante a la hora de decidir si este audio (I_12_N) es real o generado.

Ahora, buscando también algo que complemente la explicación del Shap, vamos a usar Lime para explicar los mismos modelos y la misma instancia particular (I_12_N).

1. Energía: En este modelo podemos ver como la kurtosis es la única característica que tiene un impacto relevante en la predicción del modelo. [3.29](#)

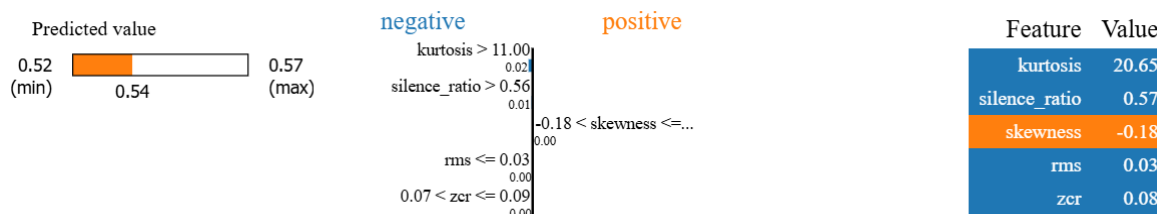


Figura 3.29: Explicación local energía Lime.

2. Espectrales: En este modelo podemos ver como todas las características tienen algo de impacto, sin embargo predominan claramente el espectral bandwidth y el espectral centroid hacia reducir la predicción y el espectral rolloff hacia aumentarla.

Imagen del lime de los espectrales: [15](#)

3. Pitch: En este modelo se ve como el pitch std influye más que el pitch mean. Ambas características empujan la predicción hacia abajo.

Imagen del lime del pitch: [16](#)

4. Formantes: En este modelo la f1 domina sobre las demás siendo la que más empuja la predicción hacia alguna dirección, en este caso, contribuye a reducirla mientras que la f3 la empuja hacia arriba aunque con menos fuerza de lo que la f1 lo hace hacia abajo.

Imagen del lime de las formantes: [17](#)

5. MFCC: En este modelo los coeficientes 5,7,9 y 11 son los que más influyen en las decisiones del modelo siendo que el 5 es el coeficiente que más disminuye la predicción mientras que el 7, el 9 y el 11 son los que más la aumentan.

Imagen del lime de los mfccs: [18](#)

Ahora que ya tenemos tanto Shap como Lime de todas las características de los audios, vamos a agregar los resultados de los cinco surrogate models para ver de forma general cuales son las características más influyentes a la hora de clasificar los audios en reales o generados.

Para esta tarea se ha hecho lo siguiente, primero calculamos la importancia promedio de cada característica usando sus valores Shap, luego calculamos la importancia de cada grupo de características y con esto podemos promediar la importancia de cada característica entre todos

los grupos de características. Por último los ordenamos de mayor a menor importancia y los mostramos.

En esta gráfica 3.30 podemos ver las 15 características más importantes para la predicción destacando sobre todo el centroide espectral y el 5º coeficiente cepstral de Mel.

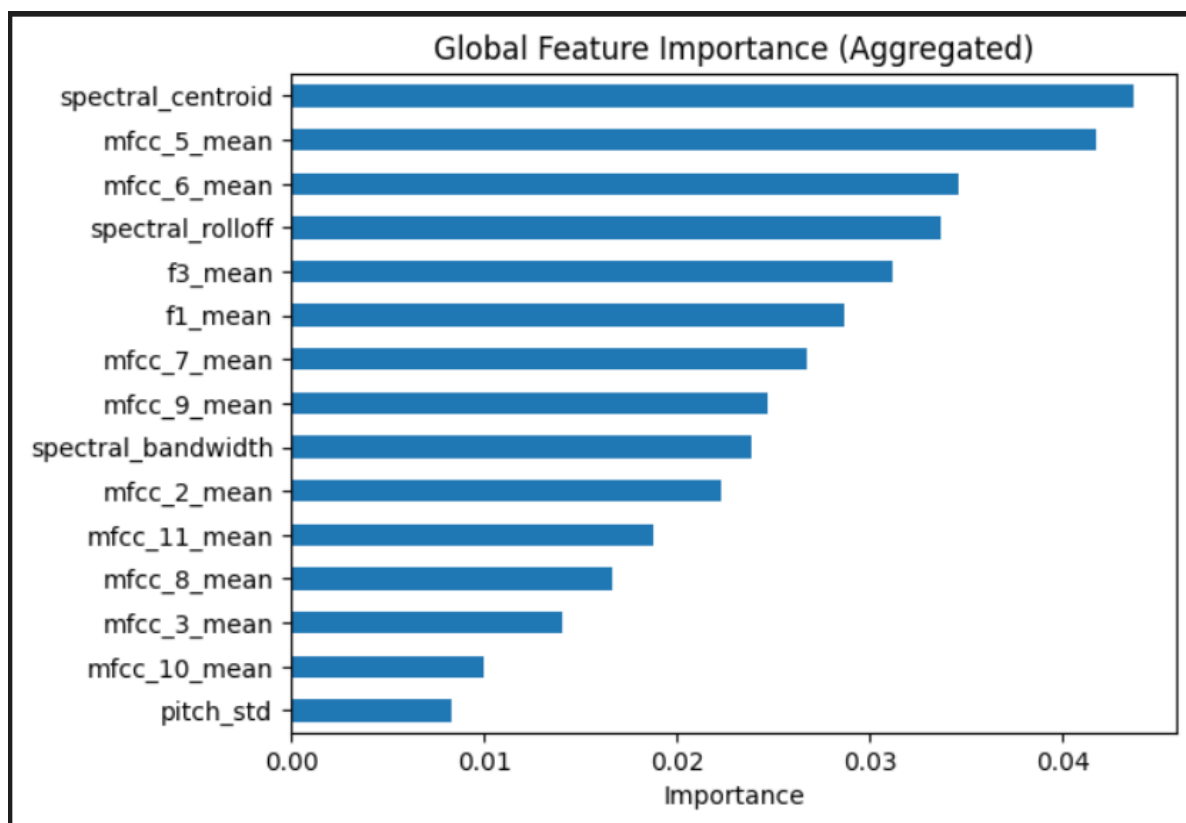


Figura 3.30: Ranking características Shap.

Para Lime hacemos exactamente lo mismo.

Aquí 3.31 vemos que destacan las mismas características que en la de Shap pero con ligeras variaciones de posición entre ellas.

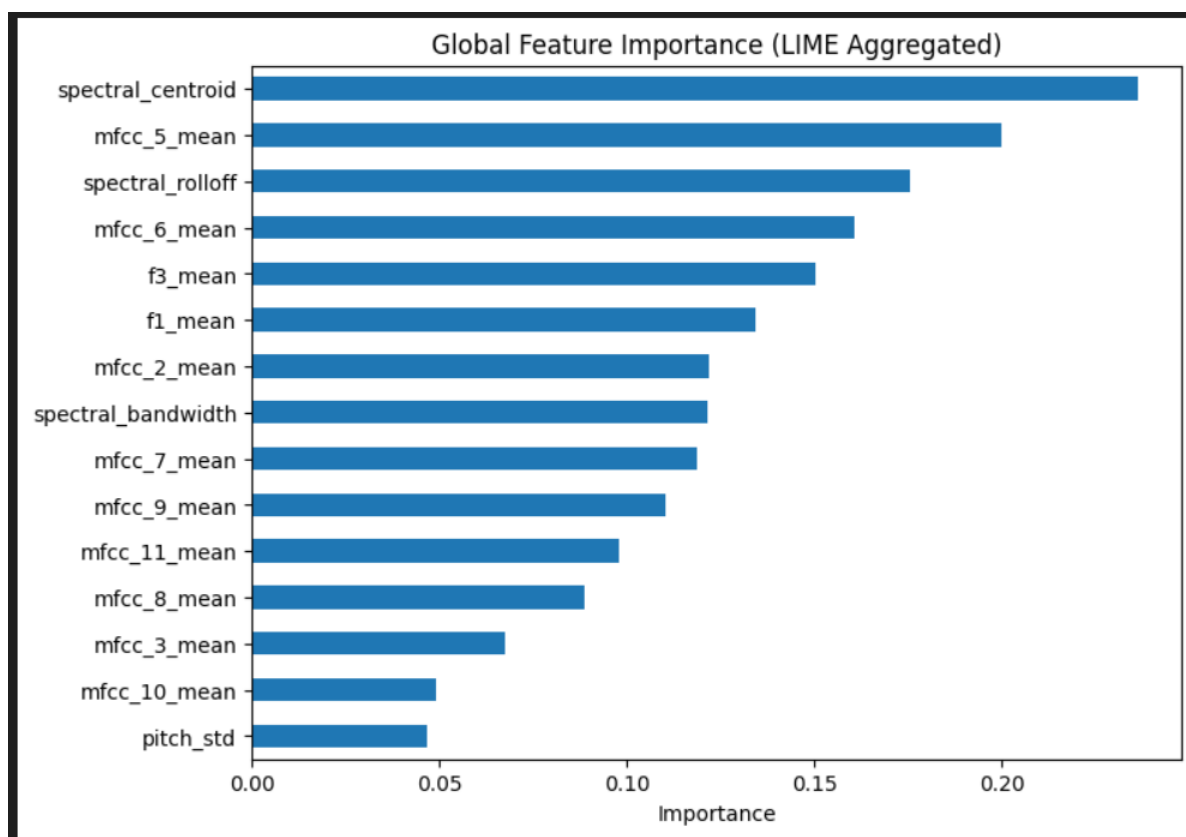


Figura 3.31: Ranking características Lime.

Para acabar con la explicación de las características, vamos a juntar ambas explicaciones en una sola combinada para obtener el orden global de influencia de las características. Este orden será calculado haciendo la media aritmética de su importancia en Shap y Lime.

En esta gráfica combinada [3.32](#) podemos ver el ranking global de las características según su importancia. Podemos observar que a la hora de agregar ambas gráficas las diferencias respecto a las gráficas anteriores de los rankings de Shap y Lime no son demasiado grandes ya que ambas exponían uno similar.

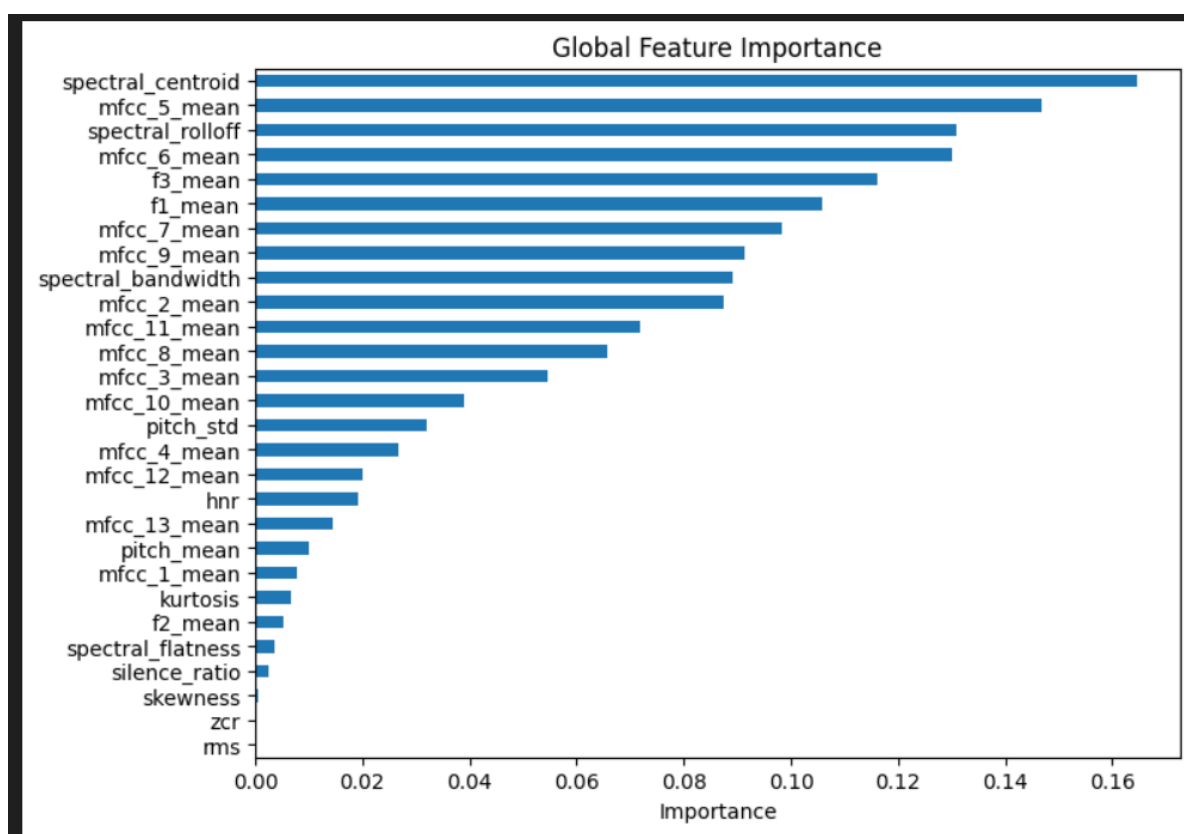


Figura 3.32: Ranking agregado de características.

Sobre el sexto surrogate model aplicaremos Shap, Lime, Integrated gradients y Grad cam.

De cada una de estas técnicas sacaremos tres imágenes. La dos primeras se corresponderán con la actuación de la técnica de XAI sobre un audio concreto (I_12_N) y el espectrograma de dicho audio. La última imagen nos mostrará los patrones de actuación global de la técnica empleada.

Shap

Aquí podemos ver en las imágenes 3.33 cómo la parte del audio que más influye en la predicción es la parte final del audio, correspondiente a las bandas de mel entre 25 y 45, que simbolizan frecuencias medio-altas. Estas bandas contienen las formantes F2 y F3, así como elementos clave para la inteligibilidad del habla. Si nos fijamos en el espectrograma, esto coincide con el final, donde parece que el audio está distorsionado. Esto se aprecia claramente al escuchar el audio (I_12_N: audio 12 natural en inglés), donde la última palabra es “next”, y ese final que parece distorsionado corresponde al sonido de la pronunciación del grupo consonántico “xt”.

También se puede observar cómo la parte donde ya no hay espectrograma, en la explicación de SHAP, no tiene prácticamente influencia.

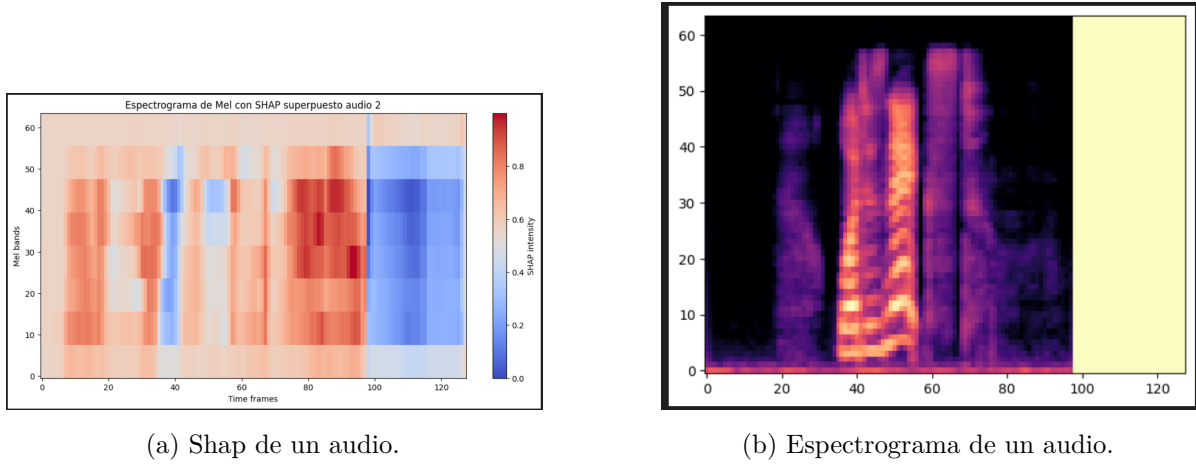


Figura 3.33: shap local espectrogramas Wav2Vec2

En esta imagen 3.34 sobre la influencia global del shap podemos ver como las partes que más influyen en la clasificación del audio son el principio y el final del audio. Comparando esta gráfica con la anterior podemos confirmar la importancia que tienen las bandas de mel medias. También podemos ver como el audio elegido para la explicación global no es representativo de la explicación global ya que esta última si que usa mucho la parte final del audio para la explicación.

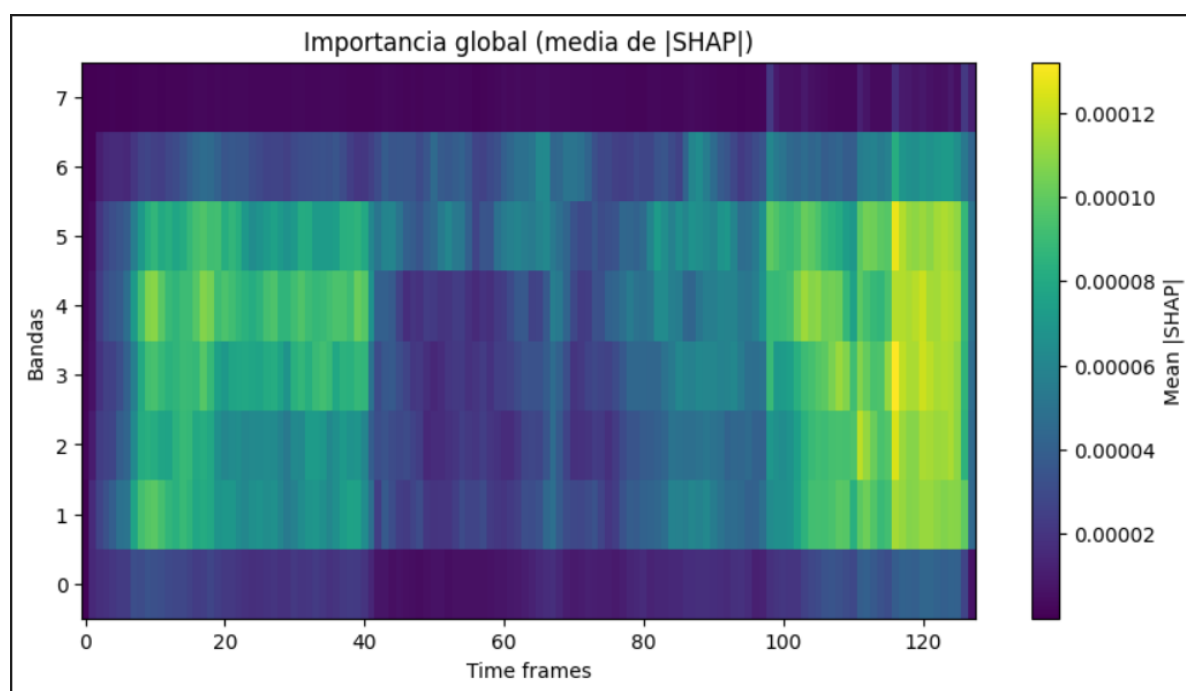


Figura 3.34: Shap general de los espectrogramas

Lime

En esta imagen ?? se muestra la explicación local de lime sobre mismo audio que en el shap. En ella podemos ver como, al igual que en el shap, las bandas medias son las más relevantes y la pronunciación final es lo que más mueve la predicción hacia la clase real. Por último podemos ver como en estas bandas medias al principio estas empujan la predicción hacia generado, esto se puede deber a que el audio tarda en comenzar y a como pronuncia el principio de la primera palabra “what”.

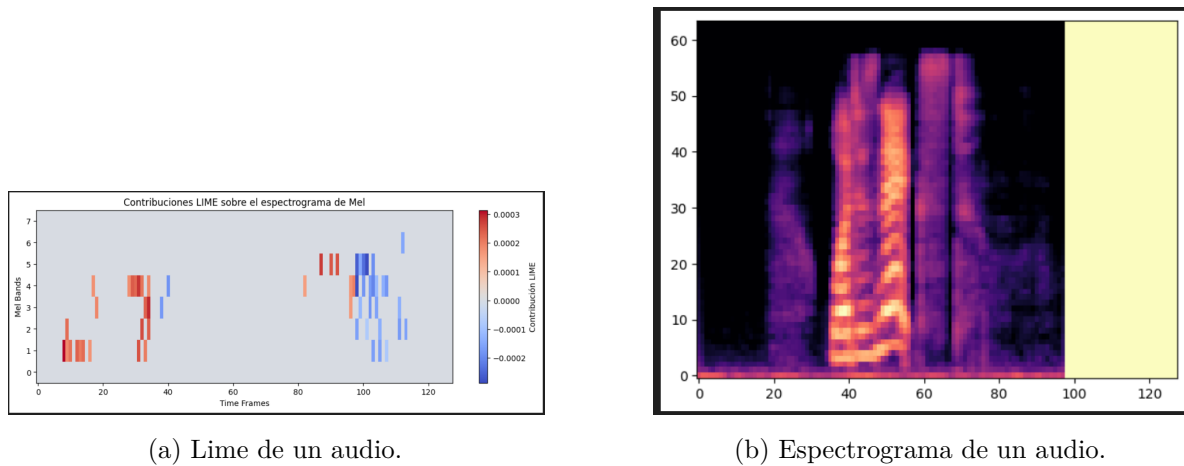


Figura 3.35: lime local espectrogramas Wav2Vec2

Aquí ?? a diferencia de en shap, según lime, el modelo toma las decisiones principalmente gracias al principio de los audios aunque el final también tiene algo de influencia.

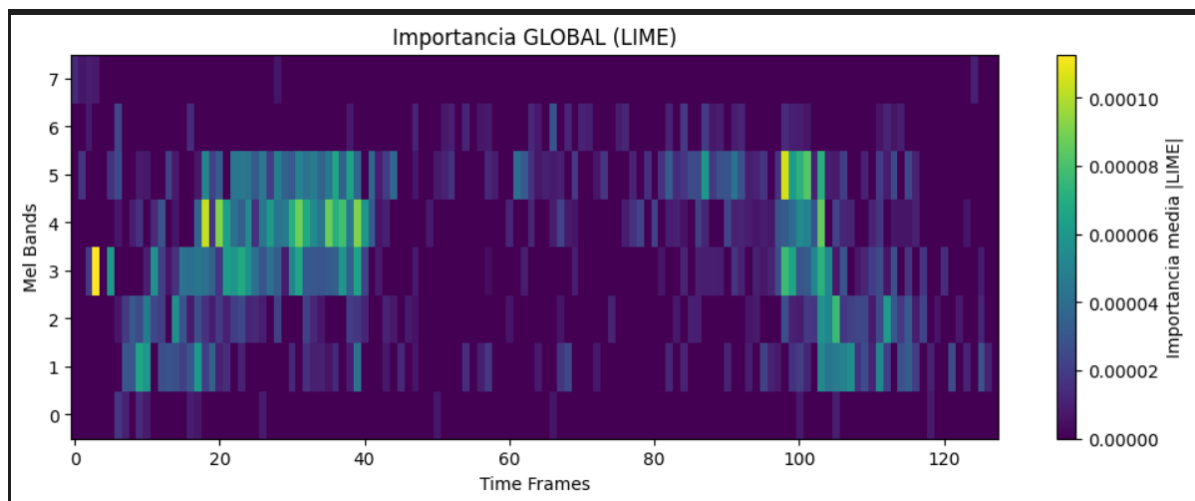


Figura 3.36: Lime general de los espectrogramas

Integrated gradients

Aquí ?? podemos ver como no está analizando el audio del todo bien ya que las partes más rojas son las que más influyen a una predicción hacia generado pero si vemos el espectrograma del audio, en esas partes como tal no hay nada, son zonas negras, también la gran zona azul del final se corresponde con la parte donde no hay ni siquiera espectrograma. Si ignoramos esas partes y nos centramos solo donde si que hay evidencia de audio, podemos ver como esas son

zonas más blancas - azuladas que empujan la predicción hacia real. También se ve como los dos golpes de audio más fuertes en el espectrograma son las partes que más empujan la predicción hacia real.

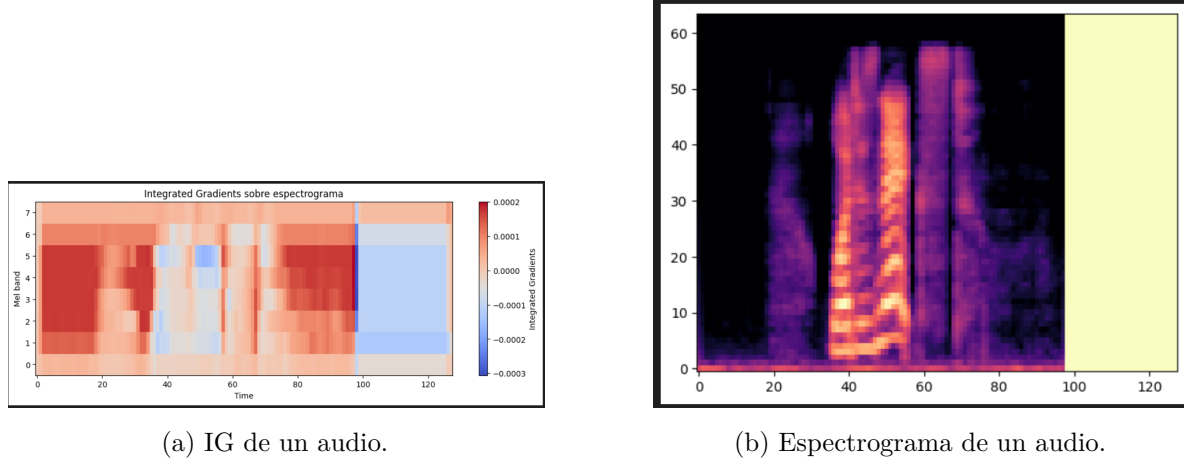


Figura 3.37: IG local espectrogramas Wav2Vec2

En la predicción global 3.38 se puede apreciar cierta similitud con las dos explicaciones generales anteriores donde las principales zonas de influencia en la clasificación son la parte inicial y la parte final de los audios en las bandas medias.

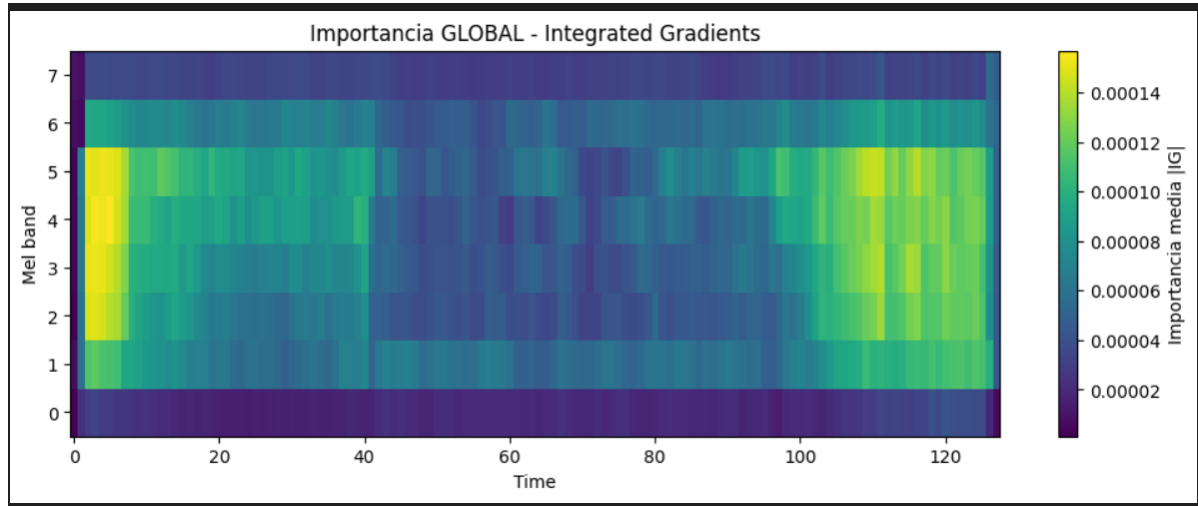
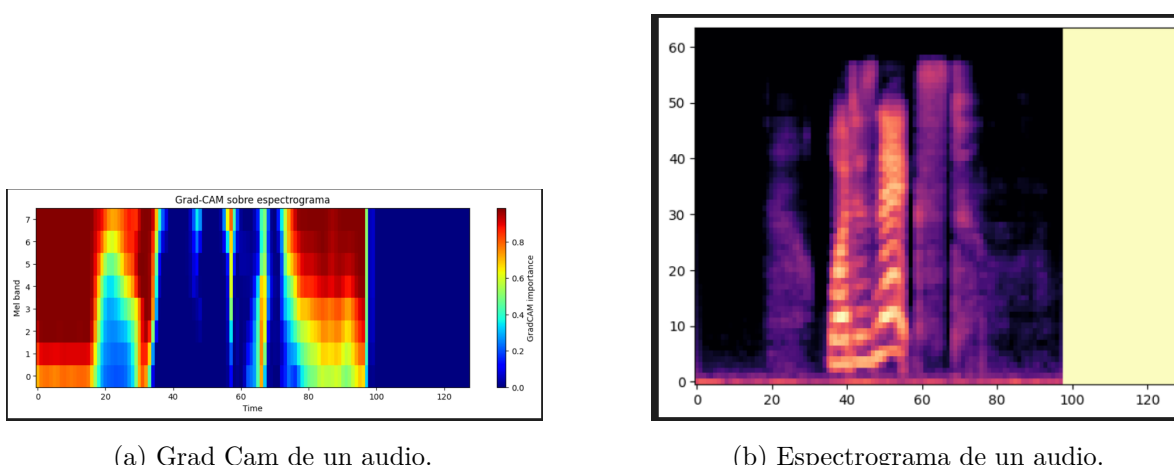


Figura 3.38: Integrated gradients general de los espectrogramas

Grad Cam

Aquí ??, al igual que en la explicación local de integrated gradients, podemos ver que en las partes donde no hay audio son las partes que más influyen en la predicción aunque aquí si que distingue bien que la parte final donde no hay espectrograma no tiene ningún peso en la predicción. De todas formas, fijándonos donde hay audio podemos apreciar como el primer golpe de voz y la parte final del audio tienen más influencia en la predicción que el resto del mismo.



(a) Grad Cam de un audio.

(b) Espectrograma de un audio.

Figura 3.39: Grad Cam local espectrogramas Wav2Vec2

La explicación global del grad cam 3.40 se caracteriza por darle el mayor peso a los instantes iniciales del espectrograma, dándole un peso prácticamente nulo al centro del espectrograma, curiosamente donde están los audios, y otorgándole un peso relevante también al final del espectrograma.

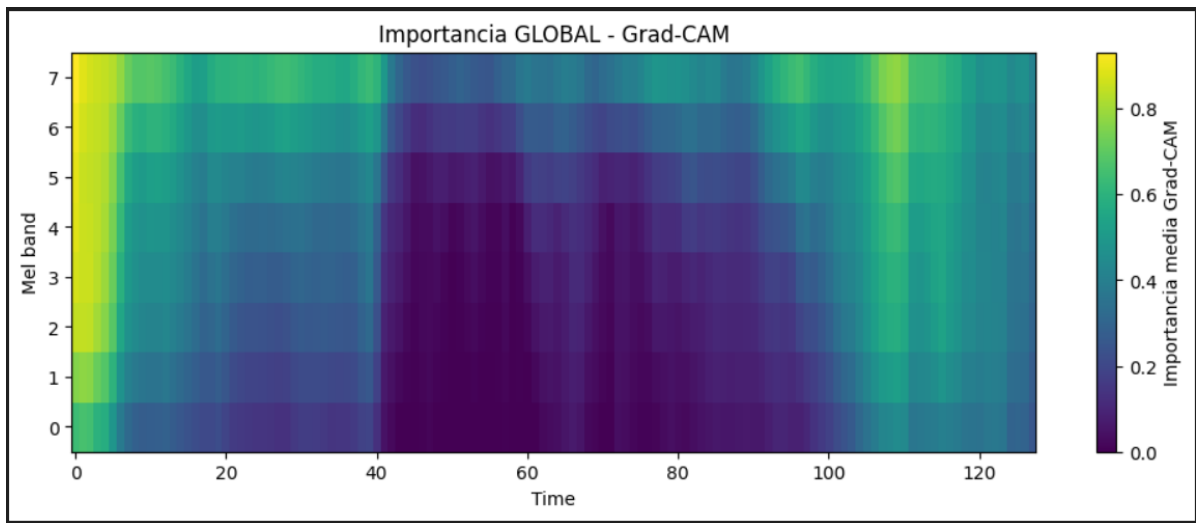


Figura 3.40: Grad cam general de los espectrogramas

Ahora vamos a ver como se ve la agregación de los cuatro modelos de XAI sobre los espectrogramas. Esto se ha calculado para cada frame del espectrograma, se ha sacado su valor en cada una de las explicaciones globales de los distintos modelos y se ha hecho la media aritmética sobre ellos. Antes de poder hacer esto, hemos tenido que normalizar todas las explicaciones ya que estaban en diferentes escalas.

La agregación de las explicaciones globales [3.41](#) nos deja claro donde se encuentran las zonas más influyentes siendo el principio y el final del espectrograma dejando al centro con un peso mucho menos relevante.

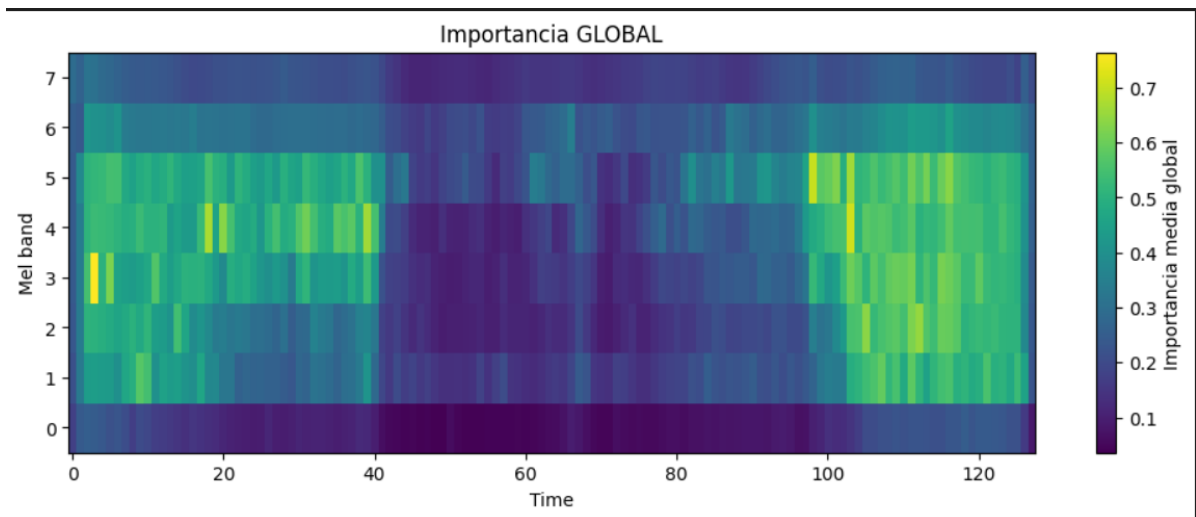


Figura 3.41: Influencia general de los espectrogramas

Como conclusiones podemos sacar que a la vista de los modelos de explicación, parece que el modelo Wav2Vec2 se fija más en los alrededores del audio que en el audio en sí mismo. Esto puede explicarse gracias a las siguientes propiedades de los espectrogramas de voz:

1. Coarticulación: Este es un fenómeno típico de la voz humana donde esta no está formada por unidades aisladas, es decir, cada sonido depende de los anteriores y posteriores. Esto hace que las transiciones entre fonemas contengan más información que el fonema en sí haciendo que lo que está alrededor del audio tenga más peso.
2. Dinámica espectro-temporal: Las partes centrales de los audios tienden a ser más estables y redundantes mientras que las zonas de cambio contienen mayor variabilidad lo que hace que contengan la mayoría de la información discriminativa del audio.
3. Estructura de formantes y transiciones: Las trayectorias de las formantes, claves para identificar fonemas, suelen estar en los bordes de los segmentos por lo que las zonas de alrededor del audio contienen más información dinámica de las formantes.
4. Efectos de ventana del espectrograma: Como el espectrograma se calcula usando ventanas, cada punto del espectrograma mezcla información de un intervalo temporal siendo que en los bordes del espectrograma se incluyen partes de dos sonidos distintos haciendo que estos sean más informativos al incluir mezclas de contextos.
5. Relación señal-ruido: En muchos audios el inicio y el final de los mismos pueden tener cambios de energía o ruido de fondo que el modelo puede usar como señales útiles.
6. Información prosódica: Los elementos como la entonación, el ritmo o el énfasis no están concentrados en un punto sino que ocurren durante las transiciones.

3.6.8. Regresión Logística

3.6.9. KNN

3.7. Discusión de los resultados

Esto puede deberse a que los audios, cuando se escuchan directamente por una persona, se detectan algunos donde no se entiende lo que está diciendo el locutor y otros donde pese a entenderse mejor, se notan claramente las pausas “robóticas” y otros aspectos que nos llevan a pensar que dichos audios son falsos.

3.8. Conclusiones

4. Conclusiones

4.1. Trabajo futuro

El trabajo desarrollado establece una base sólida para la detección de “deepfakes” de audio mediante el uso de distintos métodos de IA y técnicas XAI. Actualmente, el producto final del trabajo consiste en un pipeline que permite aplicar, sobre un audio de entrada, todos los modelos de clasificación y explicación estudiados en el capítulo de metodología. Este pipeline genera como salida los resultados obtenidos por los distintos modelos de IA, combinados con gráficas resultantes de aplicar XAI sobre dichos resultados.

Aunque estas representaciones permiten observar qué factores han influido en la decisión de los modelos, su interpretación requiere de ciertos conocimientos técnicos y la explicación no siempre resulta suficientemente directa. Por ello, como trabajo futuro se propone el desarrollo de un explicador que actúe como una última capa entre los valores calculados por las técnicas XAI aplicadas en el pipeline y el resultado final mostrado. La idea principal sería transformar el sistema actual, centrado en la visualización, en un enfoque basado en la interpretabilidad asistida. Es decir, el sistema no se limitaría a presentar gráficas, sino que también ofrecería explicaciones más fácilmente interpretables sobre la decisión final y sobre las evidencias que apoyan esta clasificación.

Asimismo, se sugieren dos tipos de explicadores que podrían resultar complementarios entre ellos. El primero estaría basado en **modelos grandes de lenguaje o LLM**. El objetivo de este primer explicador sería convertir salidas técnicas en explicaciones textuales más comprensibles y sintetizadas. Este enfoque presenta varias ventajas. En primer lugar, permitiría a usuarios no expertos comprender el motivo de la clasificación final. En segundo lugar, facilitaría la comparación entre los resultados de los distintos modelos. En tercer lugar, permitiría generar informes automáticos que resuman las evidencias principales utilizadas para la clasificación del audio.

El segundo tipo de explicador propuesto estaría basado en **contrafactuales**. Mientras que las técnicas XAI responden principalmente a qué partes o características han influido en la decisión

final, los contrafactuales responden a una pregunta diferente: “¿qué tendría que cambiar en el audio para que su clasificación fuera distinta?”. Este tipo de explicación es especialmente interesante porque también permite analizar la sensibilidad de los modelos ante modificaciones concretas de la entrada.

Este tipo de explicador podría indicar qué alteración mínima hace que los modelos cambien la clasificación original del audio. Según el tipo de modificación se contemplan tres posibilidades:

- **Contrafactuales por segmentos temporales:** el sistema podría modificar, ocultar o sustituir segmentos determinados del audio y observar cómo cambian las predicciones.
- **Contrafactuales sobre espectrogramas:** este enfoque actuaría igual que el anterior pero realizando modificaciones sobre la representación espectral del audio.
- **Contrafactuales sobre características acústicas:** el explicador modificaría características acústicas como los MFCC o el pitch, con el objetivo de ver si se altera la salida de los modelos.

Sin embargo, esta propuesta contiene limitaciones que deben considerarse. En primer lugar, es necesario garantizar la fidelidad de las explicaciones respecto al comportamiento real de los modelos. En segundo lugar, se debe asegurar la trazabilidad, es decir, cada afirmación generada por el explicador se debe poder relacionar con una salida concreta del pipeline. En tercer lugar, los contrafactuales deben ser realistas y no consistir en modificaciones alejadas de la coherencia del dominio del audio.

5. Pipeline

Como cierre para el proyecto, se ha desarrollado un pipeline capaz de aplicar, al audio recibido como entrada, todos los modelos de IA y XAI explicados en el capítulo de [Metodología](#). Este pipeline tiene como objetivo reunir en un único “notebook” todos los modelos desarrollados a lo largo del proyecto. De este modo, permite facilitar la comparación de resultados y explicaciones entre modelos.

Cabe mencionar que el diseño de este pipeline constituye una de las aportaciones prácticas del proyecto, al integrar en una misma estructura modelos de distinta naturaleza y técnicas de explicabilidad complementarias. Esto permite analizar un mismo audio desde múltiples perspectivas, comparando no solo el resultado de clasificación, sino también la forma en que cada modelo justifica su decisión.

Con el fin de representar su estructura general, se ha diseñado un diagrama de flujo simplificado. Como se puede observar en la [Figura 5.1](#), los modelos se han dividido en dos grupos según el tipo de entrada que utilizan: por un lado, aquellos que operan sobre características extraídas del audio y, por otro, los que trabajan directamente con la forma de onda cruda. En cada grupo se indican tanto los modelos implementados como las técnicas de XAI aplicadas a cada uno de ellos.

Para poder reutilizar los modelos entrenados durante el proyecto y evitar su reentrenamiento para cada caso, hemos empleado distintas técnicas para guardarlos, que se explicarán en el apartado correspondiente a cada modelo.

La primera parte del “notebook” consta de una celda inicial con los imports necesarios, una segunda celda donde se especifica la ruta del audio que se quiere analizar y una tercera celda en la que se extraen las características del audio mediante la función “`extract_features`” definida en el archivo `funciones.py`. A partir de ahí, cada modelo tiene su propia sección, en la que se muestra tanto la predicción como las explicaciones obtenidas con las técnicas de XAI asociadas a dicho modelo.

Además, el “notebook” está organizado en secciones correspondientes a cada modelo, lo que permite ejecutar cada uno de ellos de forma independiente.

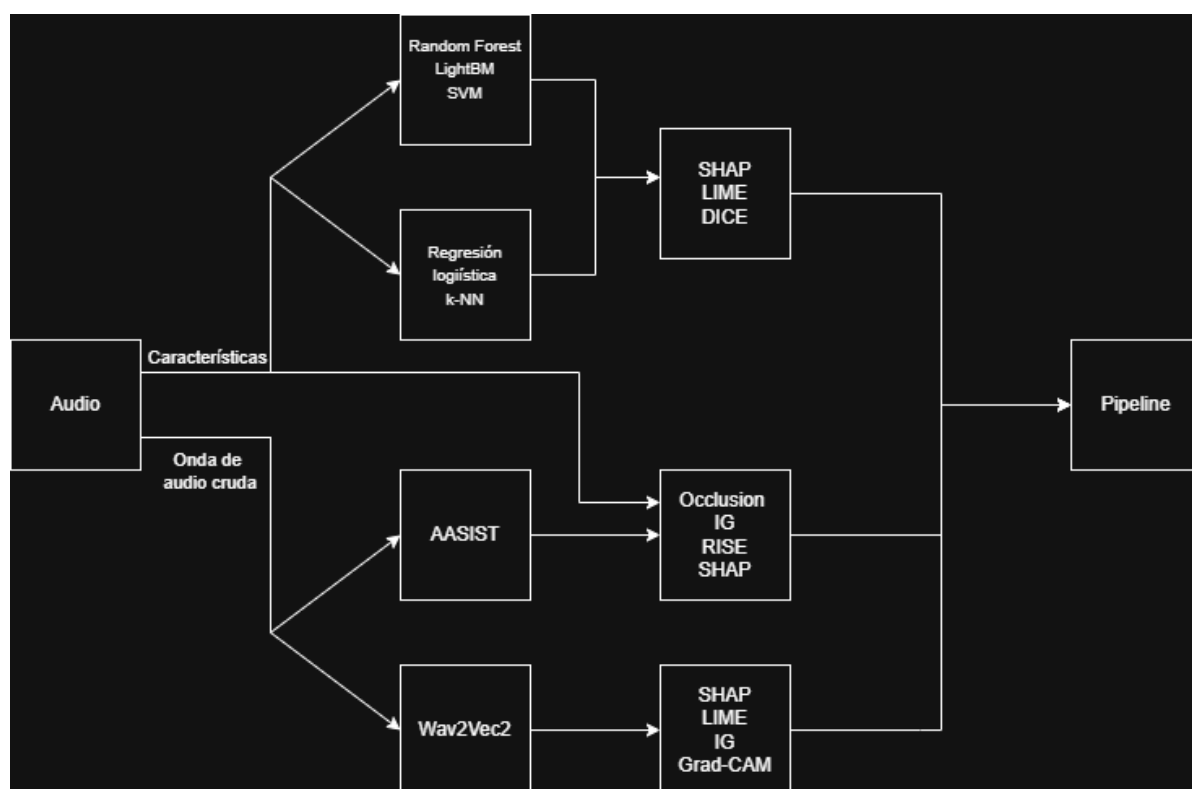


Figura 5.1: Arquitectura del pipeline.

5.1. AASIST

Para poder usar el modelo, lo primero ha sido guardarlo con la función `torch.save()`. Hemos usado esta función porque nos permite, no solo guardar el modelo para calcular la predicción final, sino también guardar la capa “`sinc-convolution`” para poder aplicar Integrated Gradients sobre ella. Para cargar el modelo, hemos usado la función `torch.load()` sobre el archivo guardado y la clase `model` del archivo `AASIST.py` de la carpeta `Modelos/German`.

Una vez cargado, se usan las funciones cargadas del archivo `funciones_AASIST.py` que contiene todas las funciones necesarias para poder aplicar el modelo y las distintas técnicas de XAI. Primero, se ejecuta la función “`score_audio`” que devuelve los resultados en forma de diccionario con cuatro claves: “`logits`”, un tensor que contiene los “`logit`” para ambas clases; “`score_bonafide`”, el valor de “`logits`” para la clase “`bona fide`”; “`prob_bonafide`”, la probabilidad de que el audio sea “`bona fide`” calculada a partir del “`logit`”, y “`pred_clase`”, que muestra la clase predicha por el modelo.

A continuación, se ejecuta la función “`aplicar_XAI`” que muestra las gráficas resultantes de aplicar las técnicas de XAI explicadas en [AASIST](#).

5.2. Wav2Vec2

Para poder reutilizar el modelo, es necesario guardar tanto el modelo entrenado como el extractor de características. Para ello, se utiliza el método `save_model()` de la clase `Trainer`, junto con el método `save_pretrained()` de `Wav2Vec2FeatureExtractor`. Ambas clases provienen de la librería `Transformers`.

Una vez guardados, con la función `from_pretrained` de las clases `Wav2Vec2ForSequenceClassification` y `Wav2Vec2FeatureExtractor` puedes cargar tanto el modelo como el extractor de características.

Tras cargar lo que necesitamos, emplearemos la función `generar_predicciones` del archivo `funciones_wav2vec2` a la que pasándole el modelo, el extractor y el audio te devolverá tanto la salida del modelo (`logits`) como su predicción para ese audio (`score`).

Para generar las explicaciones de ese audio necesitamos usar los 200 audios de nuestro dataframe junto con su label. Con los audios usaremos la función `preprocesamiento` de `funciones_wav2vec2` que nos devolverá los espectrogramas de todos los audios en forma de tensor de `torch` y agruparemos todos los labels en otro tensor.

Ahora que tenemos nuestro `X_train` (tensor de los espectrogramas) y nuestro `y_train`, podemos usar la función `build_surrogate` y posteriormente la función `fit` de `torch` para entrenar nuestro surrogate model sobre el que aplicaremos las técnicas de XAI.

Por último, usaremos la función `mostrar_graficas` de `funciones_wav2vec2` que nos generará la gráfica del espectrograma del audio y las explicaciones del Shap e Integrated Gradients.

Contribución de Iván Pastor Sacristán

- Investigación del estado del arte: Investigué sobre el estado actual de la detección de deepfakes. Durante la investigación leí dos papers. En el primero hablaban sobre adaptación de los modelos a nuevos conjuntos de datos distintos de los que se usaron para entrenarlo y probarlo. En el segundo paper se centraban en un modelo capaz de clasificar correctamente los audios, aún y cuando los generados por IA habían sido modificados tanto en el dominio de la frecuencia como en el del tiempo para añadirles defectos característicos de los audios humanos como el ruido.
- Prueba del dataset WaveFake: Hice una primera versión del extractor de características entre las que se incluyen rms, zer, pitch, mfcc's, duración, transcripción, espectral centroid, espectral rolloff y el espectral bandwidth.
- Prueba del dataset ASVSpooof2021: Usé el mismo extractor de características e hice un clasificador difuso en el que, basándome en las características extraídas, clasifique los audios por género y edad aparente.
- Modelo compuesto: El modelo se basa en hacer un embedding de las características con un perceptrón multicapa (MLP), hacer un embedding del espectrograma del audio con una red neuronal convolucional (CNN), concatenarlos, y luego usar esos embeddings como entrada de un MLP + clasificador difuso que es lo que se encargará realmente de la tarea de clasificación.
- Fine-tuning Wav2Vec2: Usé el modelo desarrollado por Meta Wav2Vec2 y le hice un fine-tuning con nuestro dataset para que se enfocara en la clasificación de audios. Luego procesé la salida del modelo para pasar de la tupla que me devuelve el modelo a un valor entre 0 y 1 que indique la confianza de que un audio sea generado.
- XAI sobre Wav2Vec2: Sobre el modelo Wav2Vec2, no se puede aplicar directamente IA explicable ya que la entrada son embeddings, para ello lo que he hecho ha sido dividir entre las características y el espectrograma de Mel. Para las características usa Shap y Lime sobre surrogate models que imitan

la salida de Wav2Vec2 y para los espectrogramas de Mel use una CNN a la que le apliqué Shap, Lime, Integrated gradients y Grad Cam.

- Estado del arte: Escritura de las secciones del perceptrón multicapa, del Wav2Vec2 y de la regresión Ridge.
- Datasets: Redacción de los experimentos con los datasets de WaveFake y ASVSpooof2021. Además me encargué de escribir el análisis del dataset final que usamos durante el proyecto.
- Modelos: Escritura de los modelos del perceptrón multicapa y del wav2vec2.
- Pipeline (memoria): Escritura de la sección del Wav2Vec2 del pipeline.
- Pipeline: Adaptación del código del Wav2Vec2 para que funcione a partir de un enlace único a un audio. También he creado un archivo con todas las funciones del código adaptado para que se ejecuten las predicciones y las explicaciones en una celda cada uno.
- Readme: Agregué la explicación de la carpeta que corresponde con los modelos que he desarrollado.

Contribución de Jorge Aured Zarzoso

- Investigué sobre las motivaciones para investigar sobre la detección de voz generada por IA con dos papers. En estos se hablaba de cómo las personas hemos perdido la capacidad de detectar correctamente cuando un audio es real o no.
- Investigué sobre qué características son relevantes en la voz y cómo extraerlas mediante código. Esto ya que es crucial saber qué representa cada característica para tener una noción de qué puede tener en cuenta los modelos a la hora de predecir si un audio es real o no.
- Investigué parcialmente sobre la extracción de gráficas de la voz, para su uso en redes convolucionales (CNN).
- paper deepShap
- paper peru
- Investigué sobre el estado del arte de los modelos LightGBM, Random Forest y SVM para clasificación. En dicha investigación profundicé en el funcionamiento de los modelos, viendo qué hiperparámetros tiene cada uno y para qué sirven, y qué ventajas tienen para la clasificación de audios en reales o generados.
- Investigué sobre datasets que nos pudieran servir para la clasificación. Encontré audioMNIST y HABLA, el primero de los números del 0 al 9 y el segundo son voces en español de hispanoamérica. audioMNIST solo cuenta con voces reales, mientras que HABLA tiene tanto clonaciones de voz como ataques (hacer que una voz se parezca a otra).
- He hecho un pipeline completo para cada uno de los tres modelos, entrenándolos de varias formas para ver cuál funcionaba mejor. Este pipeline consta de un modelo baseline, una fase de cross validation y un grid search optimizando los parámetros. En cada paso visualicé los resultados de los modelos viendo su rendimiento. Al final del pipeline guardé como archivo .pkl los mejores modelos

- He aplicado técnicas de Inteligencia Artificial explicable (XAI) a mi modelo óptimo de Random Forest, aunque las pruebas son extrapolables a los demás modelos. Dichas técnicas muestran cómo el modelo hace las predicciones. Las he hecho globales y locales. He aplicado SHAP (local y global), LIME (local) y DiCE (local, contrafactual).

Contribución de Germán Bravo Quintián

- Realicé una investigación inicial sobre el estado del arte de la detección de “deepfakes”. Para ello, leí varios papers contextualizando su situación y, en concreto, dos papers que presentaban dos modelos diseñados para esta tarea. El primer paper trataba sobre AASIST, un modelo cuya innovación principal era el combinar huellas espectrales con temporales para ser más robusto frente a ataques. El segundo, trataba sobre RawNet2, el cual es un modelo diseñado originalmente para la detección del hablante. Sin embargo, el paper presentaba una adaptación para detectar audio generado por IA que trabajaba con la onda de audio cruda.
- Además, en la primera investigación, busqué datasets que nos pudieran servir. Primero, encontré el paper sobre el dataset ASVspoof2021, el cual es introducido como una ampliación de los anteriores añadiendo mejoras como condiciones de transmisión, codificación y comprensión. Además de probar este primer dataset, probé dos más: In-the-Wild, que contiene audios reales y falsos en inglés con voces de 58 personas famosas; y ODSS (Open Dataset of Synthetic Speech), que incluye 158 voces en inglés, alemán y español, y en el que cada audio falso tiene su correspondiente versión real.
- Usando los datasets recién mencionados y los investigados por mis compañeros, creé un dataset propio para realizar las pruebas de los modelos durante el desarrollo del proyecto. Este dataset está formado por 200 audios, la mitad en inglés y la otra mitad en español, con cada audio falsificado con su versión real.
- Investigué sobre transcriptores de audio, intentando implementar el paquete de Python de Whisper pero debido a dificultades en la tarea lo descarté. Finalmente, implementé Vosk, creando una función que usando su modelo devolviera la transcripción.
- Profundicé la investigación sobre el estado del arte de AASIST. Para ello, leí 3 papers que incluían mejoras sobre el modelo básico ya mencionado. El primer paper presenta una solución para intentar solventar la dificultad de

AASIST para analizar correctamente los audios cortos. El segundo paper presenta otra versión mejorando las capas de atención y distintas partes de la arquitectura para el procesado del audio. El tercero presenta un modelo que recibe información de la persona objetivo a la que quiere suplantar para facilitar la detección de patrones artificiales.

- He hecho un notebook en el cual he desarrollado código para poder aplicar el modelo AASIST a nuestro dataset o cualquier audio entrante.
- He aplicado “fine-tuning” sobre AASIST para mejorar los resultados de este. Para ello, usé el dataset propio, dividiéndolo de varias formas y contemplando una separación entre idiomas para obtener con qué partición se conseguían los mejores resultados.
- Realicé una investigación exhaustiva sobre técnicas XAI que pudiera aplicar a AASIST. Como resultado, encontré las técnicas Occlusion y RISE. Además, para aplicar SHAP sobre AASIST, busqué modelos surrogate que pudieran ser entrenados para imitar a AASIST ya que por la complejidad de la arquitectura del modelo de clasificación no se le podía aplicar la técnica mencionada.
- He hecho un notebook en el cual desarrollo el código necesario para aplicar las técnicas XAI mencionadas en metodología a AASIST. Para aplicar Occlusion y RISE, tuve que desarrollar código propio ya que no son técnicas pensadas para la detección de audio y realicé adaptaciones de ellas manteniendo la idea. Asimismo, tuve que hacer código para aplicar Integrated Gradients sobre una de las capas internas de AASIST y también para poder preprocesar correctamente la información que necesitaban los modelos surrogate.
- Hice un último notebook en el que aplicaba las técnicas del segundo notebook a un pequeño conjunto del dataset.
- Creé un archivo que contiene todas las funciones desarrolladas en los tres notebooks mencionados para poder reusar el código de otros notebooks y poder utilizar AASIST desde el pipeline.
- Para poder aplicar AASIST desde el pipeline general del proyecto, investigué formas de poder comprimir el modelo ya entrenado en un archivo recuperable desde el propio pipeline. Para ello, realicé pruebas con varias técnicas ya que al tener que aplicar IG sobre una capa interna del modelo, necesitaba un archivo de guardado que no solo me guardara un modelo que me diera la probabilidad de “bona fide” de un audio, sino que también me dejara acceder a las capas internas de la arquitectura.

- Del estado del arte de la memoria, redacté la introducción y las conclusiones. Además, mejoré la versión base de modelos de explicación. Estas mejoras consistieron en mejorar la redacción de la sección y profundizar en los conceptos teóricos básicos de cada técnica para mejorar su comprensión, añadiendo, a su vez, explicaciones más visuales como fórmulas en algunos casos. Además, clasifiqué cada técnica según el alcance de la explicación, a qué conjuntos de modelos se pueden aplicar y en qué momentos aparece la interpretabilidad dentro del proceso de aprendizaje. Asimismo, redacté desde cero las subsecciones de Occlusion y RISE.
- He redactado todas las partes de la memoria relacionadas con AASIST, estas son las subsecciones del modelo en el estado del arte, implementación y explicación en metodología, y pipeline.
- He redactado la sección de trabajo futuro, analizando posibles mejoras e investigando sobre los dos tipos de explicadores explicados en esa sección.
- He redactado la introducción del capítulo de pipeline, explicando sus objetivos y todo lo necesario para poder usarlo. Además, diseñé el diagrama simplificado de la arquitectura del pipeline.
- He redactado el README del repositorio del proyecto.

Contribución de Rafael López Rodríguez

- Investigué sobre las motivaciones sociales y técnicas detrás de la creación de deepfakes, así como los problemas críticos que presenta la detección de audio clonado en la actualidad.
- Investigué sobre el estado del arte de los modelos de clasificación Regresión Logística y K-Nearest Neighbors (k-NN), profundizando en sus fundamentos matemáticos, sus hiperparámetros y su idoneidad para el manejo de características acústicas.
- Investigué exhaustiva sobre las técnicas de IA Explicable (XAI), analizando cuáles de ellas son aplicables específicamente al dominio del audio y la detección de fraude. Esto incluyó tanto técnicas post-hoc (que explican modelos ya entrenados) como técnicas ante-hoc (modelos interpretables por diseño).
- Estudié la viabilidad de aplicar arquitecturas intrínsecamente explicables como ProtoPNet y SincNet para la detección de audio, realizando pruebas para determinar si estas técnicas de explicabilidad ante-hoc aportarían valor frente a los métodos tradicionales.
- Desarrollé el código necesario para la implementación de los algoritmos de Regresión Logística y k-NN. Diseñé un flujo de trabajo que incluyó el ajuste fino de hiperparámetros y un análisis detallado de las métricas de rendimiento para optimizar la capacidad de detección.
- Apliqué técnicas de explicabilidad post-hoc a los modelos desarrollados, implementando SHAP (global y local), LIME (local) y DiCE (generación de explicaciones contrafactuales) para interpretar las decisiones del sistema.
- Redacté los apartados de Objetivos y Planificación dentro de la Introducción del proyecto.
- Redacté, en el bloque de Estado del Arte, las secciones correspondientes a los modelos de clasificación (Regresión Logística y k-NN) y a la totalidad de las técnicas de explicabilidad analizadas (SHAP, LIME, DiCE, Integrated Gradients, Grad-CAM, SincNet y ProtoPNet).

- Elaboré los apartados técnicos de la Metodología relativos a la implementación y el análisis de resultados de los modelos de Regresión Logística y k-NN.

Anexo

Tabla 1: Valores e interpretación de características acústicas del audio I_12_N

Característica	Valor	Interpretación
Energía		
RMS	0.01956	Esto nos indica que el audio tiene un volumen muy bajo, algo por debajo del rango típico de voz estándar pero coherente con voces sin procesar.
Silence ratio	0.76056	Esto nos dice que el 76 % del audio es silencio.
Skewness	-0.49481	Esto muestra que el audio tiene una ligera asimetría negativa.
ZCR	0.09770	Esto muestra que el audio es estable y no tiene demasiado ruido donde predominan las vocales sonoras.
Kurtosis	28.54873	Esto es una kurtosis extremadamente alta que encaja con voz intermitente o silencios largos junto con palabras puntuales.
Espectrales		
Spectral centroid	1636.52417	Presenta un centro de masa típico de voces habladas.
Spectral bandwidth	1590.95008	Esto indica una dispersión moderada del espectro lo que es típico de voces complejas.
Spectral rolloff	2916.26320	Este es un rolloff típico de voces con consonantes.
Spectral flatness	0.03462	Esto nos muestra que es una señal altamente tonal, es decir, sin ruido.
HNR	11.75470	Presenta un valor típico para las voces (entre 10 y 20).
Pitch		

Característica	Valor	Interpretación
Pitch std	55.34092	Este valor muestra la variabilidad del pitch siendo en este caso un valor de variabilidad alto.
Pitch mean	266.19288	Este es un valor alto para el pitch lo que denota voces agudas.
Formantes		
F1 mean	1001.88668	Este es un valor alto de F1, típico de las vocales abiertas (A, E, O).
F2 mean	2104.97462	Este es un valor típico de las vocales frontales o anteriores (E, I). Las vocales posteriores tendrían valores menores de 1200.
F3 mean	3271.62780	Este valor muestra una voz con un timbre bastante claro y resonante típico de voces femeninas o voces masculinas poco graves.
MFCCs		
MFCC 1 mean	-456.79852	Es un valor bastante bajo, típico de audios con pausas frecuentes.
MFCC 2 mean	45.54136	Esto indica que es una voz con más énfasis en fonemas graves y medios.
MFCC 3 mean	2.06269	Esto nos dice que el espectro es bastante equilibrado.
MFCC 4 mean	12.16472	Esto nos habla de la presencia de consonantes agudas.
MFCC 5 mean	-1.45521	Esto muestra que es una voz natural y sin exceso de brillo.
MFCC 6 mean	12.41849	Esto indica que la voz hablada es clara.
MFCC 7 mean	4.94370	Esto indica que la señal es estable y natural.
MFCC 8 mean	10.45284	Esto nos dice que la voz presenta consonantes articuladas.
MFCC 9 mean	8.34968	Esto indica que la voz es clara y con información armónica preservada.
MFCC 10 mean	3.03896	Esto muestra que es voz natural, es decir, sin ruido blanco.
MFCC 11 mean	10.38782	Esto refuerza la claridad de la voz.
MFCC 12 mean	12.44464	Esto nos habla de que es voz articulada.

Característica	Valor	Interpretación
MFCC 13 mean	10.57839	Esto nos dice que la voz no presenta distorsión.

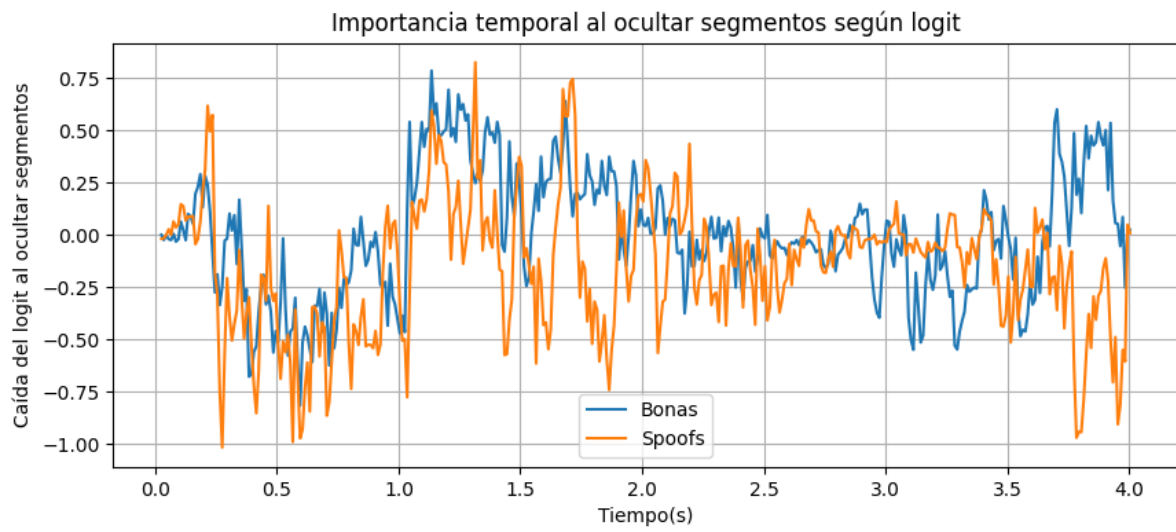


Figura 1: Resultados de “logit” de la ocultación temporal en AASIST.

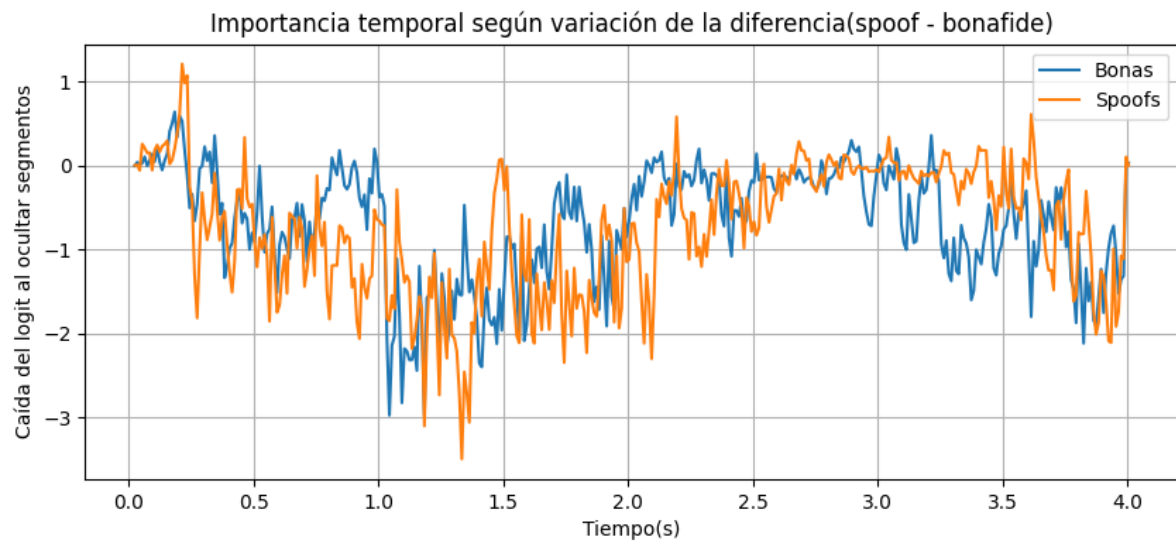


Figura 2: Resultados de margen de la ocultación temporal en AASIST.

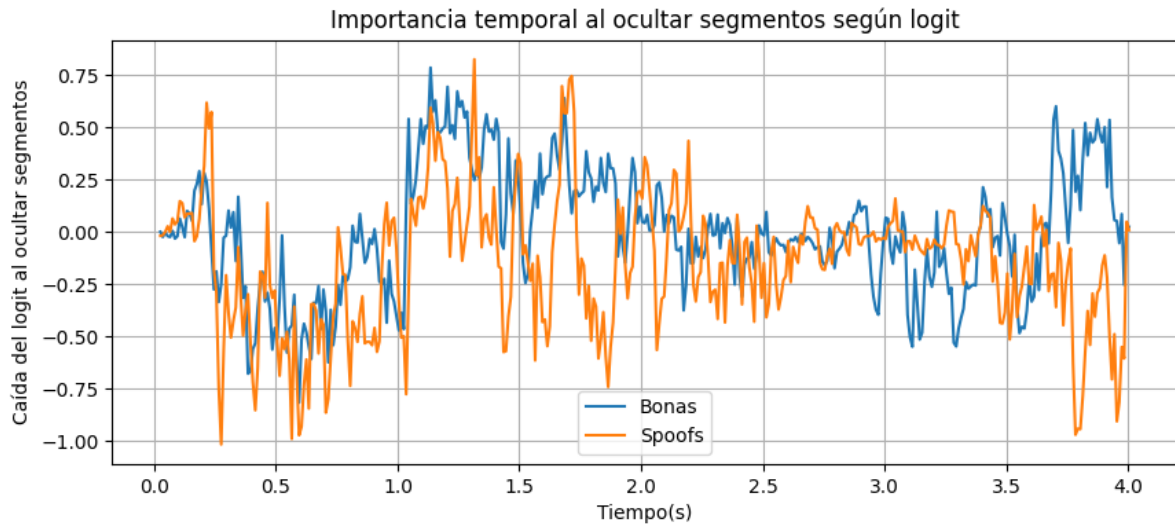


Figura 3: Resultados de “logit” de la ocultación frecuencial en AASIST.

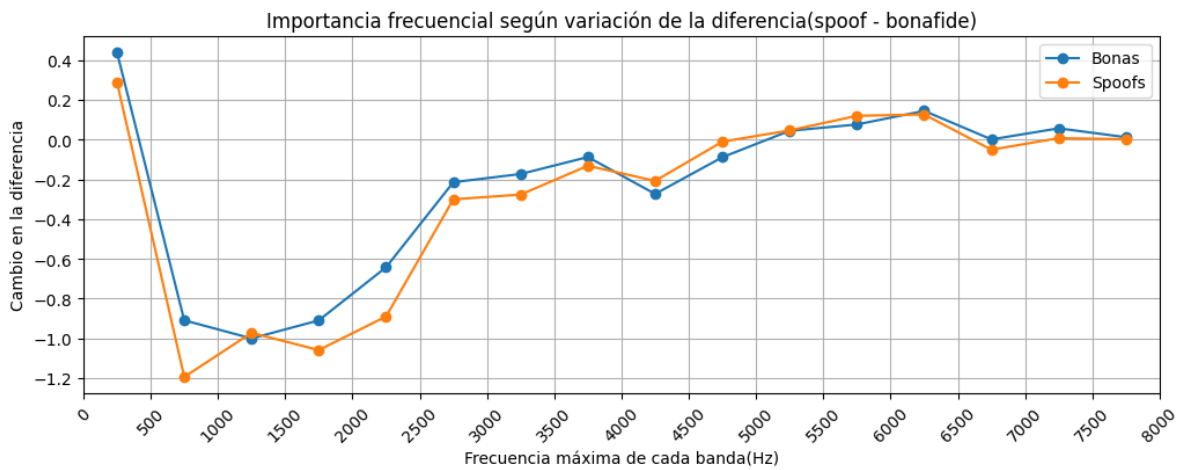
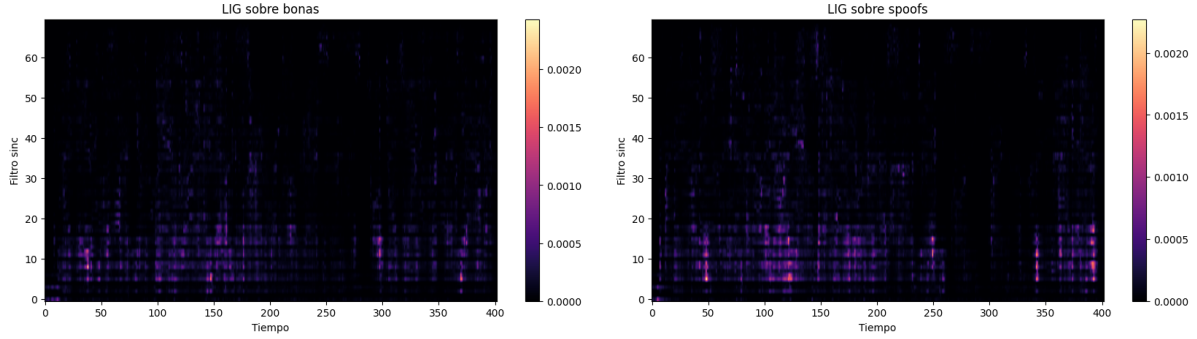
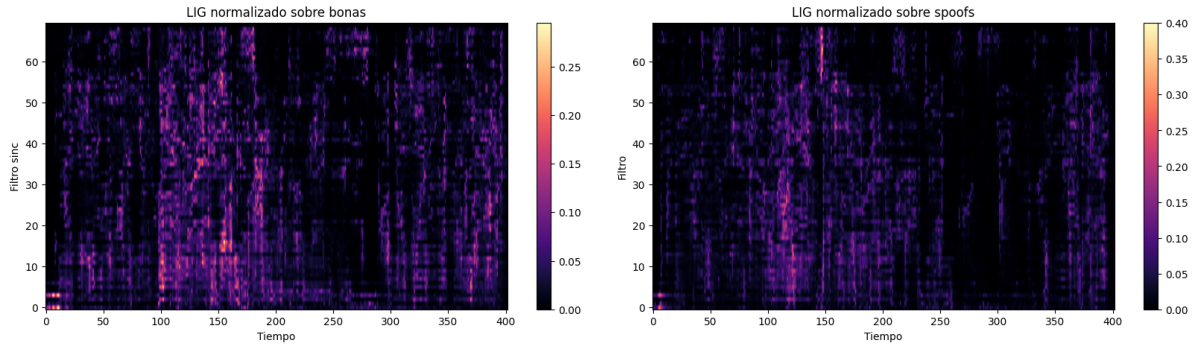


Figura 4: Resultados de margen de la ocultación frecuencial en AASIST.



(a) Mapa filtro-tiempo generado con IG para “bonas”. (b) Mapa filtro-tiempo generado con IG para “spoofs”.

Figura 5: Mapas filtro-tiempo de la capa sinc-convolution generados con IG.



(a) Mapa filtro-tiempo normalizado generado con IG para “bonas”. (b) Mapa filtro-tiempo normalizado generado con IG para “spoofs”.

Figura 6: Mapas filtro-tiempo de la capa sinc-convolution generados con IG normalizados.

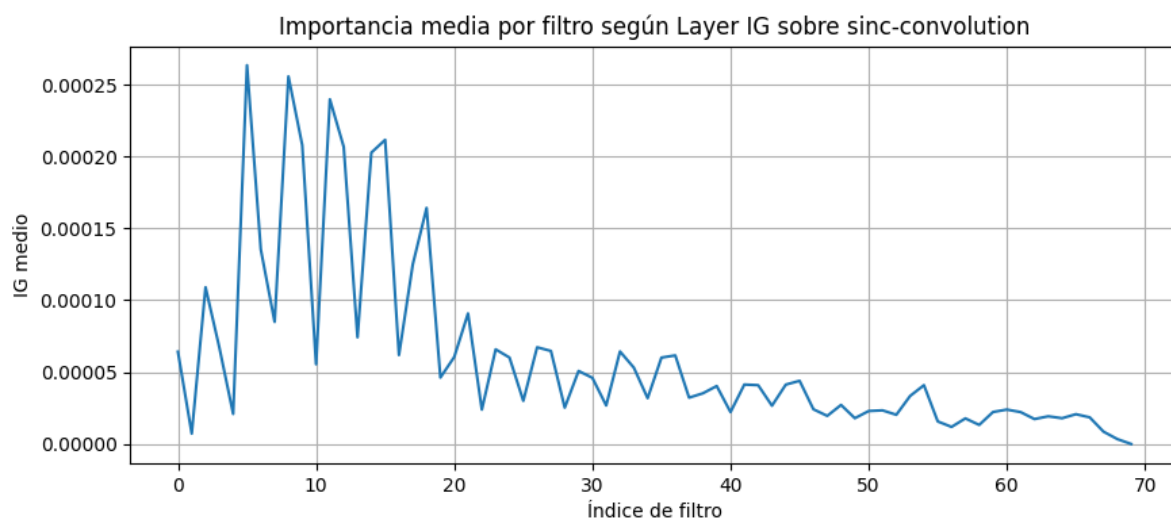


Figura 7: Gráfica filtro-tiempo de la capa sinc-convolution.

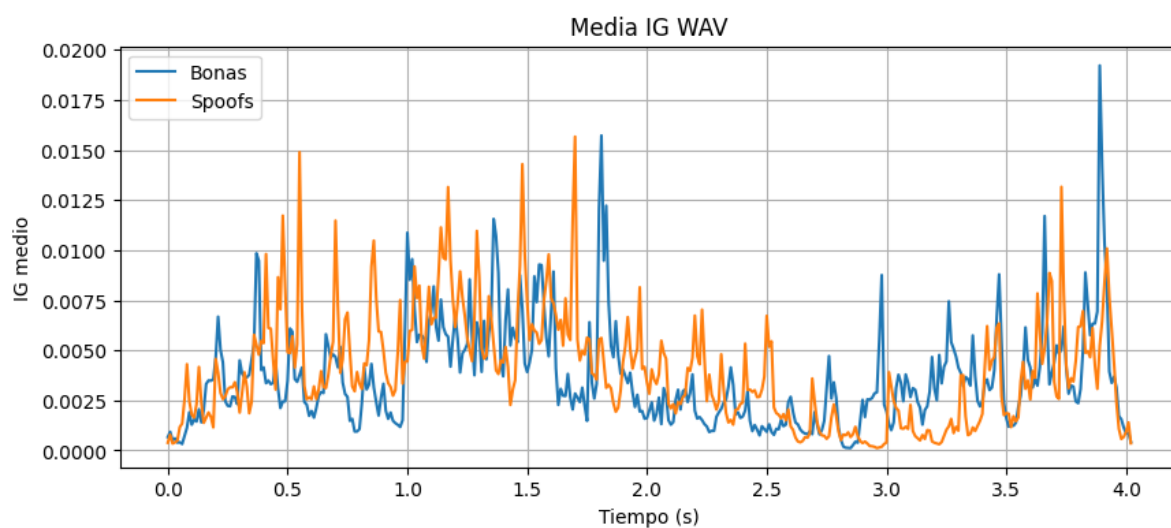
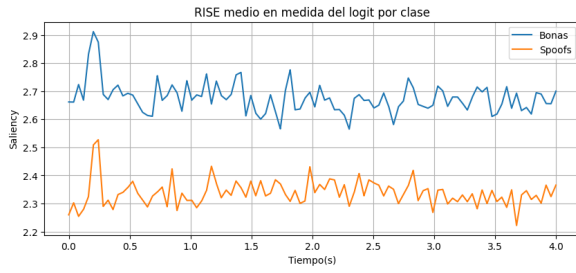
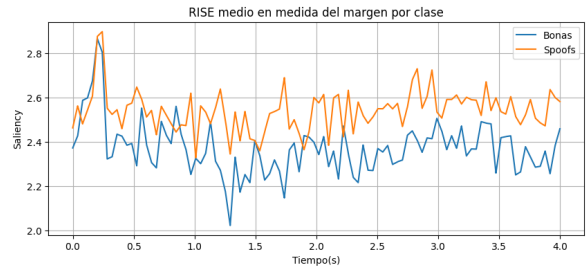


Figura 8: Importancia temporal al aplicar IG sobre onda de audio cruda.

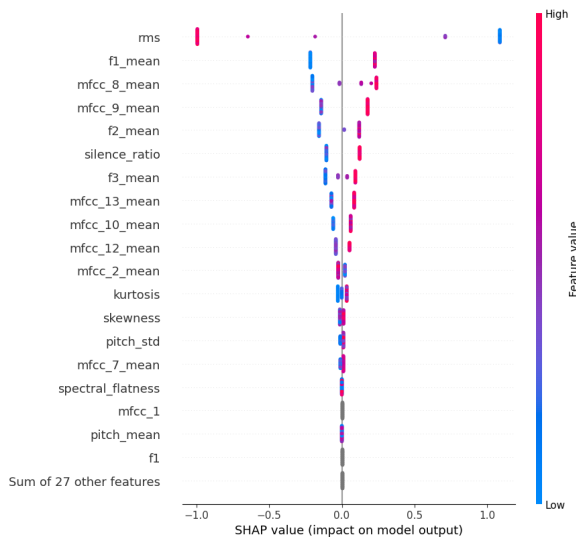


(a) Resultados de “logit” usando RISE.

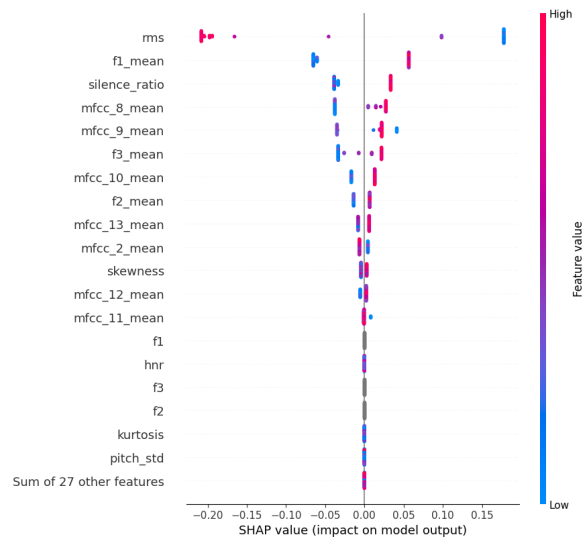


(b) Resultados de “margen” usando RISE.

Figura 9: Resultados de RISE.

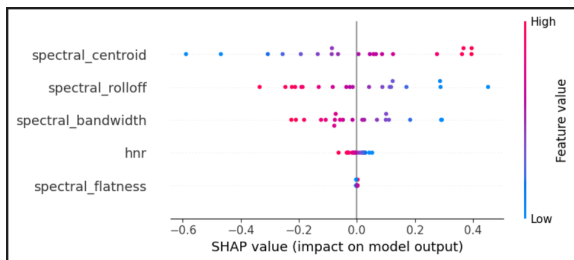


(a) Agrupación de las gráficas SHAP del clasificador.

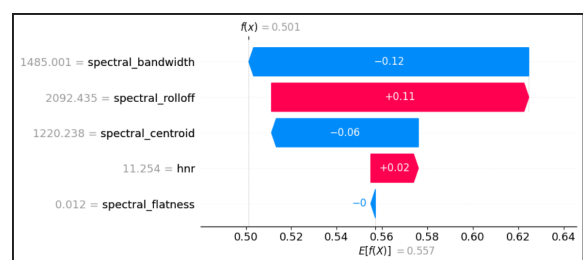


(b) Agrupación de las gráficas SHAP del regresor.

Figura 10: Gráficas de importancia global de SHAP para el clasificador y el regresor.

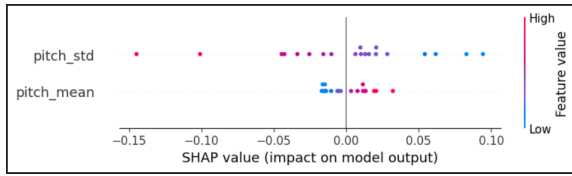


(a) Shap global espectrales

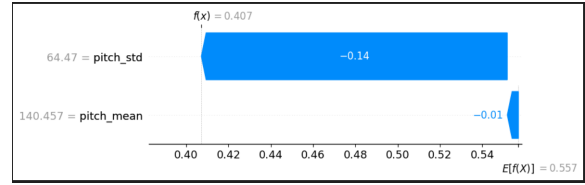


(b) Ejemplo concreto (I_{12_N})

Figura 11: Influencia de características espectrales en Wav2Vec2

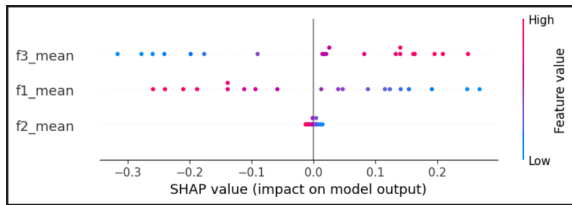


(a) Shap global pitch

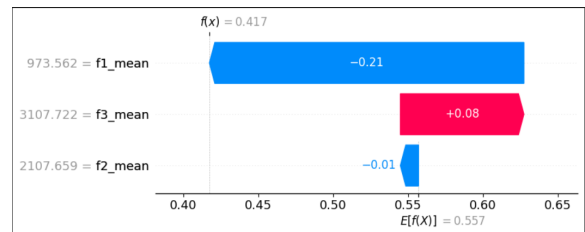


(b) Ejemplo concreto (I_12_N)

Figura 12: Influencia de características espectrales en Wav2Vec2

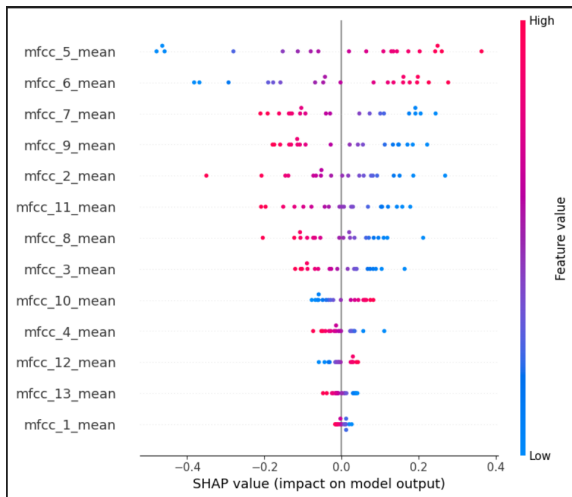


(a) Shap global formantes.

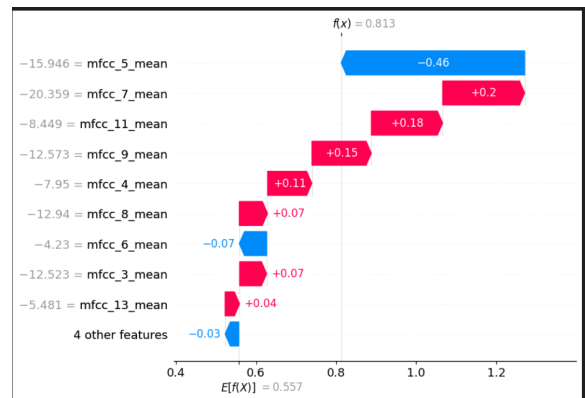


(b) Ejemplo concreto (I_12_N)

Figura 13: Influencia de características espectrales en Wav2Vec2



(a) Shap global MFCCs.



(b) Ejemplo concreto (I_12_N)

Figura 14: Influencia de características espectrales en Wav2Vec2

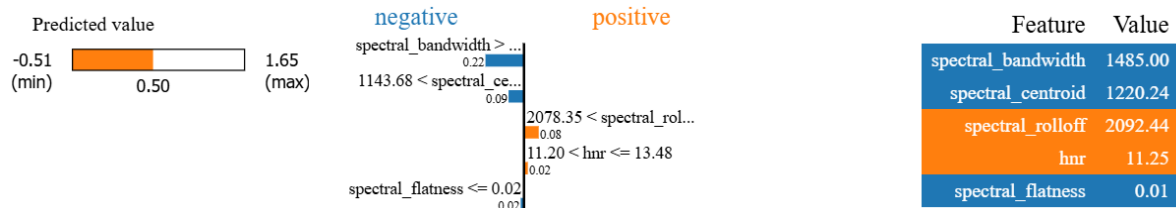


Figura 15: Explicación local espectrales Lime.



Figura 16: Explicación local pitch Lime.

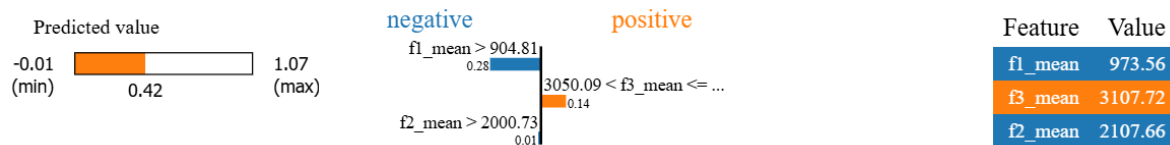


Figura 17: Explicación local formantes Lime.

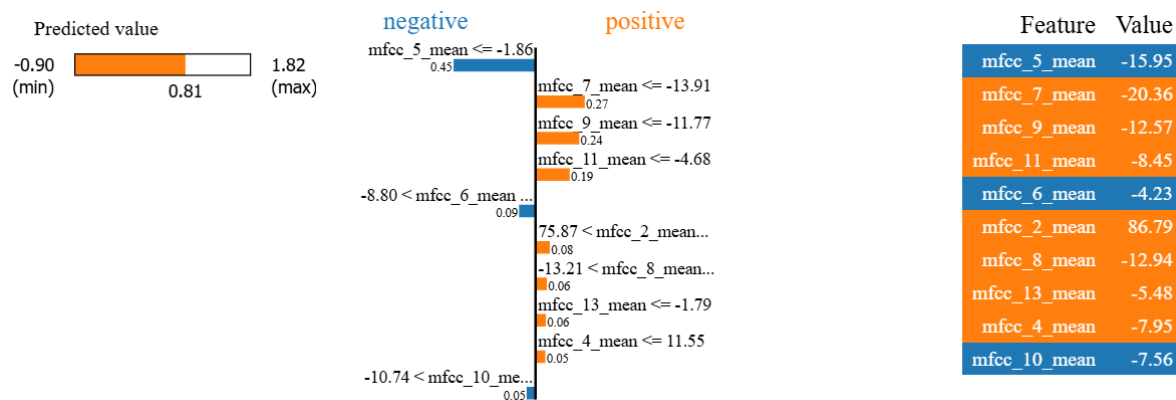


Figura 18: Explicación local mfccs Lime.